psoc_e84-edgi-talk SDK

开发文档

Edgi-Talk 开发板基于 PSoC™ E84 MCU，为工程师提供了一个灵活、全面的开发平台。板上集成了丰富的外设接口和示例模块，助力开发者快速完成AI+多传感器、显示和通信应用的开发。

Version 1.0.0 | 中文 | 2026年10月17日

www.rt-thread.org

Copyright (c) 2006-2025, RT-Thread Development Team

目录

- 1. 基础篇
  - 1.1. 简介
    - 1.1.1. 软件说明
    - 1.1.2. 使用方法
    - 1.1.3. 注意事项
    - 1.1.4. 启动流程
- 2. 驱动篇
  - 2.1. Edgi-Talk_M33_Driver_All 多功能 Demo 工程
  - 2.2. Edgi-Talk_M55_Driver_All 多功能 Demo 工程
  - 2.3. Edgi_Talk_M55_WIFI 示例工程
- 3. 示例
  - 3.1. Edgi-Talk_M55_LVGL 示例工程
  - 3.2. 小智示例工程
- 4. USB篇
  - 4.1. Edgi_Talk_Extend_Screen CherryUSB USB 拓展屏示例工程
  - 4.2. Edgi_Talk_M33_USB_RoleSwitch CherryUSB 示例工程
  - 4.3. Edgi_Talk_M55_USB_RoleSwitch CherryUSB 示例工程
  - 4.4. Edgi_Talk_M55_USB_UVC CherryUSB 示例工程
  - 4.5. Edgi_Talk_USB_Deepcraft 示例工程
- 5. IPC篇
  - 5.1. Edgi_Talk_M33_IPC 双核通信示例工程
  - 5.2. Edgi_Talk_M55_IPC 双核通信示例工程
- 6. AI篇
  - 6.1. Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision
- 7. 常见问题
  - 7.1. 常见问题解答（FAQ）
  - 7.2. PWM Device Configurator 配置指南

1. 基础篇

1.1. 简介

Edgi_Talk_M33_Template 示例工程

中文 | English

本模板工程基于 **Edgi-Talk 平台**，在 Cortex-M33 核上运行最小 **RT-Thread** 应用。
它用于完成板级初始化、启动 Cortex-M55 核，然后保持空闲；所有可选外设 demo 默认关闭。

共享的 Secure M33 固件包位于：`libraries/components/infineon-pse84-secure-firmware-latest`，同时保留 Template 自己的最小 `.config`。

1.1.1. 软件说明

- 工程基于 **Edgi-Talk** 平台开发。
- 已启用 `SOC_Enable_CM55`，板级初始化阶段会启动 M55 核。
- AHT20、LSM6DS3、Audio、ADC、RTC、SD 卡、文件系统、LCD、Wi-Fi 等可选外设 demo 均保持关闭。
- 板级初始化会将外部 Wi-Fi/音频电源控制脚拉低，避免 Template 默认打开外设电源。

1.1.2. 使用方法

编译与下载

1. 打开工程并完成编译。
2. 使用 **板载下载器 (DAP)** 将开发板的 USB 接口连接至 PC。
3. 通过编程工具将生成的固件烧录至开发板。

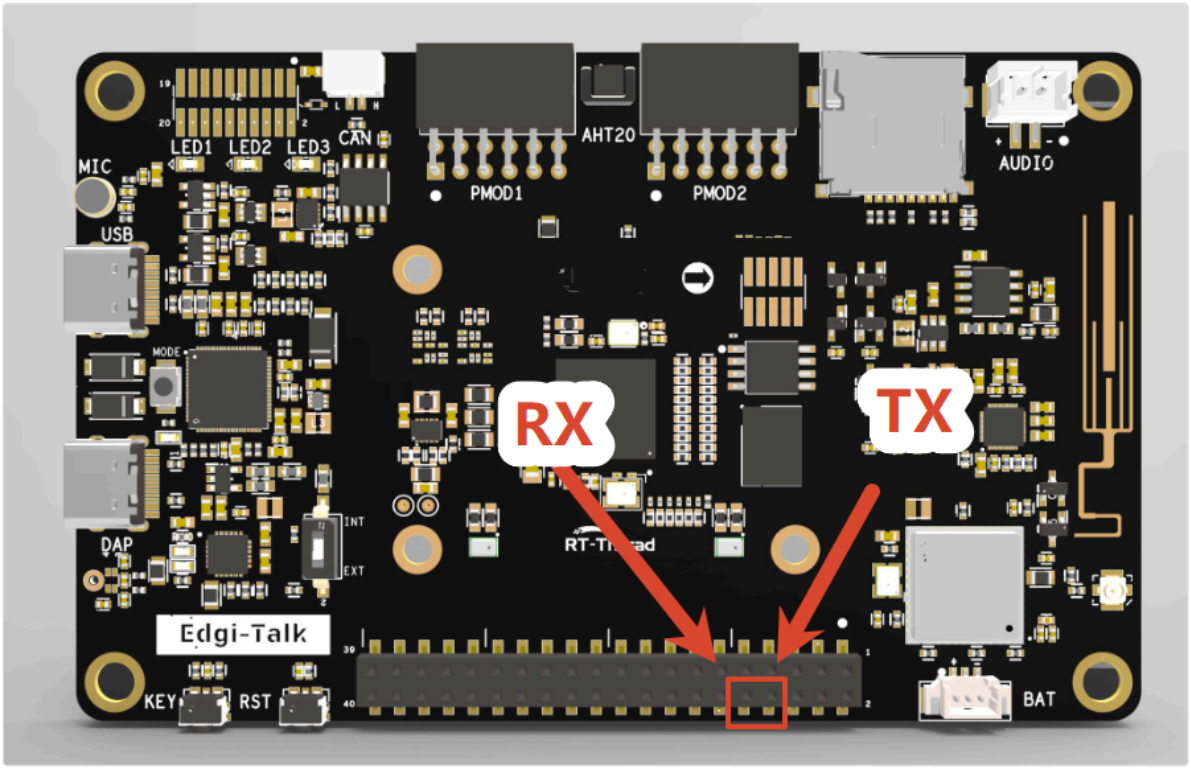
运行效果

- 烧录完成后，开发板上电即可运行示例工程。
- M33 初始化 RT-Thread，启动 M55，打印简短启动信息，然后保持空闲。
- Template 不会自动闪灯，也不会自动启动外设 demo。

1.1.3. 注意事项

> ⚠️ ***注意：*** *本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- M33 工程的串口日志不会通过板载 DAP 虚拟串口直接输出。查看 `msh />` 和 demo 日志时，需要额外连接 USB 转串口硬件，例如 CH340。
  位置如图下所示，RX 接串口硬件的 TX，TX 接串口硬件的 RX，上位机波特率 115200：



- 如需修改工程的 **图形化配置**，请使用以下工具打开配置文件：

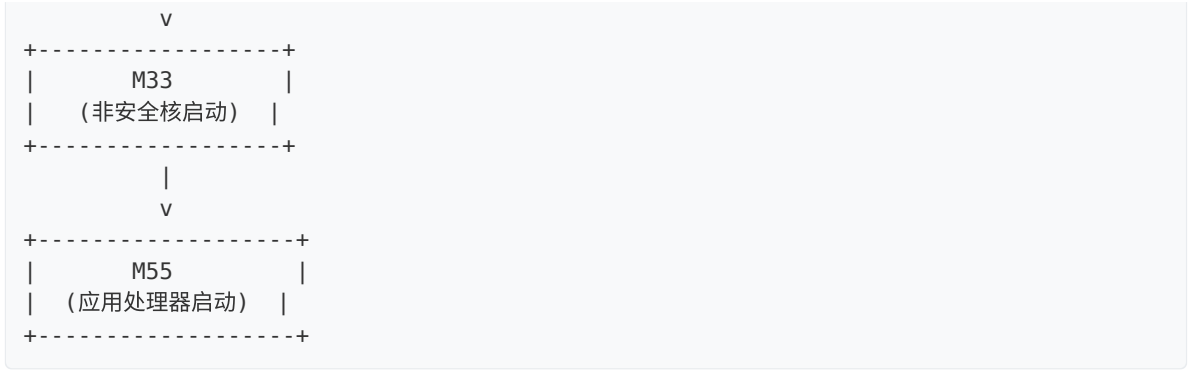
```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- 修改完成后保存配置，并重新生成代码。
- 如需运行 M55 工程，建议先烧写 **Edgi_Talk_M33_Template**。该工程是干净的 M33 工程，适合作为 M55 启动前的基础固件。

1.1.4. 启动流程

系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
```



⚠️ 请严格按照以上顺序烧写固件，否则系统可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- 若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```

2. 驱动篇

2.1. Edgi-Talk_M33_Driver_All 多功能 Demo 工程

中文 | **English**

简介

本工程基于 **Edgi-Talk 平台**，运行于 **RT-Thread 实时操作系统 (M33 核)**，集中集成多个常用外设 demo，便于在同一个固件中验证传感器、ADC、HyperRAM、按键中断、RTC、SD 卡文件系统、片外 Flash 文件系统和 WAV 音频播放功能。

当前工程包含以下 MSH 命令：

命令	功能
`demo_aht20`	读取 AHT10/AHT20 温湿度
`demo_lsm6ds3 [count] [delay_ms]`	读取 LSM6DS3 六轴传感器数据
`demo_audio start/stop/status [speaker_volume] [mic_gain]`	音频录音到播放实时回环
`wavplay -s <file>`	播放 WAV 音频文件
`demo_adc`	读取 ADC1 Channel 1 电压
`demo_hyperram`	HyperRAM 基础读写校验
`demo_hyperram_speed [size_kb] [loops]`	HyperRAM 带宽测试
`demo_key`	注册 P8.3 按键中断，按下后翻转蓝色 LED
`demo_rtc [YYYY MM DD HH MM SS]`	读取或设置 RTC 时间
`demo_sdcard`	SD 卡文件写入和读取测试
`demo_sdcard_speed [total_kb] [block_kb]`	SD 卡顺序读写测速
`demo_flash_speed [total_kb] [block_kb]`	`/flash` 文件系统顺序读写测速
`core_mark`	CoreMark CPU 性能基准测试

软件说明

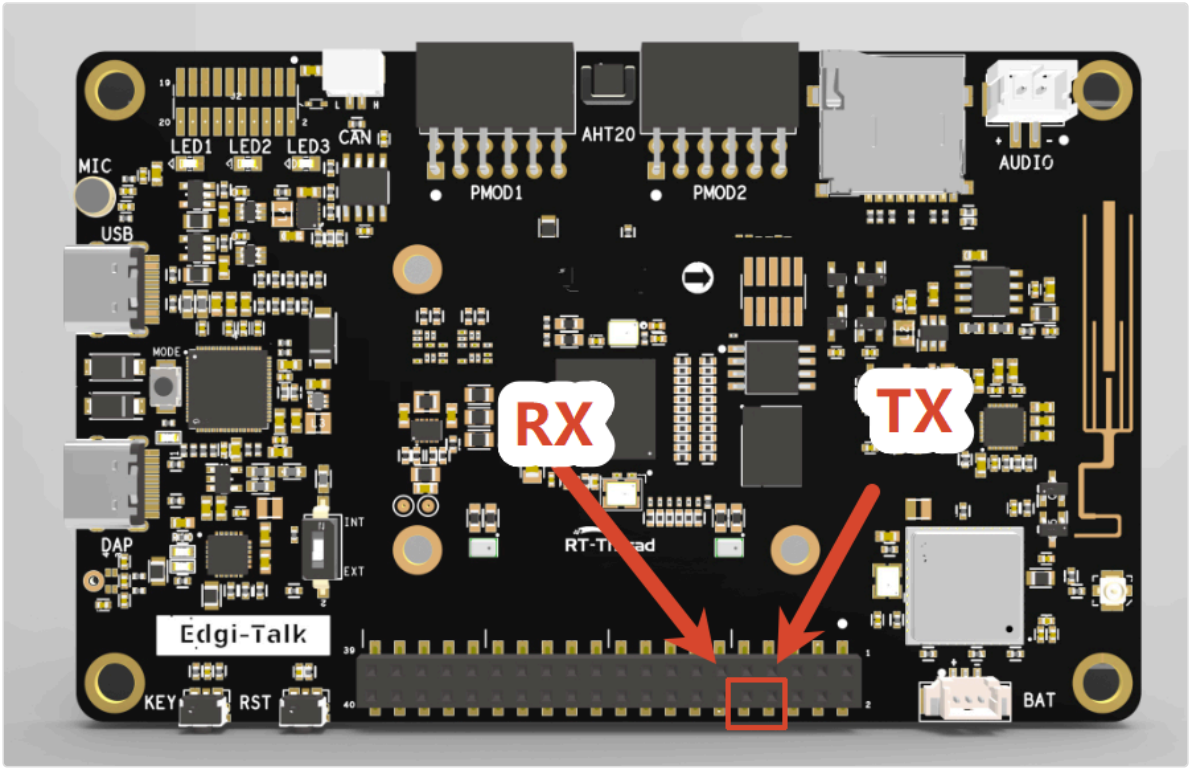
- 工程基于 **Edgi-Talk** 平台开发。

- 使用 **RT-Thread** 作为操作系统内核。
- 各 demo 通过 MSH 命令手动触发，避免多个外设测试上电后同时运行。
- Audio demo 使用 RT-Thread Audio 框架、PDM 麦克风和 ES8388 Codec；启动后会占用 `mic0` 与 `sound0`。
- WavPlayer demo 使用 RT-Thread `wavplayer` 软件包播放 WAV 文件，播放时会占用 `sound0`，不能和 `demo_audio` 回环同时运行。
- CoreMark 使用 RT-Thread `CoreMark` 软件包，默认迭代次数为 `36000`，通过 MSH 命令手动运行。
- SD 卡挂载由公共文件系统初始化逻辑处理，插卡后自动挂载，拔卡后自动卸载。
- 片外 Flash 通过 FAL 注册为 `norflash0`，其中 `filesystem` 分区会以 littlefs 挂载到 `/flash`。
- 普通 SDIO `CMD5 arg=0` 超时探测日志已静默处理，避免正常 SD 卡识别过程中反复刷屏。

注意事项

> ***注意：** 本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- M33 工程的串口日志不会通过板载 DAP 虚拟串口直接输出。查看 `msh />` 和 demo 日志时，需要额外连接 USB 转串口硬件，例如 CH340。
  位置如图下所示,RX接串口硬件的TX,TX接串口硬件的RX,上位机波特率115200:



- 修改完成后保存配置，并重新生成代码。
- SD 卡建议格式化为 FAT16/FAT32。
- 如果 SD 卡测速结果偏低，优先检查卡本身速度等级、文件系统碎片、SDIO 时钟、总线位宽和读写块大小。

- `/flash` 使用片外 Flash 的 `filesystem` 分区，首次挂载失败时会自动格式化；如果需要保留该分区数据，请不要随意执行整片擦除。
- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本工程。
- 如需运行 M55 工程，建议先烧写 **Edgi_Talk_M33_Template**。该工程是干净的 M33 工程，适合作为 M55 启动前的基础固件。

AHT20 温湿度 Demo

功能说明

`demo_aht20` 使用 AHT10/AHT20 软件包，通过 `i2c1` 读取一次温度和湿度，并通过串口打印结果。设备句柄会缓存，重复执行命令时不会重复初始化传感器。

使用方法

```
demo_aht20
```

示例输出：

```
AHT10: temp 26.3 C, humidity 48.7 %
```

LSM6DS3 六轴传感器 Demo

功能说明

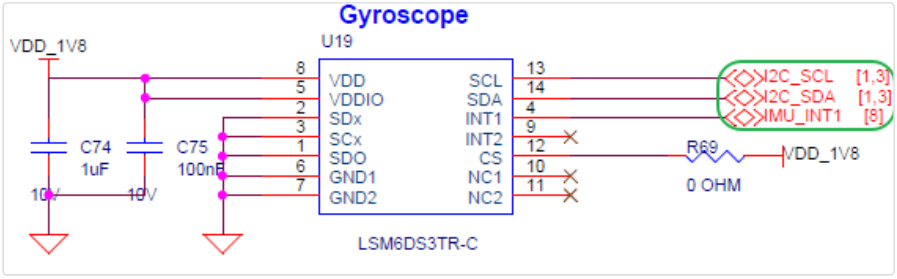
本 demo 移植自 `Edgi_Talk_M33_LSM6DS3` 示例工程，用于演示 **LSM6DS3TR-C 六轴惯性测量单元（IMU）** 的使用方法。LSM6DS3TR-C 集成三轴加速度计、三轴陀螺仪和温度传感器，支持 I2C/SPI 接口，常用于姿态检测、运动识别、手势识别和可穿戴设备。

当前命令会：

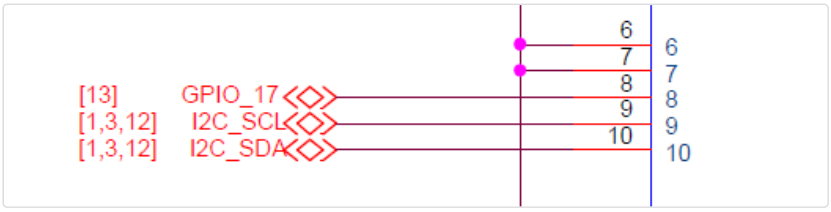
- 通过 `i2c0` 检测 LSM6DS3 设备 ID。
- 复位传感器并配置输出速率、加速度量程和陀螺仪量程。
- 轮询读取三轴加速度、三轴角速度和温度。
- 通过串口打印数据；不会在上电后自动循环打印。

硬件说明

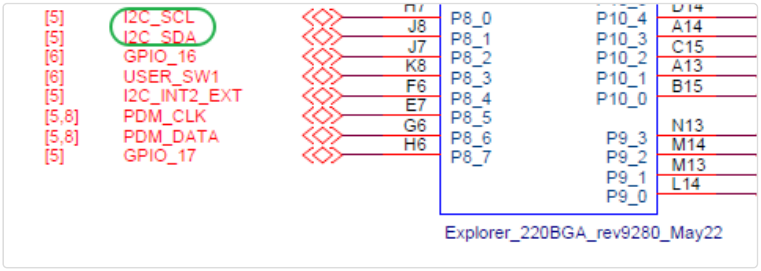
LSM6DS3TR-C 接口



BTB 座子



MCU 引脚



使用方法

```
demo_lsm6ds3
demo_lsm6ds3 10 200
```

参数说明：

- `count`：采样次数，默认 5；设置为 0 时持续采样。
- `delay_ms`：每次采样间隔，默认 500 ms。

示例输出：

```
LSM6DS3: found on i2c0 addr 0x6A
LSM6DS3 sample: count=5, delay=500 ms
Acceleration [mg]: 15.23  -3.12  1000.45
Angular rate [mdps]: 2.50  -1.25  0.75
Temperature [degC]: 26.54
```

Audio 录放 Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_Audio` 示例工程，用于演示 **PDM 麦克风采集 + ES8388 Codec 播放** 的实时音频回环。Audio 设备由音频总线接口、控制总线接口、Codec、扬声器和麦克风组成，RT-Thread Audio 框架负责音频设备注册、打开关闭、读写、音量调节和流控制。

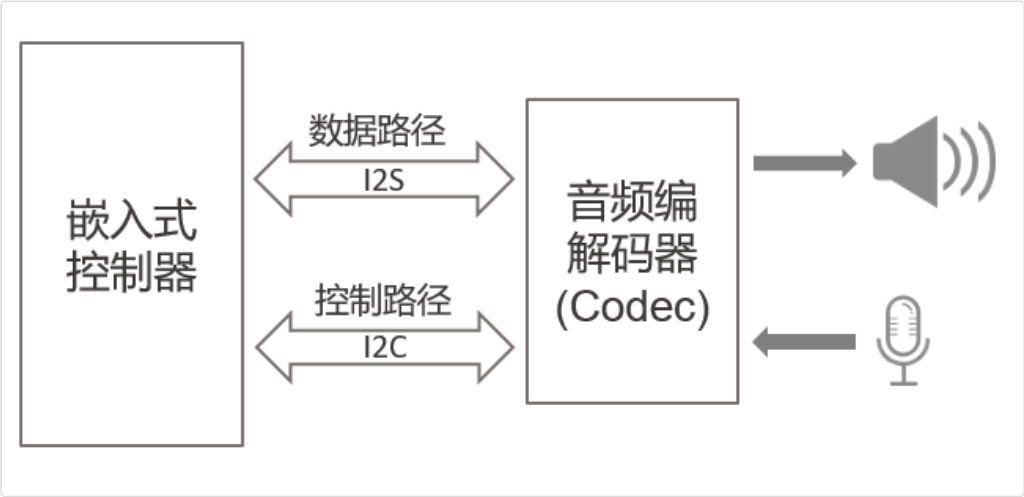
当前命令会：

- 打开 `mic0` 录音设备和 `sound0` 播放设备。
- 显式配置音频格式为 16 kHz、单声道、16 bit，和 PDM 麦克风输出保持一致。
- 设置扬声器音量和麦克风增益。
- 创建后台线程，持续读取麦克风数据并写入扬声器。
- 通过 MSH 命令手动启动/停止，不会在上电后自动进入回环。

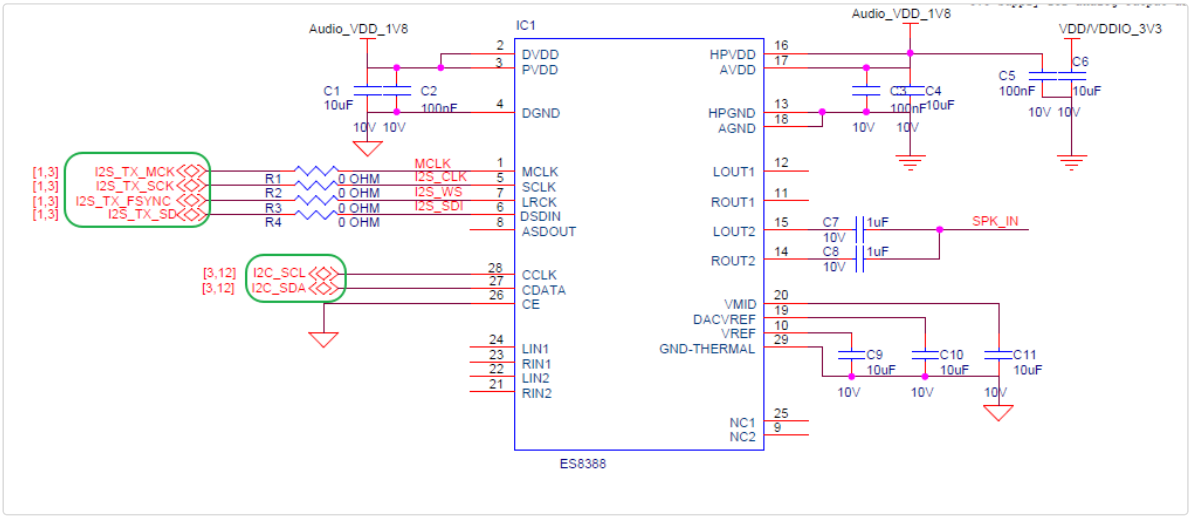
> ***注意：*** *原 Audio 工程使用 P8.3 按键切换播放状态。当前多 demo 工程中 P8.3 已用于 `demo_key`，因此 Audio demo 改为 MSH 命令控制，避免两个 demo 同时注册同一个按键中断。*

硬件说明

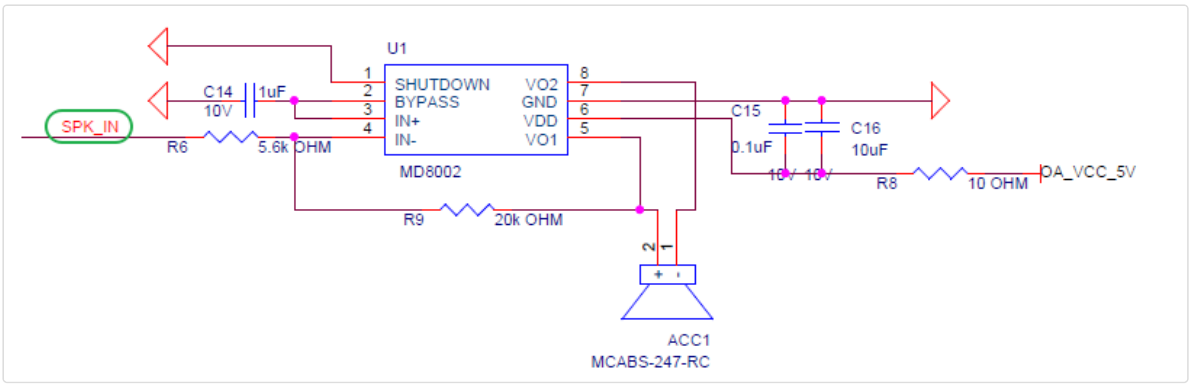
嵌入式音频系统组成



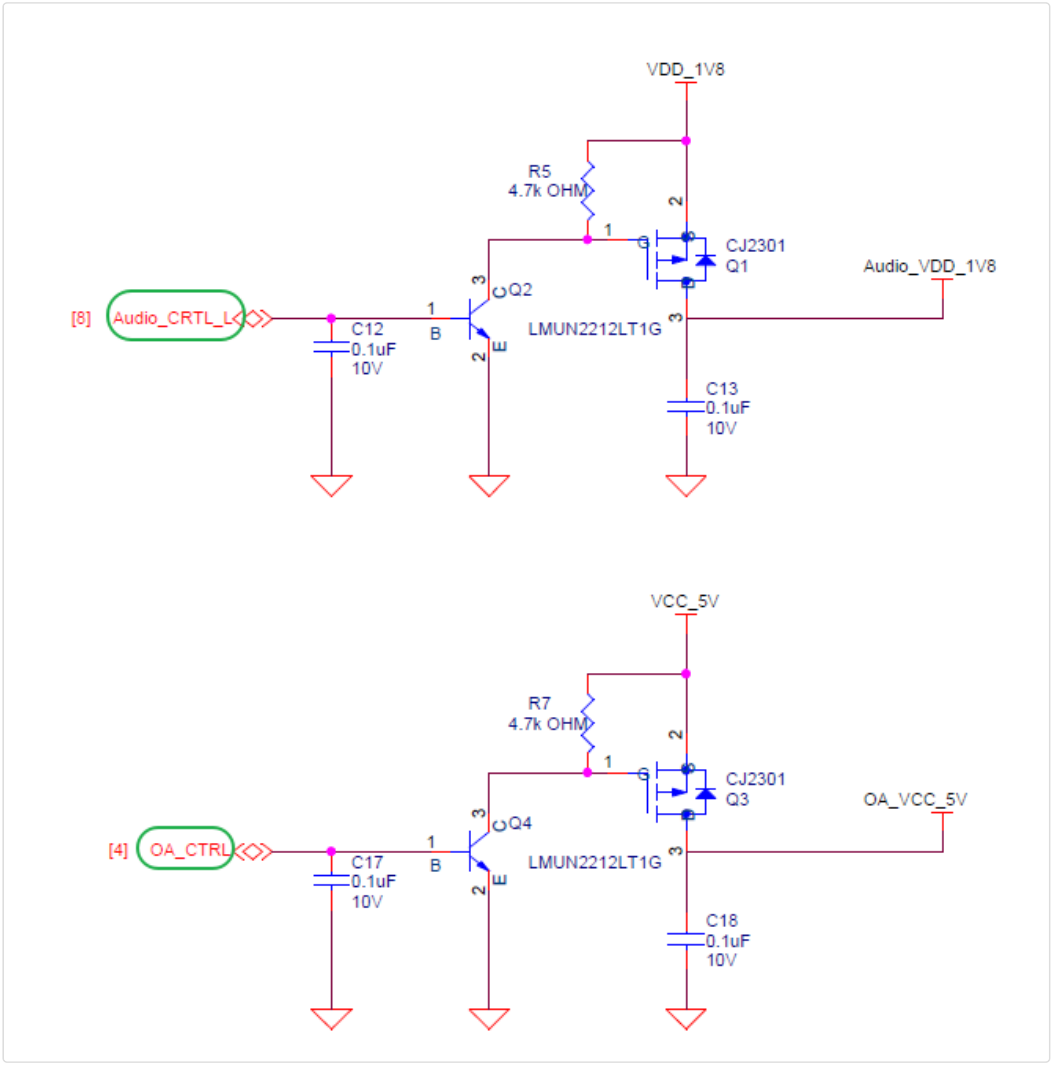
ES8388 连接接口


Audio_VDD_1V8
IC1
Audio_VDD_1V8
VDD/VDDIO_3V3
DVDD
PVDD
DGND
HPVDD
AVDD
HPGND
AGND
LOUT1
ROUT1
LOUT2
ROUT2
VMID
DACVREF
VREF
GND-THERMAL
NC1
NC2
MCLK
SCLK
LRCK
DSDIN
ASDOUT
CCLK
CDATA
CE
LIN1
RIN1
LIN2
RIN2
I2S_TX_MCK
I2S_TX_SCK
I2S_TX_FSYNC
I2S_TX_SDI
I2C_SCL
I2C_SDA
SPK_IN
ES8388

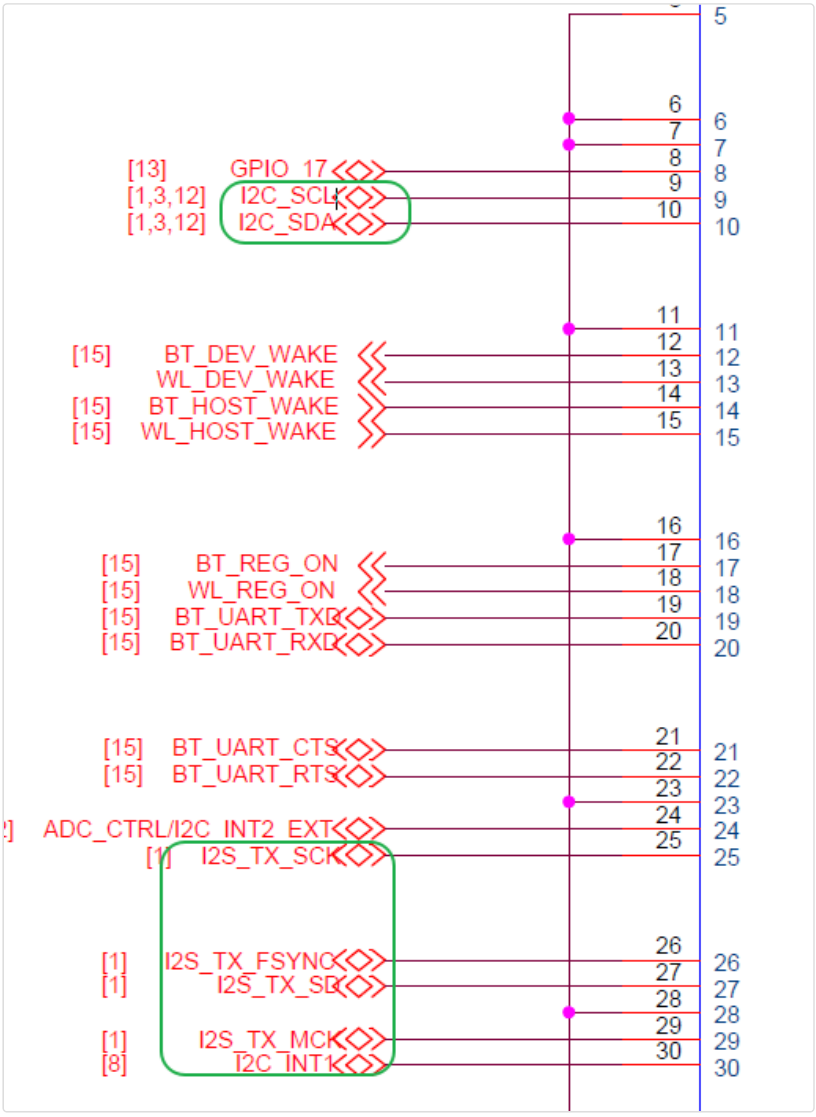
喇叭接口


U1
SHUTDOWN
BYPASS
IN+
IN-
VO2
GND
VDD
VO1
MD8002
SPK_IN
R6
5.6k OHM
R9
20k OHM
R8
10 OHM
DA_VCC_5V
ACC1
MCABS-247-RC

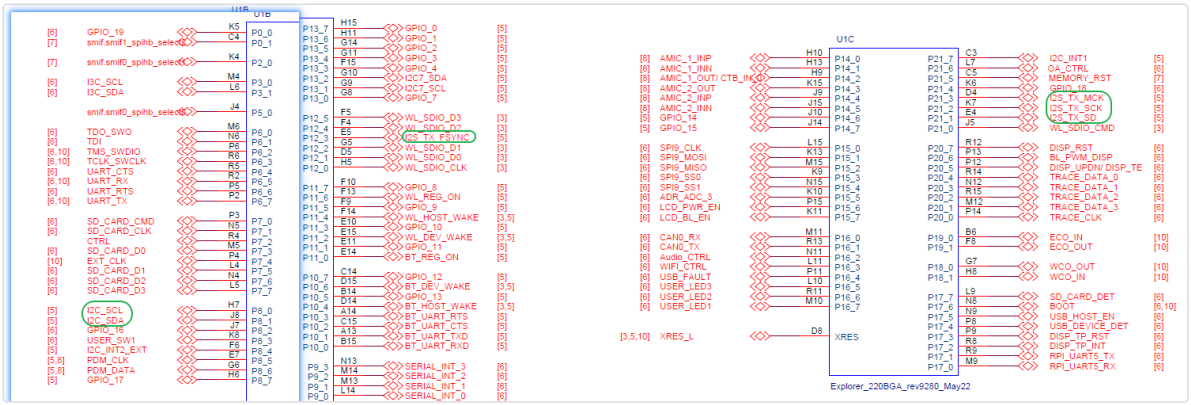
控制引脚


VDD_1V8
R5
4.7k OHM
CJ2301
Q1
Audio_VDD_1V8
[8]
Audio_CRTL_L
C12
0.1uF
10V
Q2
LMUN2212LT1G
C13
0.1uF
10V
VCC_5V
R7
4.7k OHM
CJ2301
Q3
OA_VCC_5V
[4]
OA_CTRL
C17
0.1uF
10V
Q4
LMUN2212LT1G
C18
0.1uF
10V

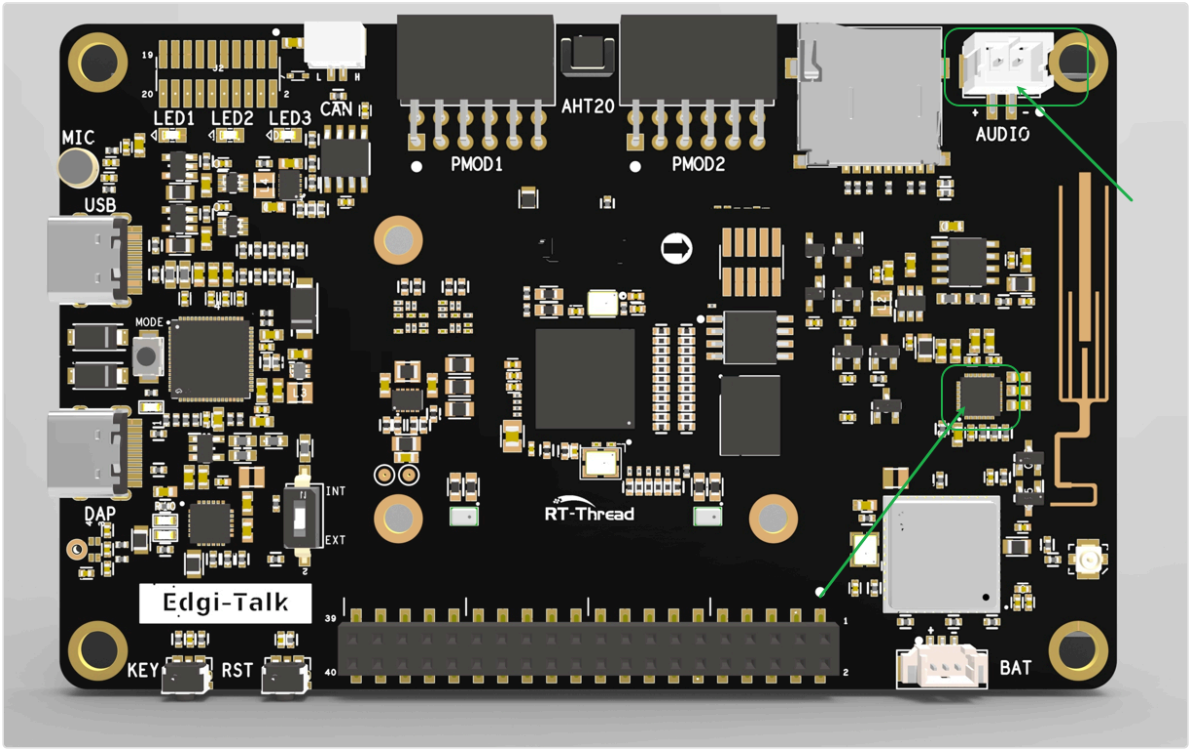
BTB 座子


[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
[15] BT_UART_CTS
[15] BT_UART_RTS
ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT1

MCU 引脚



实物图位置



使用方法

```
demo_audio start
demo_audio start 60 30
demo_audio status
demo_audio stop
```

参数说明：

- `speaker_volume`：扬声器音量，默认 60。
- `mic_gain`：麦克风增益，默认 30。

示例输出：

```
Audio loopback started: speaker=60, mic_gain=30
Audio loopback: running, speaker=60, mic_gain=30
Stopping audio loopback...
Audio loopback stopped
```

WavPlayer 音频播放 Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_WavPlayer` 示例工程，用于演示 **RT-Thread wavplayer 软件包 + Audio 设备驱动 + 文件系统** 的 WAV 文件播放流程。`wavplayer` 会读取 WAV 文件头，按文件中的采样率、声道数和 16 bit 位宽配置 `sound0`，再通过 ES8388 Codec 输出到扬声器。

当前工程启用了：

- `wavplay` 播放命令。
- `optparse` 命令行参数解析软件包。

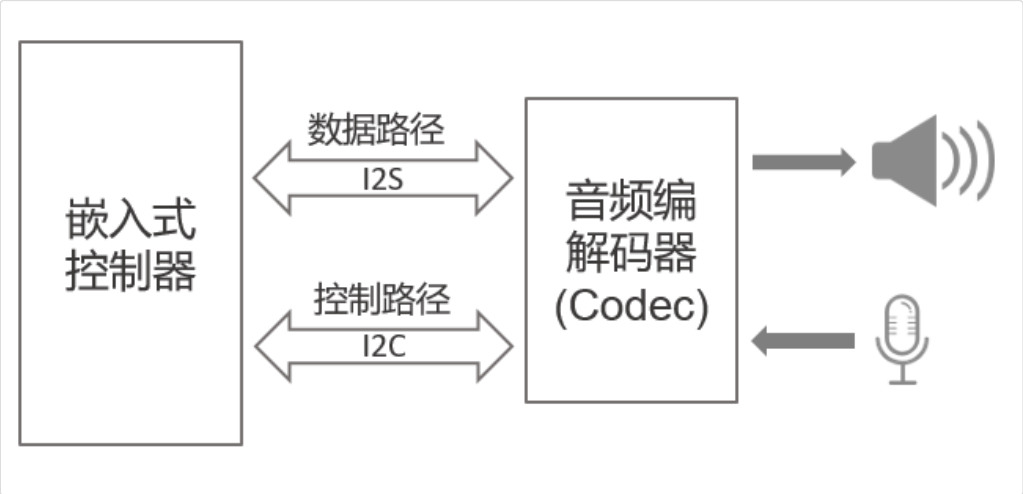
- 播放设备 `sound0`。

当前工程未启用 `wavrecord` 录音命令；录音功能仍建议使用 `demo_audio` 回环验证，避免和多 demo 工程里的音频资源管理互相干扰。

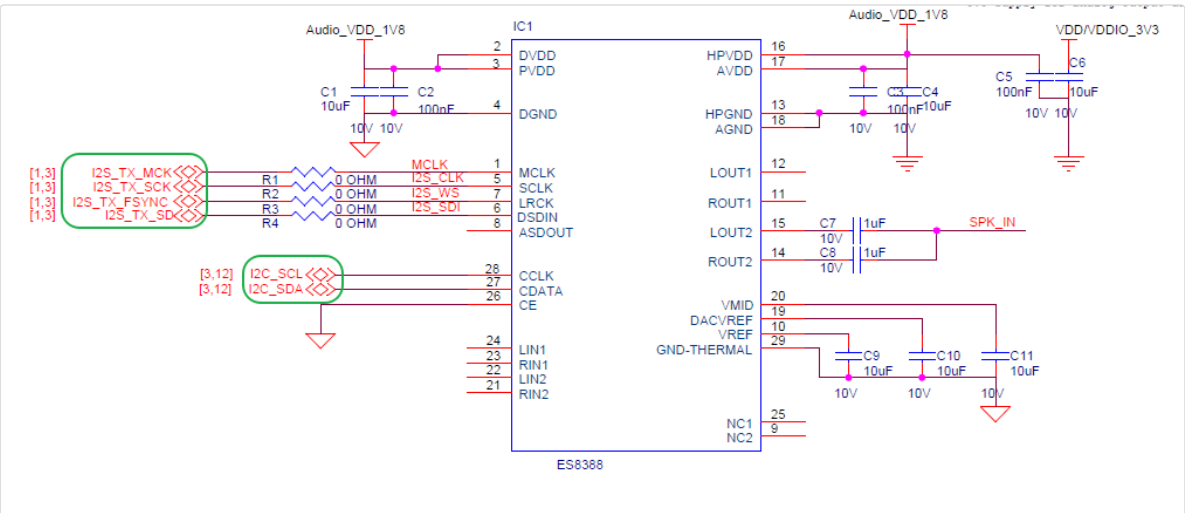
> ***注意：*** *`wavplay` 和 `demo_audio` 都会占用 `sound0`。播放 WAV 前请先执行 `demo_audio stop`，确认回环 demo 已停止。*

硬件说明

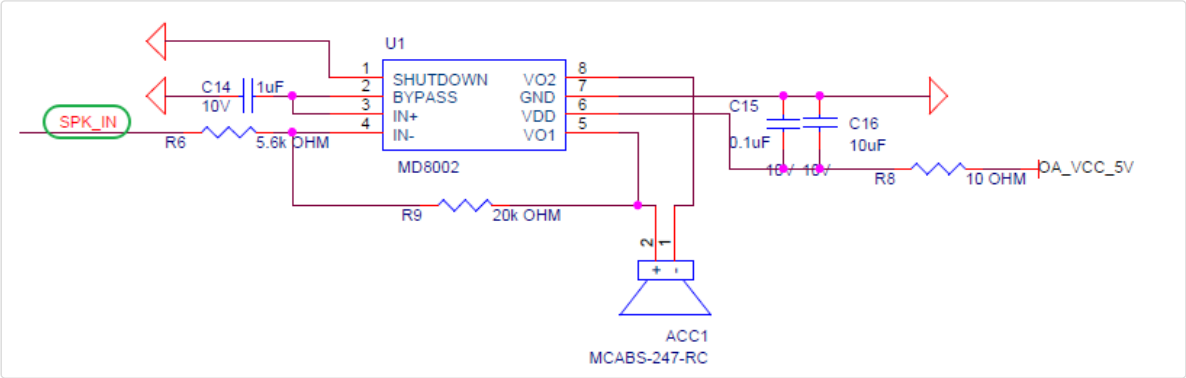
音频系统组成



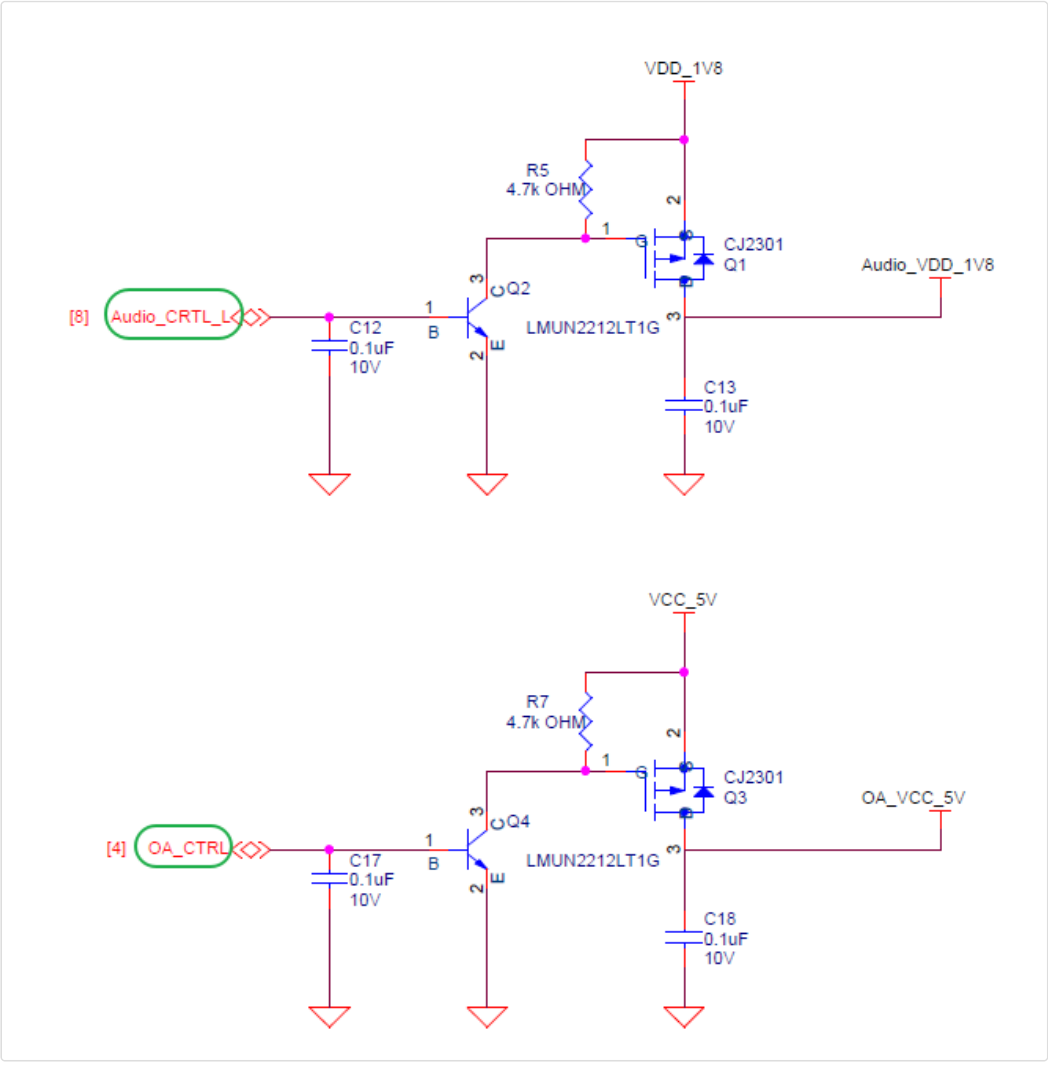
ES8388 连接接口



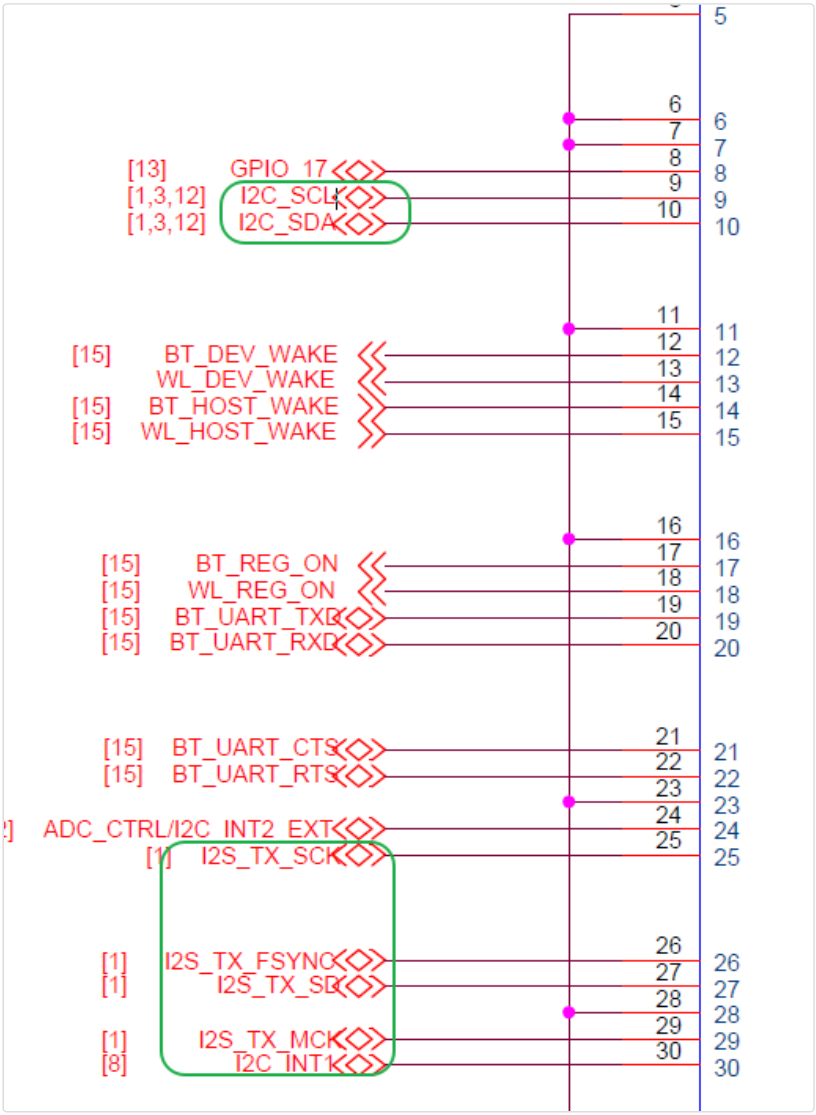
喇叭接口


U1
SHUTDOWN
BYPASS
IN+
IN-
VO2
GND
VDD
VO1
MD8002
SPK_IN
C14
1uF
10V
R6
5.6k OHM
R9
20k OHM
C15
0.1uF
C16
10uF
10V
10V
R8
10 OHM
OA_VCC_5V
ACC1
MCABS-247-RC

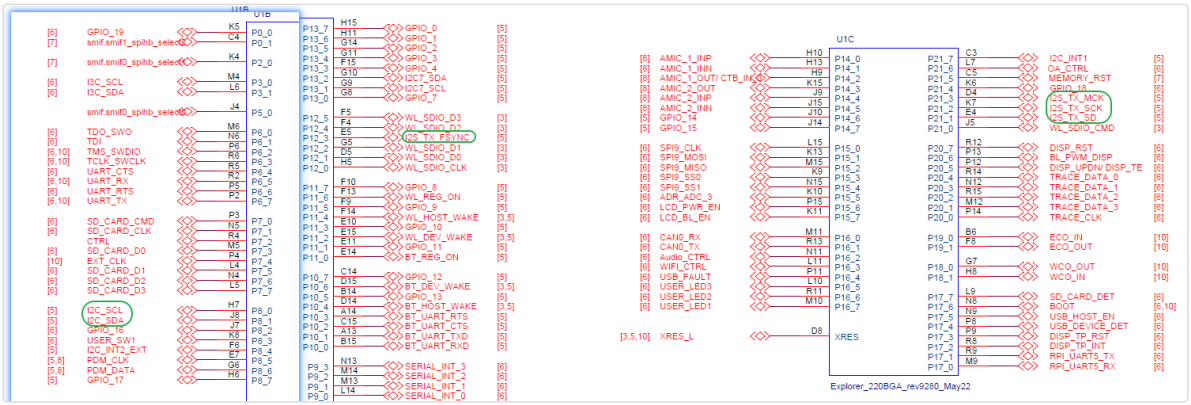
控制引脚


VDD_1V8
R5
4.7k OHM
CJ2301
Q1
Audio_VDD_1V8
[8]
Audio_CRTL_L
C12
0.1uF
10V
Q2
LMUN2212LT1G
C13
0.1uF
10V
VCC_5V
R7
4.7k OHM
CJ2301
Q3
OA_VCC_5V
[4]
OA_CTRL
C17
0.1uF
10V
Q4
LMUN2212LT1G
C18
0.1uF
10V

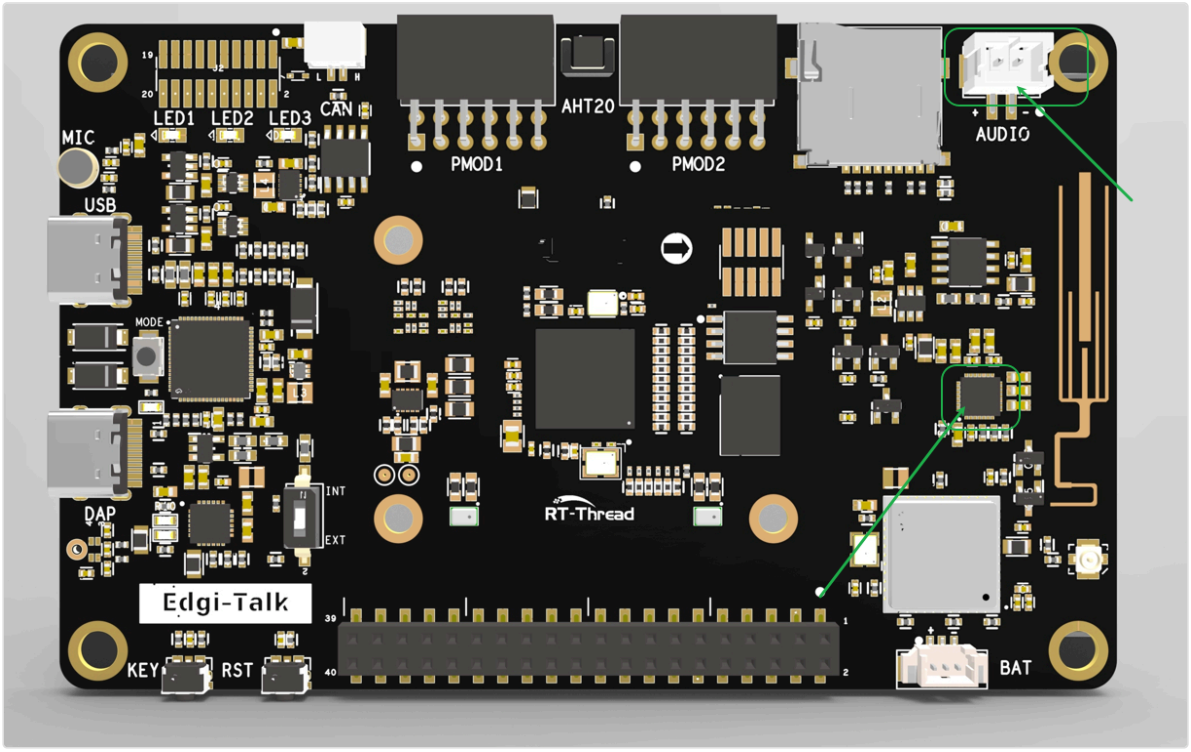
BTB 座子


[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
[15] BT_UART_CTS
[15] BT_UART_RTS
ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT1

MCU 引脚


U1B
U1C
Explorer_220BGA_rev9280_May22

实物图位置



使用方法

将 WAV 文件放到已挂载的文件系统中，例如 SD 卡挂载到 `/sdcard` 后：

```
cd /sdcard
wavplay -s 16000.wav
```

也可以直接使用完整路径：

```
wavplay -s /sdcard/16000.wav
```

常用命令：

```
wavplay -h
wavplay -s /sdcard/16000.wav
wavplay -t
wavplay -p
wavplay -r
wavplay -v 80
```

参数说明：

- `-s <file>`：开始播放指定 WAV 文件。
- `-t`：停止播放。
- `-p`：暂停播放。
- `-r`：恢复播放。
- `-v <0-100>`：设置播放音量。

示例输出：

```
msh /sdcard>wavplay -s 16000.wav
```

```
[I/WAV_PLAYER] play start, uri=16000.wav
```

CoreMark 性能测试 Demo

功能说明

本 demo 启用了 RT-Thread `CoreMark` 软件包，用于进行 CPU 核心性能基准测试。当前配置参考 `Edgi_Talk_M55_CoreMark` 工程，默认迭代次数为 `36000`，未启用浮点版本。

使用方法

```
core_mark
```

运行后会打印 CoreMark 参数、耗时、校验结果和最终分数。测试时建议停止其他高负载 demo，避免 SD 卡、音频或传感器轮询影响结果。

ADC Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_ADC` 示例工程，用于演示 **ADC（模数转换器）** 的使用方法。ADC 会将连续的模拟电压转换为数字值，供 MCU 后续处理。

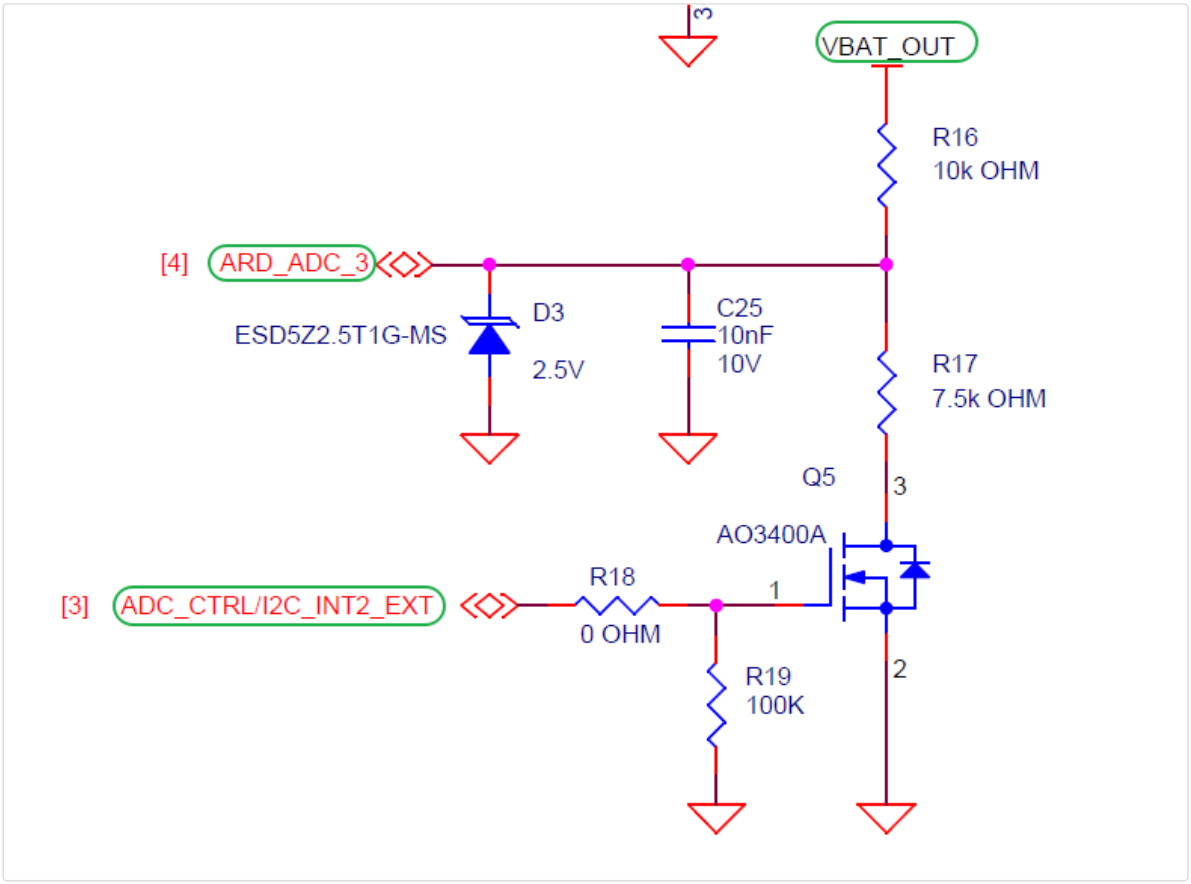
> ***注意：*** *当前硬件连接中，ADC 仅用于采集电池电压；树莓派接口及其他外部接口未接入该 ADC 通道，不能通过本命令采集。*

当前命令会：

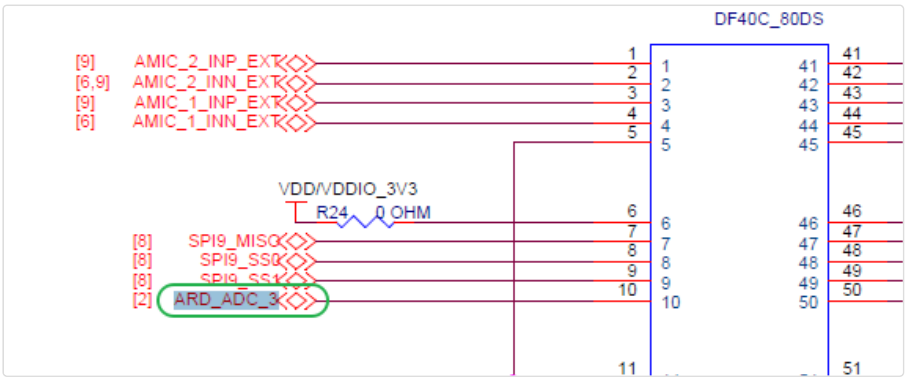
- 使能 ADC1 Channel 1。
- 首次执行时拉高 P8.4 ADC 电源控制引脚。
- 读取一次 ADC 原始值并换算为电压。

硬件说明

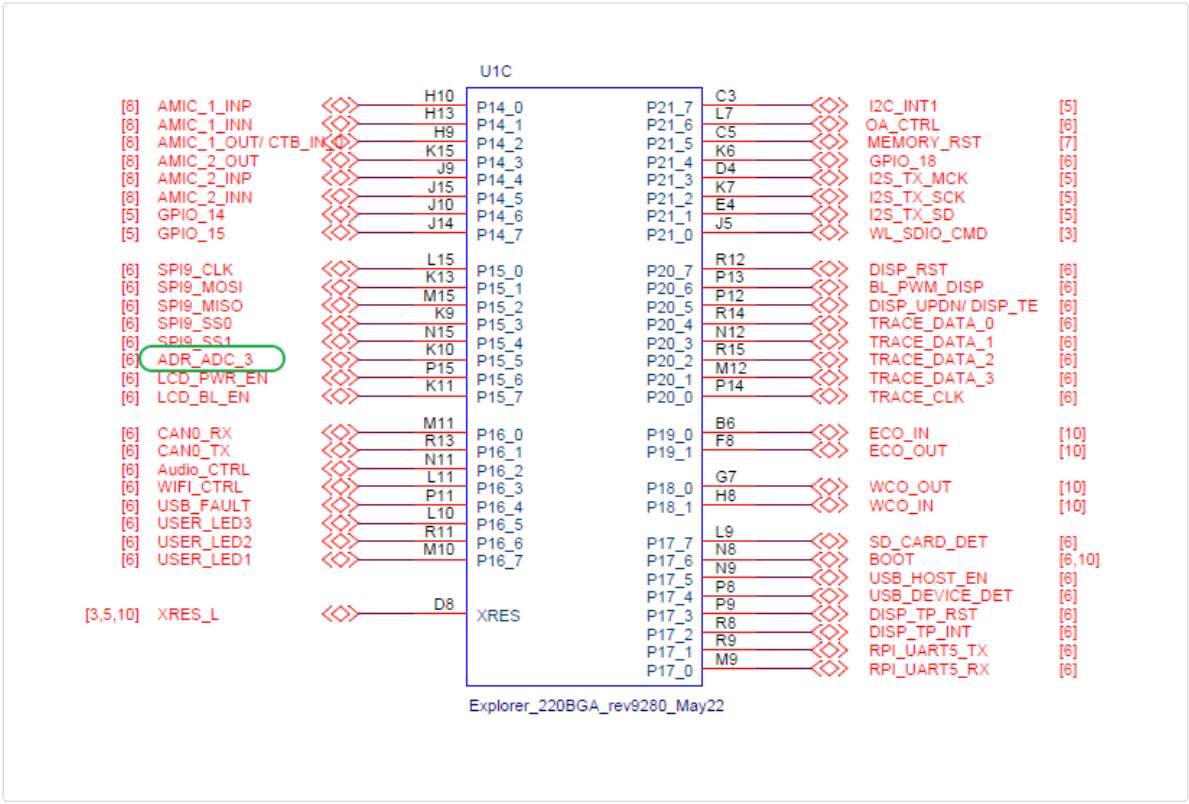
连接接口


3
VBAT_OUT
R16
10k OHM
[4] ARD_ADC_3
D3
ESD5Z2.5T1G-MS
2.5V
C25
10nF
10V
R17
7.5k OHM
Q5
3
AO3400A
[3] ADC_CTRL/I2C_INT2_EXT
R18
0 OHM
1
2
R19
100K

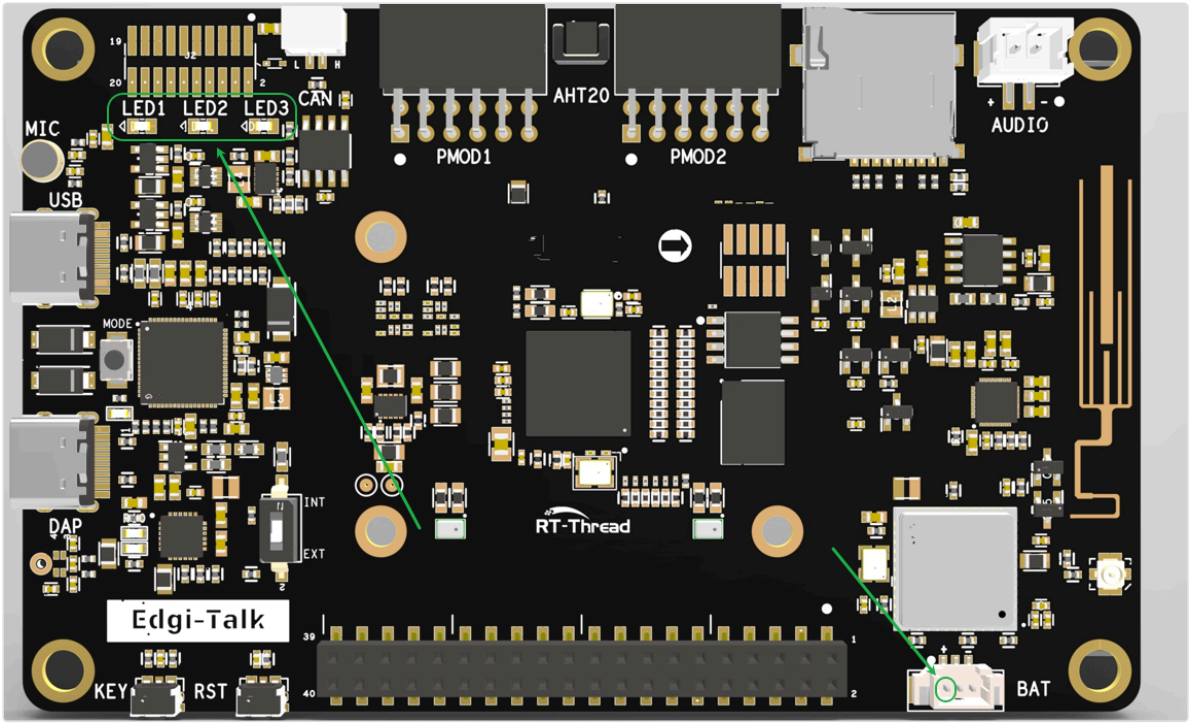
BTB 座子


DF40C_80DS
[9] AMIC_2_INP_EXT
[6,9] AMIC_2_INN_EXT
[9] AMIC_1_INP_EXT
[6] AMIC_1_INN_EXT
VDD/VDDIO_3V3
R24 0 OHM
[8] SPI9_MISO
[8] SPI9_SS0
[8] SPI9_SS1
[2] ARD_ADC_3

MCU 引脚



实物图位置



使用方法

```
demo_adc
```

示例输出：

```
CH1: 3.123 V (raw=1340)
```

HyperRAM Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_HyperRam` 示例工程，用于验证 **HyperRAM** 映射、基础读写和带宽性能。HyperRAM 驱动初始化后，内存区域会映射到系统地址空间，应用可用于大块缓存或外部堆。

基础读写校验

```
demo_hyperram
```

该命令会申请一段 HyperRAM 空间，写入测试数据并读回校验。

带宽测试

```
demo_hyperram_speed
demo_hyperram_speed 1024 8
```

参数说明：

- `size_kb`：单次测试数据大小，默认值由 demo 内部设置。
- `loops`：循环次数，数值越大，统计结果越稳定。

测试内容包括顺序写、顺序读和内存拷贝速度。

按键中断 Demo

功能说明

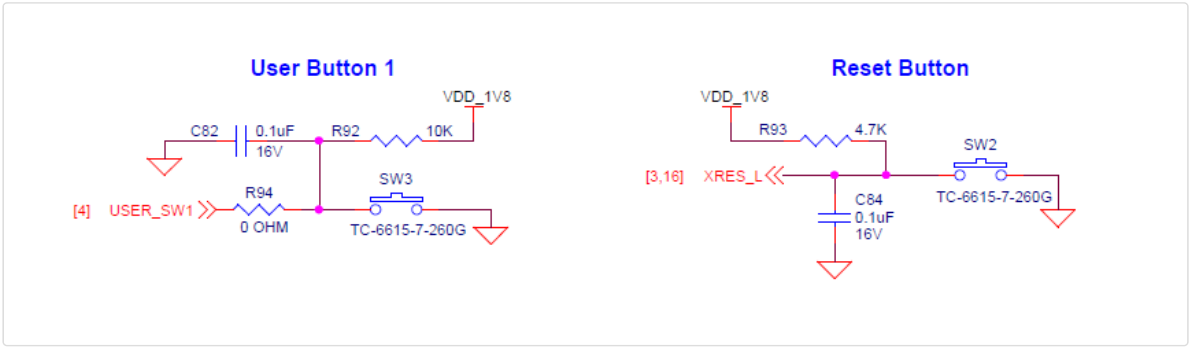
本 demo 移植自 `Edgi_Talk_M33_Key_Irq` 示例工程，用于演示 RT-Thread PIN 驱动的 GPIO 中断用法。

当前命令会：

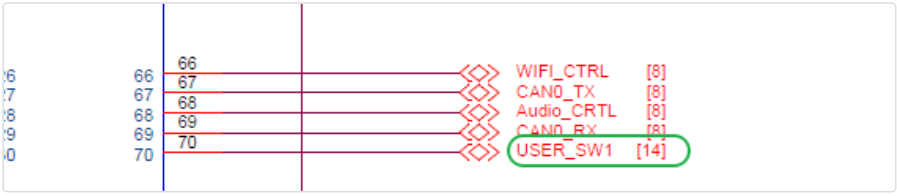
- 配置 P8.3 为上拉输入。
- 注册下降沿中断回调。
- 每次按键触发后翻转 P16.5 蓝色 LED，并在串口打印状态。

硬件说明

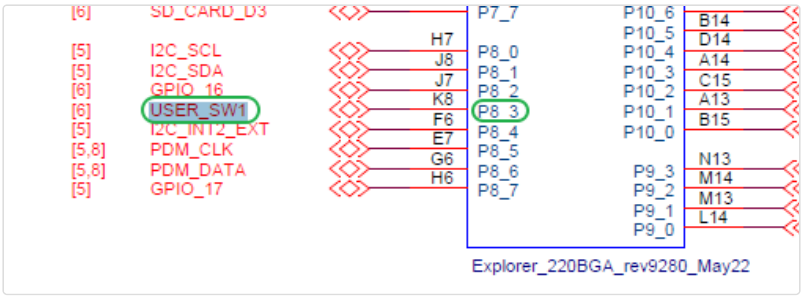
按钮接口



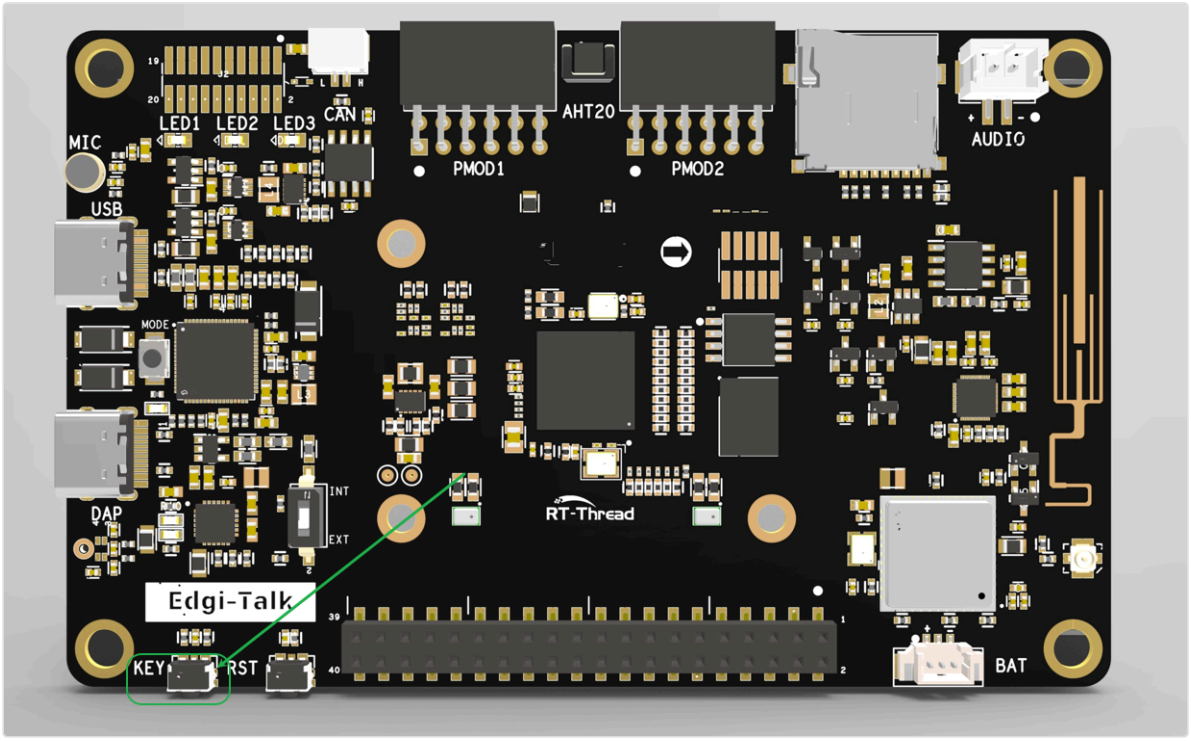
BTB 座子



MCU 接口



实物图位置



使用方法

```
demo_key
```

示例输出：

```
Key IRQ ready. Press the button on P8.3.
button pressed (led ON)
button pressed (led OFF)
```

RTC Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_RTC` 示例工程，用于演示 **RTC（实时时钟）** 的读取和设置。RTC 可用于系统时间、日志时间戳、定时任务和低功耗唤醒等场景。

使用方法

读取当前时间：

```
demo_rtc
```

设置时间后读取：

```
demo_rtc 2026 7 17 10 30 0
```

示例输出：

```
Fri Jul 17 10:30:00 2026
```

SD 卡 Demo

功能说明

本 demo 移植自 `Edgi_Talk_M33_SDCARD` 示例工程，用于演示 SD 卡挂载、文件写入、文件读取和读写测速。SD 卡是一种非易失性存储设备，常用于日志记录、配置文件保存、音视频缓存等场景。

当前工程已加入热插拔处理：

- 上电过程中插入 SD 卡会自动挂载到 `/sdcard`。
- 运行过程中拔出 SD 卡会自动卸载。
- 未插卡时会低频重扫，避免刷屏和线程栈溢出。

文件读写测试

```
demo_sdcard
```

该命令会向 `/sdcard/demo_test.txt` 写入一段文本，然后读回并打印。

顺序读写测速

```
demo_sdcard_speed
demo_sdcard_speed 8192 64
```

参数说明：

- `total_kb`：测试文件总大小，默认 `4096 KB`。
- `block_kb`：单次读写块大小，默认 `64 KB`，最大 `64 KB`。

Flash 文件系统 Demo

功能说明

本工程启用了 **FAL + MTD NOR + littlefs**，启动时会将片外 Flash 的 `filesystem` 分区自动挂载到 `/flash`。如果首次挂载失败，初始化逻辑会尝试格式化该分区并重新挂载。

Flash 分区配置如下：

分区	FAL 偏移	大小	用途
`whd_firmware`	`0x00000`	`384 KB`	Wi-Fi 固件
`whd_clm`	`0x60000`	`64 KB`	Wi-Fi CLM 数据
`whd_nvram`	`0x70000`	`64 KB`	Wi-Fi NVRAM
`bt_image`	`0x80000`	`512 KB`	蓝牙镜像

分区	FAL 偏移	大小	用途
`filesystem`	`0x100000`	`1024 KB`	`/flash` littlefs 文件系统

> ***注意：*** *`demo_flash_speed` 会在 `/flash` 下创建临时测试文件，测试完成后自动删除。该命令会触发 Flash 擦写，请避免在有重要数据的 `/flash` 分区上反复运行大容量测试。*

顺序读写测速

```
demo_flash_speed
demo_flash_speed 512 4
```

参数说明：

- `total_kb`：测试文件总大小，默认 `512 KB`。
- `block_kb`：单次读写块大小，默认 `4 KB`，取值范围 `1 KB` 到 `64 KB`。

示例输出：

```
Flash speed test
file=/flash/flash_speed.bin, total=512 KB, block=4 KB
write  524288 bytes, 1234 ms, 0.41 MB/s (414 KB/s)
read   524288 bytes, 78 ms, 6.41 MB/s (6564 KB/s)
```

编译与下载

1. 打开工程并完成编译。
2. 使用 **板载下载器 (DAP)** 将开发板 USB 接口连接至 PC。
3. 通过编程工具将生成的固件烧录至开发板，也可以在工程根目录执行：

```
   .\m33_program.ps1
```

如需先整片擦除再烧录，可执行：

```
   .\m33_program.ps1 -EraseAll $true
```

4. 打开串口终端，进入 `msh />` 后执行对应 demo 命令。

启动流程

系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
```

```
|   （非安全核启动）  |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  （应用处理器启动）  |
+-------------------+
```

> ***注意：*** *安全核心固件已集成在 M33 工程的构建和打包流程中。请严格按照以上启动顺序准备并烧写固件，否则系统可能无法正常运行。*

若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```

2.2. Edgi-Talk_M55_Driver_All 多功能 Demo 工程

中文 | **English**

简介

本工程基于 **Edgi-Talk 平台**，运行于 **RT-Thread 实时操作系统 (M55 核)**，集中集成多个常用外设 demo，便于在同一个固件中验证传感器、ADC、HyperRAM、按键中断、RTC、SD 卡文件系统、片外 Flash 文件系统、音频播放和 CoreMark 性能测试功能。

所有 demo 都通过 MSH 命令手动触发，避免多个外设测试在上电后同时运行。

命令总览

命令	功能
`demo_aht20`	读取 AHT10/AHT20 温湿度
`demo_lsm6ds3 [count] [delay_ms]`	读取 LSM6DS3 六轴传感器数据
`demo_audio start/stop/status [speaker_volume] [mic_gain]`	音频录音到播放实时回环
`wavplay -s <file>`	播放 WAV 音频文件
`demo_adc`	读取 ADC1 Channel 1 电压
`demo_hyperram`	HyperRAM 基础读写校验
`demo_hyperram_speed [size_kb] [loops]`	HyperRAM 带宽测试
`demo_key`	注册 P8.3 按键中断，按下后翻转蓝色 LED
`demo_rtc [YYYY MM DD HH MM SS]`	读取或设置 RTC 时间
`demo_sdcard`	SD 卡文件写入和读取测试
`demo_sdcard_speed [total_kb] [block_kb]`	SD 卡顺序读写测速
`demo_fs_benchmark [file] [total_kb] [block_kb] [random_ops]`	基于 DFS/POSIX 的文件系统综合测速
`demo_flash_speed [total_kb] [block_kb]`	`/flash` 文件系统顺序读写测速
`core_mark`	CoreMark CPU 性能基准测试

软件说明

- 工程基于 **Edgi-Talk** 平台开发。

- 本工程是运行在 Cortex-M55 核上的 RT-Thread 应用；M33 工程需要在板级初始化阶段开启并启动 M55 核。
- 各 demo 通过串口 MSH 命令手动触发。
- Audio demo 使用 RT-Thread Audio 框架、PDM 麦克风和 ES8388 Codec；启动后会占用 `mic0` 与 `sound0`。
- WavPlayer demo 使用 RT-Thread `wavplayer` 软件包播放 WAV 文件；播放时会占用 `sound0`，不能和 `demo_audio` 回环同时运行。
- SD 卡挂载由公共文件系统初始化逻辑处理，插卡后自动挂载到 `/sdcard`。
- 片外 Flash 通过 FAL 注册为 `norflash0`，其中 `filesystem` 分区会以 littlefs 挂载到 `/flash`。

使用方法

编译与下载

1. 确认 M33 启动工程已经开启 CM55 Core。
2. 打开本工程并完成编译。
3. 使用 **板载下载器 (DAP)** 将开发板 USB 接口连接到 PC。
4. 通过编程工具或 RT-Thread Studio 的 launch 配置将生成的固件烧录到开发板。

运行效果

- M33 启动流程拉起 M55 后，RT-Thread 在 M55 核上运行并进入 MSH。
- 在串口终端进入 `msh />` 后执行对应 demo 命令。
- 音频、SD 卡、Flash、CoreMark 等占用资源较多的 demo 建议单独运行。

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Sep  5 2025 14:13:02
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
It's cortex-m55
msh >
```

常用 Demo

AHT20 温湿度 Demo

```
demo_aht20
```

读取一次 AHT10/AHT20 温度和湿度，并通过串口打印结果。

LSM6DS3 六轴传感器 Demo

```
demo_lsm6ds3
demo_lsm6ds3 10 200
```

`count` 为采样次数，`delay_ms` 为每次采样间隔。

Audio 录放 Demo

```
demo_audio start
demo_audio start 60 30
demo_audio status
demo_audio stop
```

`speaker_volume` 为扬声器音量，`mic_gain` 为麦克风增益。该 demo 会占用 `mic0` 与 `sound0`。

WavPlayer 音频播放 Demo

```
wavplay -s /sdcard/16000.wav
wavplay -t
wavplay -p
wavplay -r
wavplay -v 80
```

`wavplay` 和 `demo_audio` 都会占用 `sound0`。播放 WAV 前请先执行 `demo_audio stop`。

ADC Demo

```
demo_adc
```

读取 ADC1 Channel 1 的电压值。

HyperRAM Demo

```
demo_hyperram
demo_hyperram_speed
demo_hyperram_speed 1024 8
```

用于验证 HyperRAM 基础读写和顺序带宽。

按键中断 Demo

```
demo_key
```

注册 P8.3 按键下降沿中断，每次按键触发后翻转 P16.5 蓝色 LED。

RTC Demo

```
demo_rtc
demo_rtc 2026 7 17 10 30 0
```

读取当前 RTC 时间，或设置时间后再读取。

SD 卡 Demo

```
demo_sdcard
demo_sdcard_speed
```

```
demo_sdcard_speed 8192 64
demo_fs_benchmark
demo_fs_benchmark /sdcard/demo_fs_benchmark.bin 5120 64 64
```

SD 卡插入后会自动挂载到 `/sdcard`。

Flash 文件系统 Demo

```
demo_flash_speed
demo_flash_speed 512 4
```

该命令会在 `/flash` 下创建临时测速文件，测试完成后自动删除。

CoreMark 性能测试 Demo

```
core_mark
```

运行 CoreMark 前建议停止其他高负载 demo，避免 SD 卡、音频或传感器轮询影响结果。

注意事项

> *注意：本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- 如需修改工程的图形化配置，请使用以下工具打开配置文件：

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- 修改完成后保存配置，并重新生成代码。
- SD 卡建议格式化为 FAT16/FAT32。
- `/flash` 使用片外 Flash 的 `filesystem` 分区，首次挂载失败时会自动格式化；如果需要保留该分区数据，请不要随意执行整片擦除。
- `demo_flash_speed` 会触发 Flash 擦写，请避免在有重要数据的 `/flash` 分区上反复运行大容量测试。
- 若本工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常。

启动流程

系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全核心启动) |
+------------------+
          |
          v
+------------------+
```

```
|       M33        |
|   (非安全核启动) |
+------------------+
          |
          v
+------------------+
|       M55        |
|  (应用处理器启动) |
+------------------+
```

M55 应用依赖 M33 启动流程。若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```

2.3. Edgi_Talk_M55_WIFI 示例工程

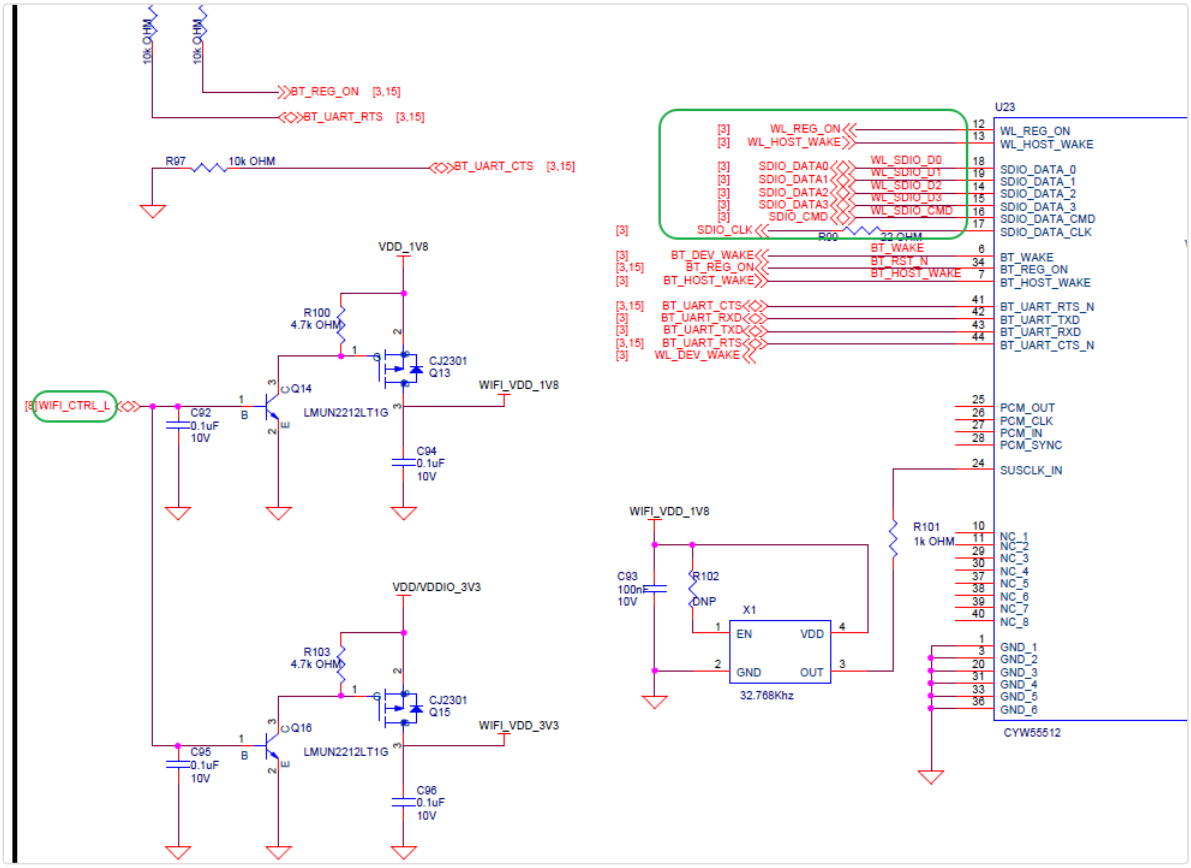
中文 | English

简介

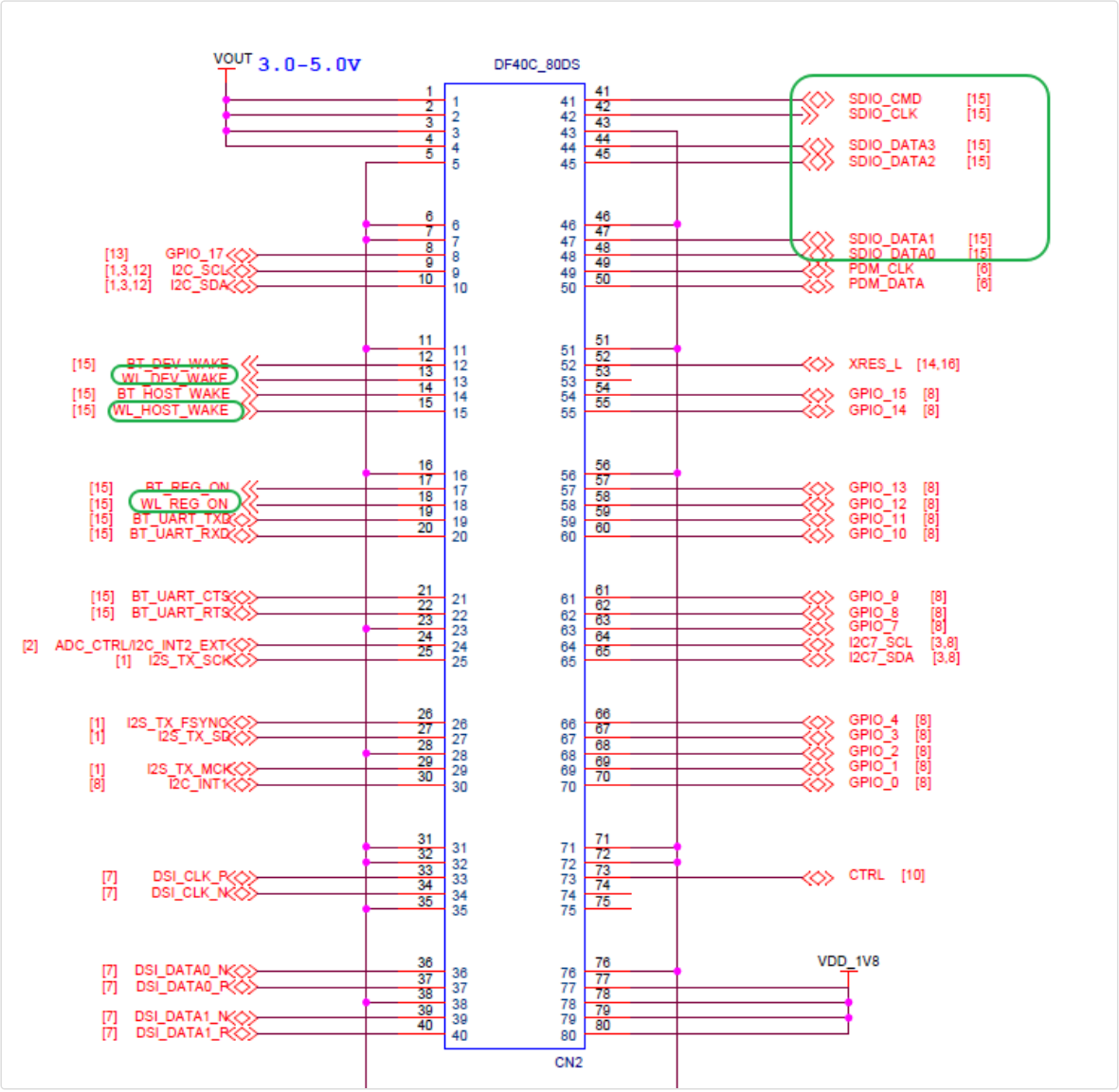
本示例工程基于 **Edgi-Talk 平台**，演示 **WIFI功能**，运行在 **RT-Thread 实时操作系统 (M55 核)** 上。
通过本工程，用户可以快速体验 WIFI的联网功能，并验证WIFI模块的接口，为后续WIFI的开发提供参考。

硬件说明

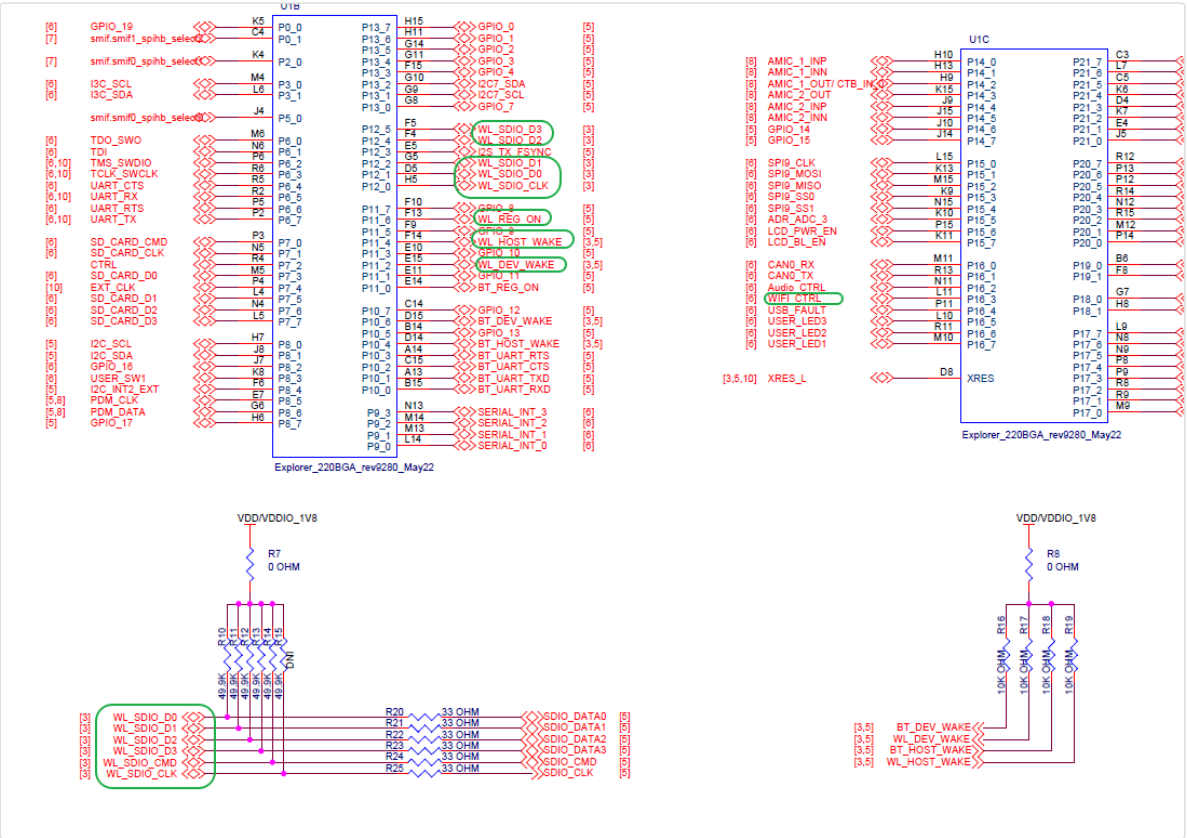
WIFI接口



BTB座子


VOUT 3.0-5.0V
DF40C_80DS
CN2
SDIO_CMD [15]
SDIO_CLK [15]
SDIO_DATA3 [15]
SDIO_DATA2 [15]
SDIO_DATA1 [15]
SDIO_DATA0 [15]
PDM_CLK [6]
PDM_DATA [6]
[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
XRES_L [14,16]
GPIO_15 [8]
GPIO_14 [8]
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
GPIO_13 [8]
GPIO_12 [8]
GPIO_11 [8]
GPIO_10 [8]
[15] BT_UART_CTS
[15] BT_UART_RTS
[2] ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
GPIO_9 [8]
GPIO_6 [8]
GPIO_7 [8]
I2C7_SCL [3,8]
I2C7_SDA [3,8]
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT1
GPIO_4 [8]
GPIO_3 [8]
GPIO_2 [8]
GPIO_1 [8]
GPIO_0 [8]
[7] DSI_CLK_P
[7] DSI_CLK_N
CTRL [10]
[7] DSI_DATA0_N
[7] DSI_DATA0_P
[7] DSI_DATA1_N
[7] DSI_DATA1_P
VDD_1V8

MCU接口



软件说明

- 工程基于 **Edgi-Talk** 平台开发。
- 示例功能包括：
- WIFI的扫描
- WIFI的连接
- Iperf测速
- 工程结构清晰，便于理解WIFI驱动和 RT-Thread 系统的配合使用。

使用方法

编译与下载

1. 打开工程并完成编译。
2. 使用 **板载下载器 (DAP)** 将开发板的 USB 接口连接至 PC。
3. 通过编程工具将 `Debug/rtthread.hex` 烧录至开发板。

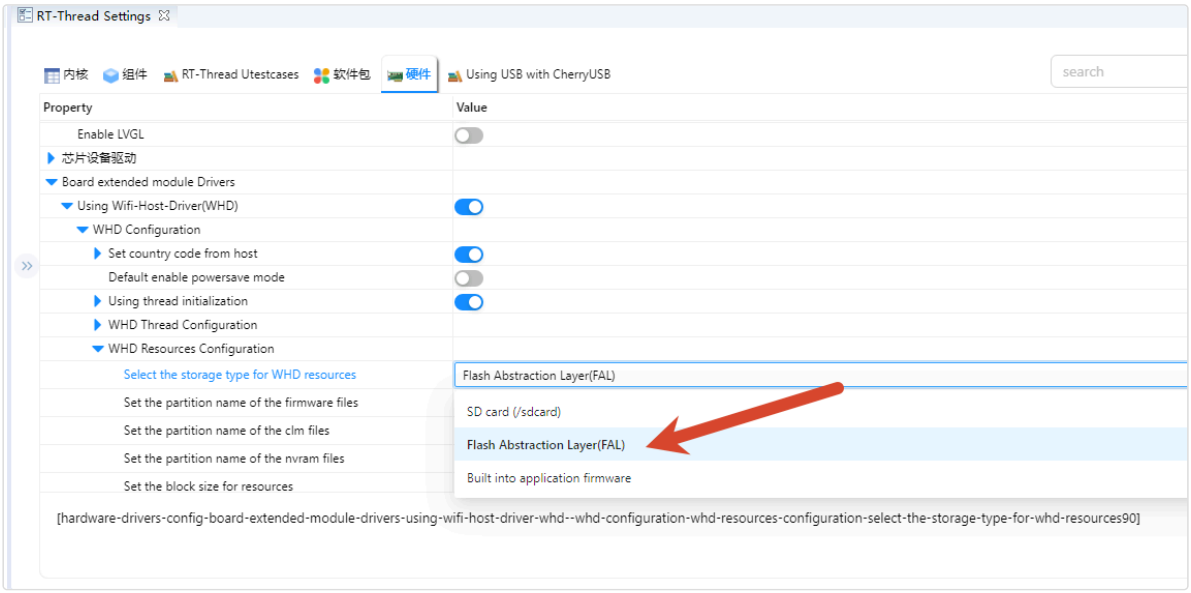
准备 Wi-Fi 资源

WHD 启动 Wi-Fi 前需要三个资源文件：固件、CLM 射频法规表和板级 NVRAM。当前文档默认介绍 FAL 分区方式，即 Wi-Fi 资源独立保存到 `whd_firmware` 、 `whd_clm` 、 `whd_nvram` 分区中。该方式下，烧录

`Debug/rtthread.hex` 只会写入应用镜像，Wi-Fi 资源需要通过串口命令单独写入。Edgi-Talk 默认提供的资源文件位于工程根目录的 `resources/` 文件夹中。

当前方式：FAL 分区加载

在 `settings` 中选择：

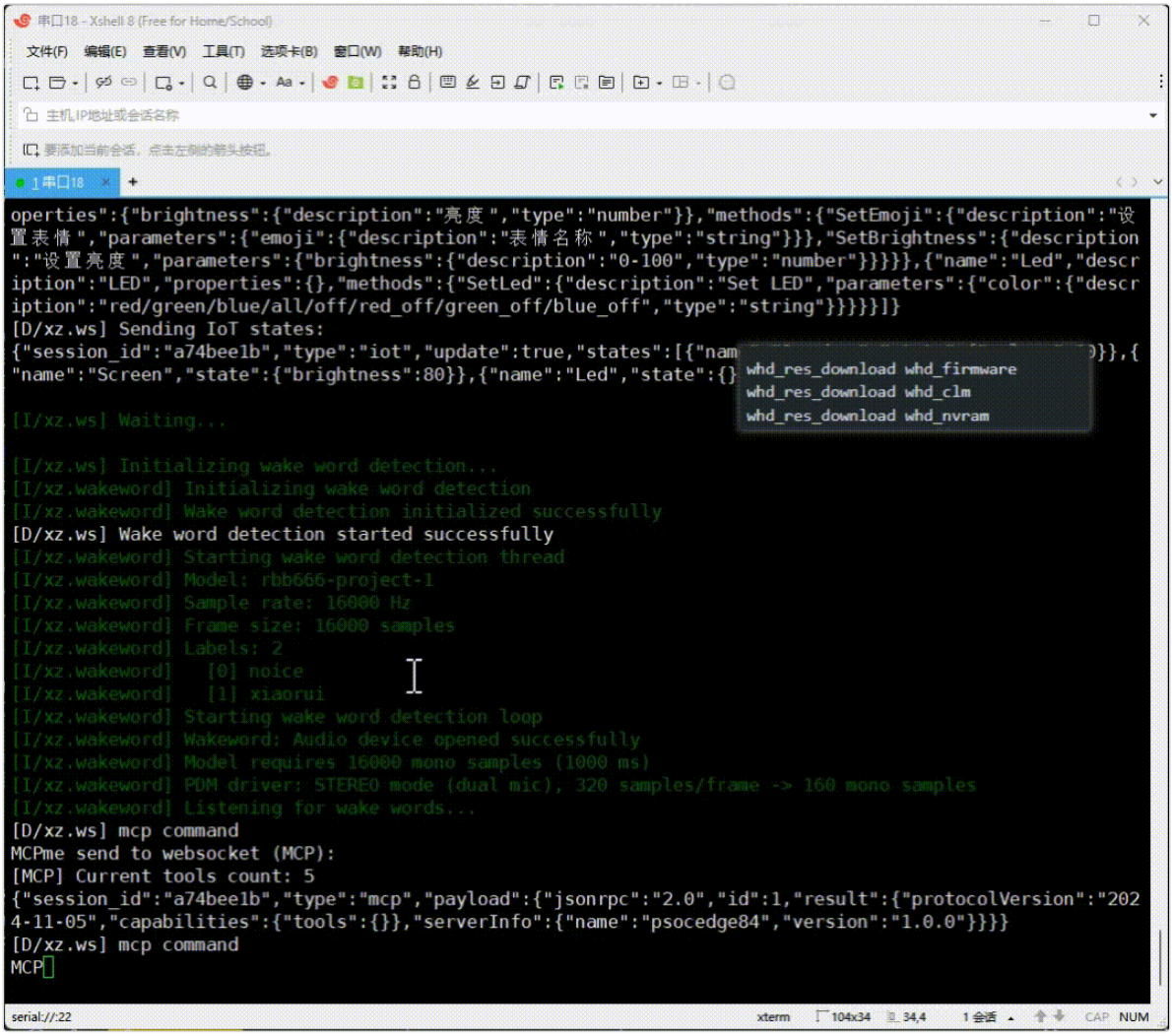


Edgi-Talk 默认提供的资源文件位于工程根目录的 `resources/` 文件夹中。

```
whd_res_download whd_firmware
whd_res_download whd_clm
whd_res_download whd_nvram
```

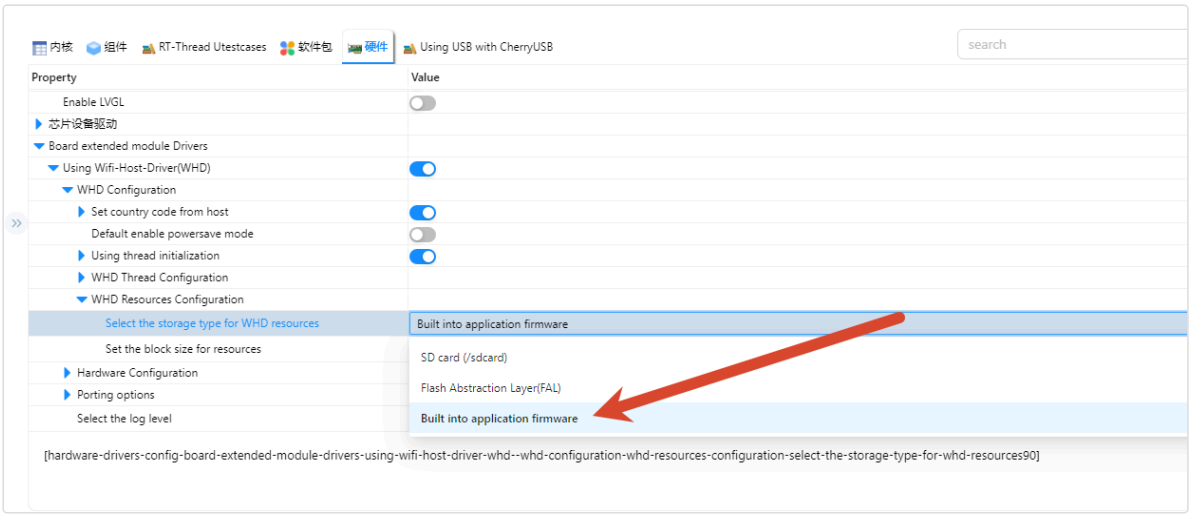
命令会切换到 YMODEM 传输模式，请使用支持 YMODEM 上传的终端软件（如 Xshell）发送 `resources/` 下对应文件。三者写入完成后重启开发板。

- 每次收到 `Download ... success` 提示后再进行下一项
- 三者写入完成后重启开发板即可让 WiFi 读取新的资源；若后续更新固件包，同样需要重新执行 `whd_res_download`。



拓展：Wi-Fi 资源随应用镜像烧录

如果希望 Wi-Fi 固件随着 `Debug/rtthread.hex` 一起烧进板子，可以切换为 `WHD_RESOURCES_IN_MEMORY` 模式：



该方式会把工程根目录 `resources/` 下的资源文件编进应用镜像：

- `resources/55500A1.trxcse`

- `resources/55500A1.clm_blob`
- `resources/cyw55513modpse84som_rev3.txt`

重新编译后，Wi-Fi 固件、CLM 和 NVRAM 会成为应用镜像的一部分。烧录 `Debug/rtthread.hex` 后不需要再执行 `whd_res_download whd_firmware`、`whd_res_download whd_clm`、`whd_res_download whd_nvram`。如果更新 Wi-Fi 资源文件，需要重新编译并重新烧录应用镜像。该方式会增加应用镜像大小，当前 CYW55500 固件和 CLM 约增加 240 KB Flash，再加上 NVRAM 文本。

注意：RT-Thread Studio 生成的 makefile 会在 `Debug` 目录下执行编译，而 ENV/SCons 会从工程根目录执行编译。当前工程已经按这个差异处理：Studio 构建时从 `../resources/` 查找资源，ENV/SCons 构建时从工程根目录的 `resources/` 生成资源代码。因此资源文件请放在工程根目录 `resources/`，不要只放在 `Debug/resources/`。

拓展：SD 卡资源加载方式

`WHD_RESOURCES_IN_SDCARD` 会在运行时从 `/sdcard` 读取资源。默认文件名为 `/sdcard/55500A1.trxcse`、`/sdcard/55500A1.clm_blob` 和 `/sdcard/cyw55513modpse84som_rev3.txt`。该模式会同时启用 BSP SD 卡文件系统挂载支持。

运行效果

- 烧录完成后，开发板上电即可运行示例工程。
- 系统会自动初始化 WIFI设备。
- 用户可在 **串口终端**使用以下命令连接WIFI：

```
wifi scan
```

```
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
wifi scan
             SSID                      MAC            security    rssi chn Mbps
------------------------------ ----------------  -------------- ---- --- ----
realthread_5G                  3c:6a:48:77:74:15  WPA2_AES_PSK   -87   36  300
realthread_5G                  3c:6a:48:c9:c2:79  WPA2_AES_PSK   -81   36  300
realthread_VIP5G               3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
realthread_VIP5G               3e:6a:48:99:c2:78  WPA2_AES_PSK   -81   36  300
realthread_5G                  3c:6a:48:77:74:15  WPA2_AES_PSK   -88   36  300
realthread_VIP5G               3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
TK-ML                          9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                    9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TK-ML                          9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                    9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TK-ML                          9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                    9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TEST                           82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
TEST                           82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
realthread                     3c:6a:48:77:74:14  WPA2_AES_PSK   -70    6  300
realthread_VIP                 3e:6a:48:17:74:14  WPA2_AES_PSK   -71    6  300
MLMEETING02                    68:dd:b7:f9:d1:49  WPA2_AES_PSK   -77   11  300
realthread_5G                  3c:6a:48:77:74:15  WPA2_AES_PSK   -87   36  300
realthread_VIP5G               3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
realthread_5G                  3c:6a:48:c9:c2:79  WPA2_AES_PSK   -80   36  300
realthread_VIP5G               3e:6a:48:99:c2:78  WPA2_AES_PSK   -80   36  300
TK-ML-GUEST                    9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
ChinaNet                       00:23:89:04:be:e1  OPEN           -78    1   54
realthread_VIP                 3e:6a:48:17:74:14  WPA2_AES_PSK   -96    6  300
TEST                           82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
realthread                     3c:6a:48:c9:c2:78  WPA2_AES_PSK   -71    6  300
realthread_VIP                 3e:6a:48:19:c2:78  WPA2_AES_PSK   -71    6  300
realthread                     3c:6a:48:77:74:14  WPA2_AES_PSK   -71    6  300
realthread_VIP                 3e:6a:48:17:74:14  WPA2_AES_PSK   -71    6  300
TEST                           82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
SH-online                      00:23:89:04:bf:30  OPEN           -72   11   54
ChinaNet                       00:23:89:04:bf:31  OPEN           -72   11   54
```

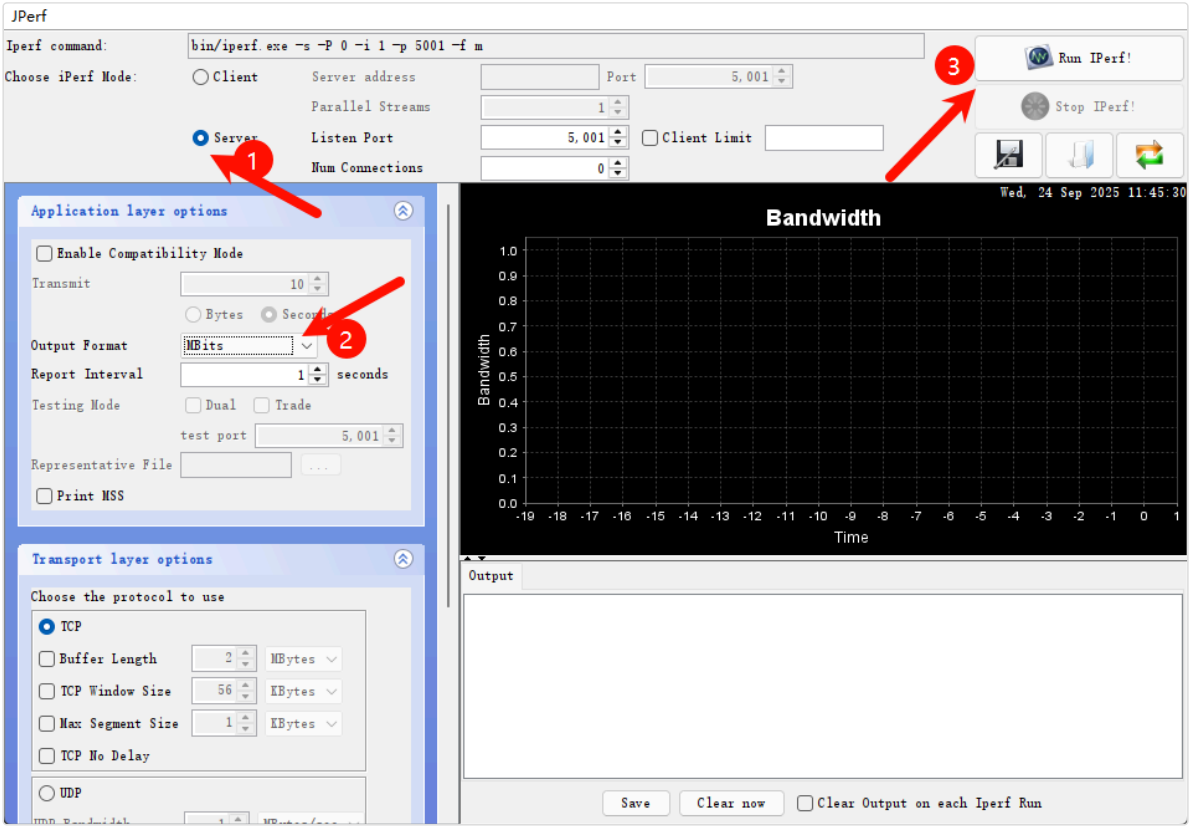
```
wifi join 名称 密码
```

```
msh />wifi join TEST 88888888
[I/WLAN.mgnt] wifi connect success ssid:TEST
msh />[I/WLAN.lwip] Got IP address : 10.19.22.51
```

```
ping www.rt-thread.org
```

```
msh />ping www.rt-thread.org
ping: not found specified netif, using default netdev w0.
60 bytes from 111.31.66.86 icmp_seq=0 ttl=52 time=61 ms
60 bytes from 111.31.66.86 icmp_seq=1 ttl=53 time=53 ms
60 bytes from 111.31.66.86 icmp_seq=2 ttl=53 time=37 ms
60 bytes from 111.31.66.86 icmp_seq=3 ttl=53 time=65 ms
```

- 网络连接完成后，可使用 iperf 进行性能测试。
- 在 packages\netutils-latest\tools 目录下提供了 jperf.rar 测速工具。
- 将其解压后，双击其中的 .bat 文件，即可启动工具，界面如下图所示：



- 在开发板终端输入以下命令（其中 电脑的 IP 请替换为实际地址），即可开始测速：

```
iperf -c <电脑的IP>
```

注意事项

- 可以使用电脑开热点进行测试，频段最好为2.4G。

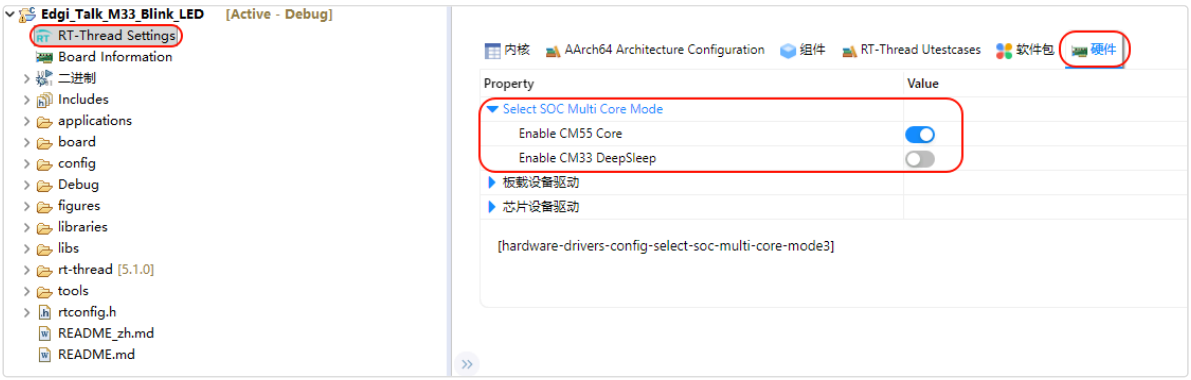
启动流程

系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|  (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|  (非安全核启动)  |
+------------------+
          |
          v
+------------------+
|       M55        |
| (应用处理器启动) |
+------------------+
```

⚠️ 请严格按照以上顺序烧写固件，否则系统可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- M55 核由 M33 启动链路拉起。当前模板中，**Edgi_Talk_M33_Template** 已在板级初始化阶段调用 `Cy_SysEnableCM55(MXCM55, CY_CM55_APP_BOOT_ADDR, 10)`，因此旧版的 `select SOC Multi Core Mode -> Enable CM55 Core` 菜单可能不会再出现。



3. 示例

3.1. Edgi-Talk_M55_LVGL 示例工程

中文 | **English**

简介

本示例工程基于 **Edgi-Talk 平台**，演示 **LVGL 图形库与多个显示示例**，运行在 **RT-Thread 实时操作系统** 的 **M55 应用核** 上。
当前工程默认启动 **Virtual3D Animated Emoji** 示例，可用于验证 LCD 显示、触摸输入、LVGL 渲染流程以及 M55 侧图形加速相关配置；也可以切换到 LVGL 官方 Music、Benchmark、Stress 等示例，为后续 GUI 应用开发提供基础参考。

LVGL 简介

LVGL（Light and Versatile Graphics Library，轻量且多功能的图形库）是一个开源的嵌入式 GUI（图形用户界面）开发框架。它的设计初衷是为资源受限的嵌入式设备提供流畅、现代化的图形界面，因此在运行效率、内存占用和可移植性方面都做了大量优化。无论是在简单的低端 MCU 上，还是在功能更强大的 MPU 平台上，LVGL 都能够高效运行，并提供丰富的图形控件和交互体验。

主要特点

1. **轻量级**
   LVGL 最大的优势在于轻量化，它对资源的需求非常低。在一个内存只有几十 KB 的微控制器上，LVGL 依然可以流畅运行。框架本身的内存开销较小，渲染算法经过精心优化，既可以保证较低的 CPU 使用率，又能在有限的硬件条件下呈现出较好的视觉效果。这使得 LVGL 特别适合那些对硬件资源敏感的应用场景，比如低功耗设备、可穿戴设备、家用电器控制面板等。

2. **跨平台**
   LVGL 的跨平台特性非常突出，它能够运行在多种操作系统之上，包括 FreeRTOS、RT-Thread、Zephyr、Linux 等，也可以直接运行在裸机环境中。开发者只需要为 LVGL 提供底层的显示驱动和输入驱动接口，便可以快速完成移植，从而在不同的硬件平台之间复用同一套 UI 代码。这种灵活性大大降低了开发成本，使得 LVGL 成为嵌入式 GUI 的通用解决方案。

3. **丰富的控件（Widgets）**
   LVGL 内置了种类丰富的 GUI 控件，例如按钮、标签、滑条、进度条、复选框、列表、表格、图表等。这些控件几乎涵盖了常见人机交互界面的需求，开发者无需从零开始设计和实现组件，大大缩短了开发周期。同时，LVGL 还允许用户基于现有控件扩展新的组件，从而构建符合特定需求的界面。无论是简单的数字显示，还是复杂的图形控制界面，LVGL 都能胜任。

4. **多样的渲染能力**
   在视觉呈现方面，LVGL 提供了丰富的渲染功能。它支持抗锯齿、透明度、渐变、阴影、边框、圆角等效果，可以让界面看起来更加美观和现代。除此之外，LVGL 还内置动画系统，支持多种缓动函数，能够实现平滑的控件移动、渐变过渡和动态效果。这些特性使得界面不仅功能完善，还能带来良好的用户体验。

5. **输入设备支持**
   LVGL 支持多种输入设备类型，包括触摸屏、电容屏、鼠标、键盘和编码器，甚至可以实现多点触控交互。它提供了统一的输入接口层，开发者只需实现底层的驱动适配，就能轻松把输入事件传递到 LVGL 的事件系统中。这样一来，开发者可以更加专注于上层的界面逻辑，而不必花费过多时间在输入事件处理上。

6. **国际化与多语言**
   LVGL 在国际化方面也有很好的支持。它使用 UTF-8 编码，可以处理几乎所有语言的字符集。同时，它支持双向文本渲染，可以正确显示如阿拉伯语、希伯来语等从右到左书写的语言。这使得 LVGL 能够应用在面向全球用户的产品中，不同国家和地区的用户都能通过本地化界面获得良好的体验。

7. **可扩展性**
   LVGL 提供了灵活的主题和样式系统，开发者可以方便地定制控件的外观样式，以实现不同风格的 UI。通过更换主题，可以快速切换界面的整体视觉风格。此外，LVGL 还可以与第三方图形库、文件系统和图像解码器结合使用，从而扩展其功能。例如，可以使用文件系统来加载外部字体和图片，或者集成 JPEG/PNG 解码库来显示复杂图像。

应用场景

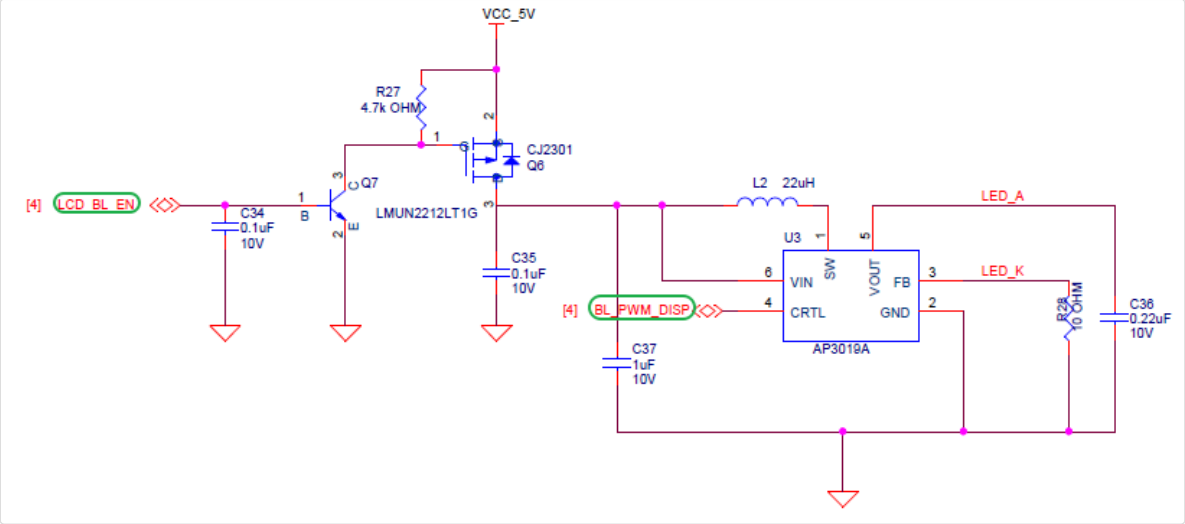
LVGL 在实际应用中有着非常广泛的覆盖范围。在消费电子领域，它常用于智能家居控制面板、家电显示屏、智能手表和健身设备等，这些产品往往需要在有限硬件资源下提供友好的用户交互界面。在工业控制领域，LVGL 被用于 HMI（人机交互界面）和各种仪器仪表，帮助用户直观地监控和操作设备。在汽车电子中，LVGL 可以驱动中控屏、副驾娱乐屏，甚至车载仪表盘。而在医疗设备方面，它则适合做小型显示界面，例如手持检测仪、便携式监护设备等。

生态与社区

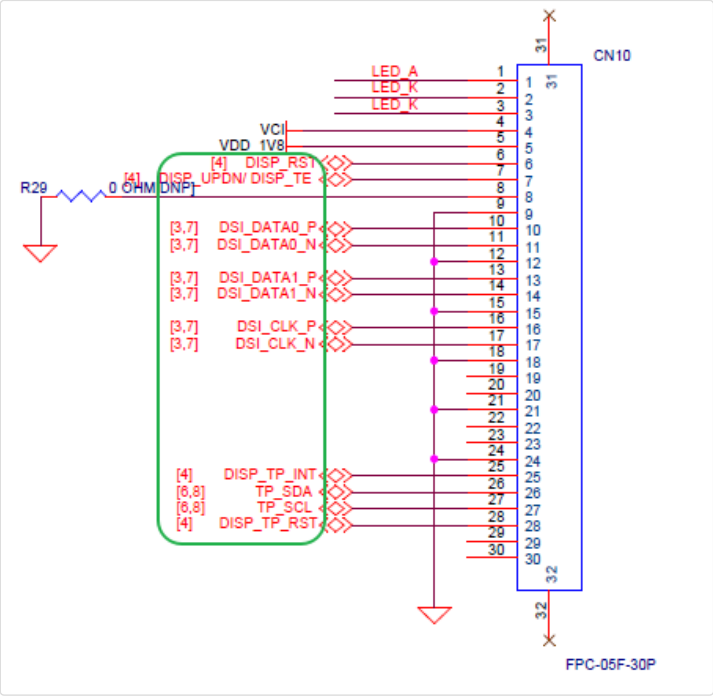
LVGL 作为一个开源项目，采用 **MIT License** 开源协议，既适合个人学习，也可以在商业产品中免费使用。围绕 LVGL，已经形成了一个活跃的生态系统。官方提供了 **SquareLine Studio** 这类可视化设计工具，支持拖拽式设计界面并导出 LVGL 代码，极大地提升了开发效率。此外，**LVGL Simulator** 可以让开发者直接在 PC 上进行界面调试，不需要每次都烧录到目标硬件上。全球开发者社区十分活跃，贡献了大量开源控件、主题和移植案例，为新手和企业开发者提供了丰富的资源和支持。

硬件说明

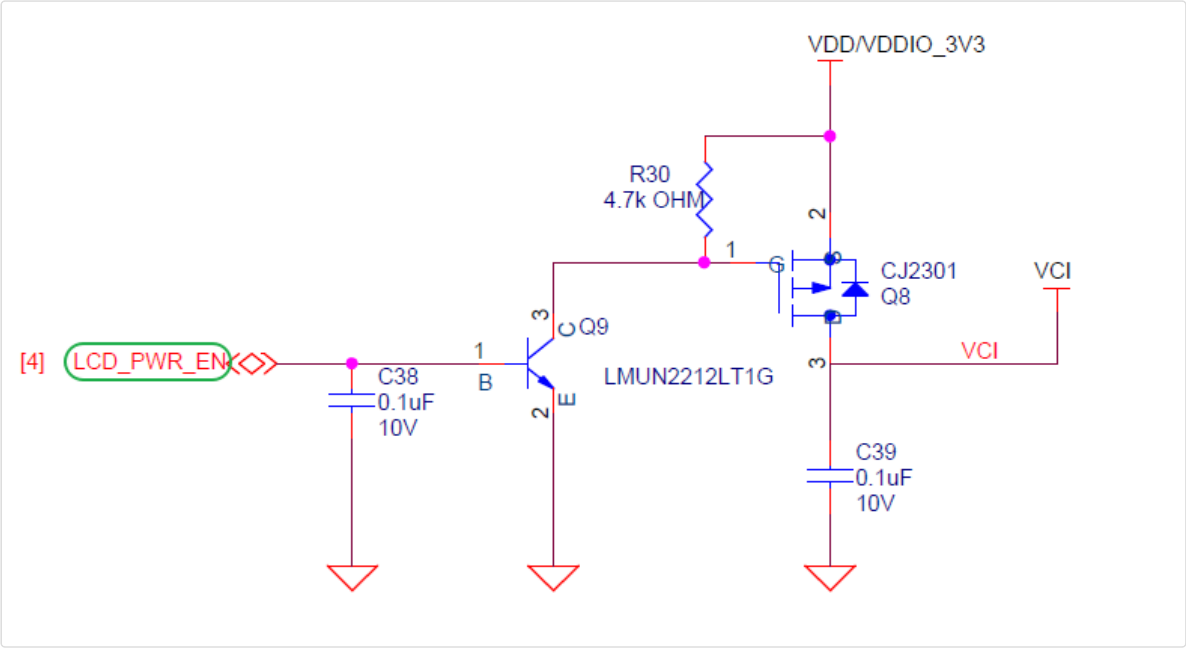
背光接口



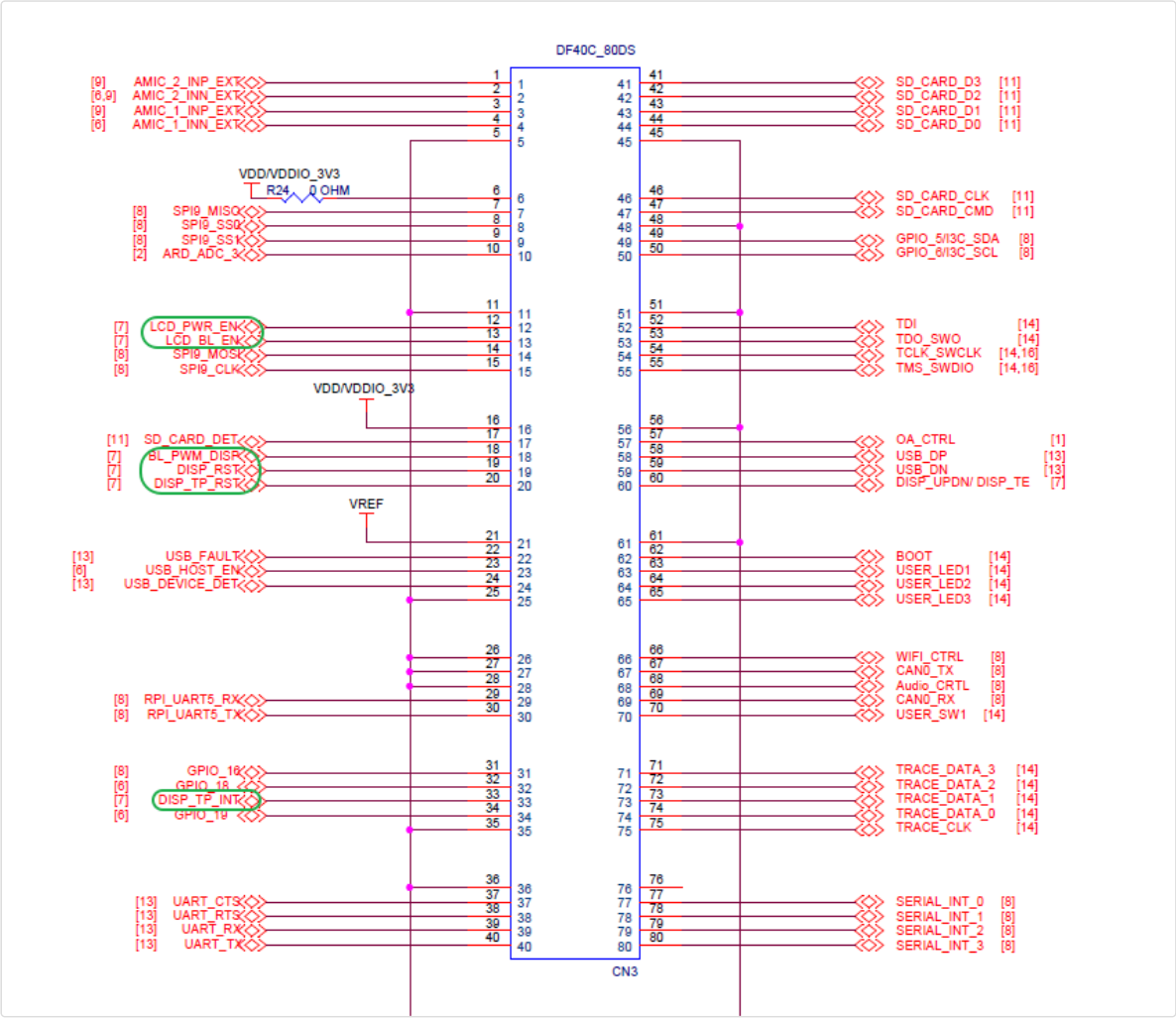
MIPI接口

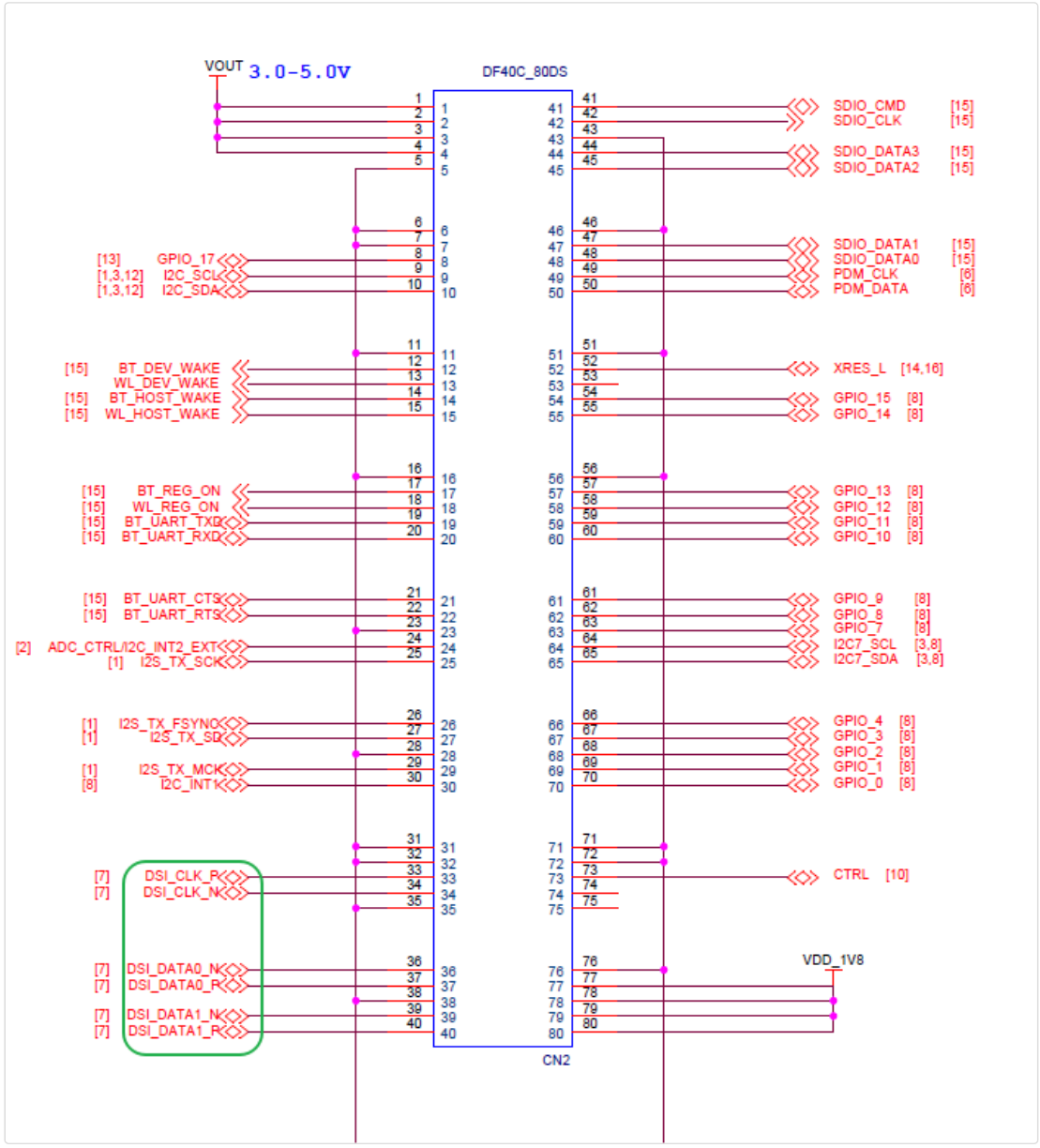


PWR接口

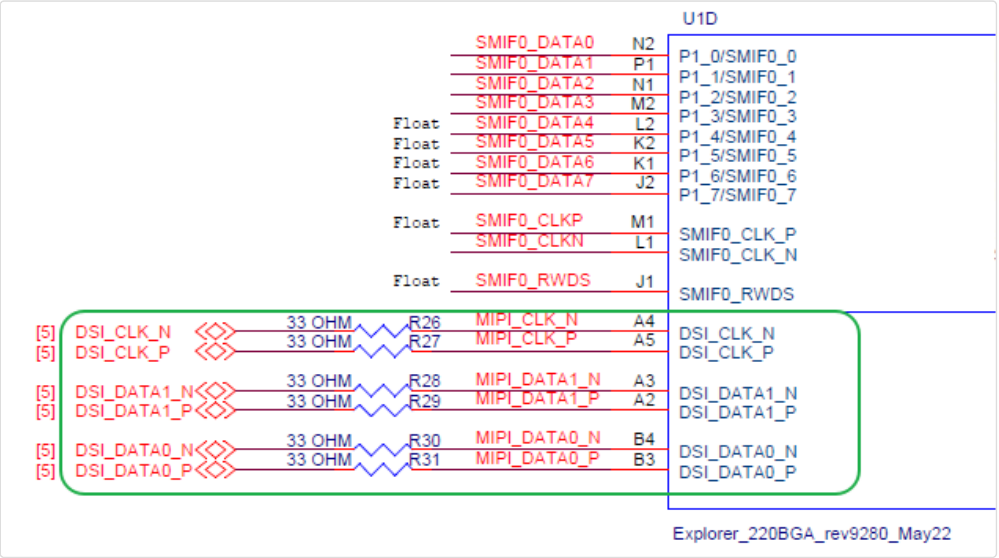

VDD/VDDIO_3V3
R30
4.7k OHM
2
1
G
CJ2301
Q8
VCI
VCI
3
Q9
[4]
LCD_PWR_EN
1
C38
0.1uF
10V
B
LMUN2212LT1G
3
2
E
C39
0.1uF
10V

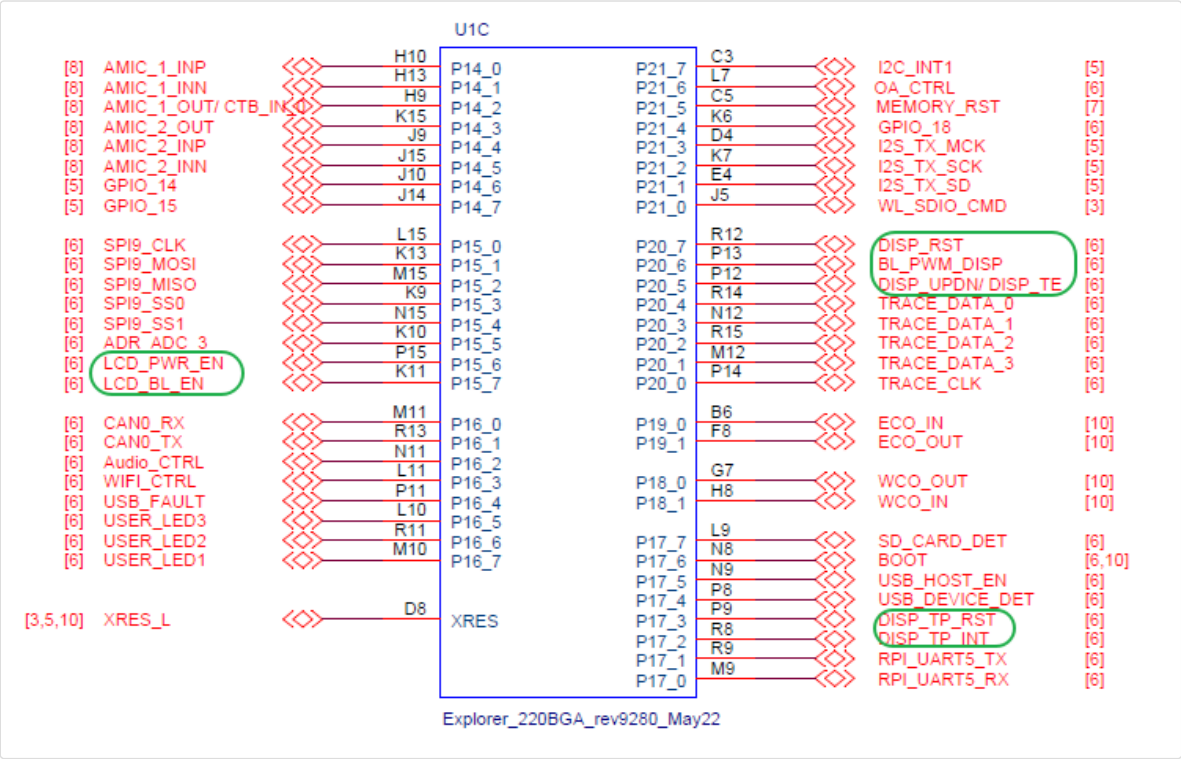
BTB座子


DF40C_80DS
CN3
VDD/VDDIO_3V3
R24 0 OHM
VDD/VDDIO_3V3
VREF
[9] AMIC_2_INP_EXT
[6,9] AMIC_2_INN_EXT
[9] AMIC_1_INP_EXT
[6] AMIC_1_INN_EXT
[8] SPI9_MISO
[8] SPI9_SS0
[8] SPI9_SS1
[2] ARD_ADC_3
[7] LCD_PWR_EN
[7] LCD_BL_EN
[8] SPI9_MOSI
[8] SPI9_CLK
[11] SD_CARD_DET
[7] BL_PWM_DISP
[7] DISP_RST
[7] DISP_TP_RST
[13] USB_FAULT
[6] USB_HOST_EN
[13] USB_DEVICE_DET
[8] RPI_UART5_RX
[8] RPI_UART5_TX
[8] GPIO_16
[6] GPIO_18
[7] DISP_TP_INT
[6] GPIO_19
[13] UART_CTS
[13] UART_RTS
[13] UART_RX
[13] UART_TX
SD_CARD_D3 [11]
SD_CARD_D2 [11]
SD_CARD_D1 [11]
SD_CARD_D0 [11]
SD_CARD_CLK [11]
SD_CARD_CMD [11]
GPIO_5/I3C_SDA [8]
GPIO_6/I3C_SCL [8]
TDI [14]
TDO_SWO [14]
TCLK_SWCLK [14,16]
TMS_SWDIO [14,16]
OA_CTRL [1]
USB_DP [13]
USB_DN [13]
DISP_UPDN/ DISP_TE [7]
BOOT [14]
USER_LED1 [14]
USER_LED2 [14]
USER_LED3 [14]
WIFI_CTRL [8]
CAN0_TX [8]
Audio_CRTL [8]
CAN0_RX [8]
USER_SW1 [14]
TRACE_DATA_3 [14]
TRACE_DATA_2 [14]
TRACE_DATA_1 [14]
TRACE_DATA_0 [14]
TRACE_CLK [14]
SERIAL_INT_0 [8]
SERIAL_INT_1 [8]
SERIAL_INT_2 [8]
SERIAL_INT_3 [8]


VOUT 3.0-5.0V
DF40C_80DS
CN2
SDIO_CMD [15]
SDIO_CLK [15]
SDIO_DATA3 [15]
SDIO_DATA2 [15]
SDIO_DATA1 [15]
SDIO_DATA0 [15]
PDM_CLK [6]
PDM_DATA [6]
XRES_L [14,16]
GPIO_15 [8]
GPIO_14 [8]
GPIO_13 [8]
GPIO_12 [8]
GPIO_11 [8]
GPIO_10 [8]
GPIO_9 [8]
GPIO_8 [8]
GPIO_7 [8]
I2C7_SCL [3,8]
I2C7_SDA [3,8]
GPIO_4 [8]
GPIO_3 [8]
GPIO_2 [8]
GPIO_1 [8]
GPIO_0 [8]
CTRL [10]
VDD_1V8
[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
[15] BT_UART_CTS
[15] BT_UART_RTS
[2] ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT1
[7] DSI_CLK_P
[7] DSI_CLK_N
[7] DSI_DATA0_N
[7] DSI_DATA0_P
[7] DSI_DATA1_N
[7] DSI_DATA1_P

MCU接口





软件说明

- 工程基于 **Edgi-Talk** 平台开发，运行在 **M55 应用核** 上。
- 示例功能包括：
- 初始化 **LVGL 9.2** 图形库、LCD 显示驱动和触摸输入驱动
- 默认启动 **Virtual3D Animated Emoji** 示例
- 支持切换 **lv_demo_music**、**lv_demo_benchmark**、**lv_demo_stress** 等 LVGL 示例
- 默认开启 M55 I-Cache/D-Cache，并使用 AXIDMAC 优化 RGB565 区域拷贝

- 工程结构简洁，便于理解 **显示驱动接口** 和 **LVGL 移植流程**。

示例说明

当前工程通过 `BSP_LVGL_DEMO_*` 配置项选择启动的 LVGL 示例，默认配置为 `BSP_LVGL_DEMO_VIRTUAL3D_EMOJI`。

配置项	示例	说明
`BSP_LVGL_DEMO_VIRTUAL3D_EMOJI`	Virtual3D Animated Emoji	当前默认示例。屏幕显示 3D Emoji 动画，支持触摸拖动交互，用于验证 Virtual3D、LVGL 显示刷新和触摸输入链路。该示例仅支持 LCD 旋转 0 度或 180 度。
`BSP_LVGL_DEMO_MUSIC`	LVGL Music Demo	LVGL 官方音乐界面示例，主要用于验证复杂控件、布局、样式和动画效果。
`BSP_LVGL_DEMO_BENCHMARK`	LVGL Benchmark Demo	LVGL 官方性能测试示例，可用于观察界面绘制性能、刷新帧率和综合评分。
`BSP_LVGL_DEMO_STRESS`	LVGL Stress Demo	LVGL 官方压力测试示例，会反复创建、刷新和销毁控件，用于验证渲染稳定性和内存使用情况。

切换示例时，可在 **RT-Thread Settings** 或 `menuconfig` 中修改 LVGL Demo 相关配置项。建议同一时间只选择一个 `BSP_LVGL_DEMO_*` 示例，修改后重新生成配置并编译下载。

使用方法

编译与下载

1. 打开工程并完成编译。
2. 使用 **板载下载器 (DAP)** 将开发板的 USB 接口连接至 PC。
3. 通过编程工具将生成的固件烧录至开发板。

运行效果

- 烧录完成后，开发板上电即可运行示例工程。
- 默认配置下，系统会自动启动 **Virtual3D Animated Emoji**，LCD 上显示 3D Emoji 动画，界面标题为 `Virtual3D Animated Emoji`，底部提示为 `Drag the 3D emoji`。
- 串口会打印当前启动的示例和 LCD 旋转角度，例如：

```
LVGL virtual3d_emoji demo start, lcd rotation=0
```

- 如需查看 Virtual3D 示例状态，可在 MSH 终端执行以下命令：

```
virtual3d_demo_stat
virtual3d_render_stat
```

- 如果切换为 Music、Benchmark 或 Stress 示例，LCD 会显示对应的 LVGL 官方示例界面。

注意事项

> ⚠️ ***注意：*** *本工程要求使用* ***RT-Thread Studio 2.2.9*** *或以上版本。*

- 如需修改工程的 **图形化配置**，请使用以下工具打开配置文件：

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- 修改完成后保存配置，并重新生成代码。
- 当前默认示例 `BSP_LVGL_DEMO_VIRTUAL3D_EMOJI` 不支持 LCD 旋转 90 度或 270 度，请使用 0 度或 180 度；如果需要横屏旋转，请切换到其它 LVGL 示例。
- 默认配置已开启 `BSP_LVGL_ENABLE_CPU_CACHE` 和 `BSP_LCD_USE_AXIDMAC_AREA_COPY`。如需修改缓存、Framebuffer 或 LCD 刷新方式，请同步检查显示缓冲区一致性。
- 若显示屏幕无输出，请检查：
- LCD 硬件连接与电源供给是否正常
- `lv_port_disp.c` 和 `lv_port_indev.c` 的配置是否与实际硬件匹配
- LCD 旋转角度是否与当前示例兼容

启动流程

系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  (应用处理器启动)  |
+-------------------+
```

⚠️ 请严格按照以上顺序烧写固件，否则系统可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- 若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```

3.2. 小智示例工程

中文 | **English**

简介

本示例工程基于 **Edgi-Talk 平台**，演示 **小智语音交互设备的基本功能**，运行在 **RT-Thread 实时操作系统**上。
通过本工程，用户可以快速验证设备 **WiFi 连接**、**按键唤醒** 与 **语音交互** 功能，为后续应用开发提供基础参考。

软件说明

- 工程基于 **Edgi-Talk** 平台开发。
- 示例功能包括：
- WiFi 连接与状态显示
- 按键唤醒与语音交互
- 设备状态管理（待机、监听、休眠等）

使用方法

准备 WiFi 资源（首次必做）

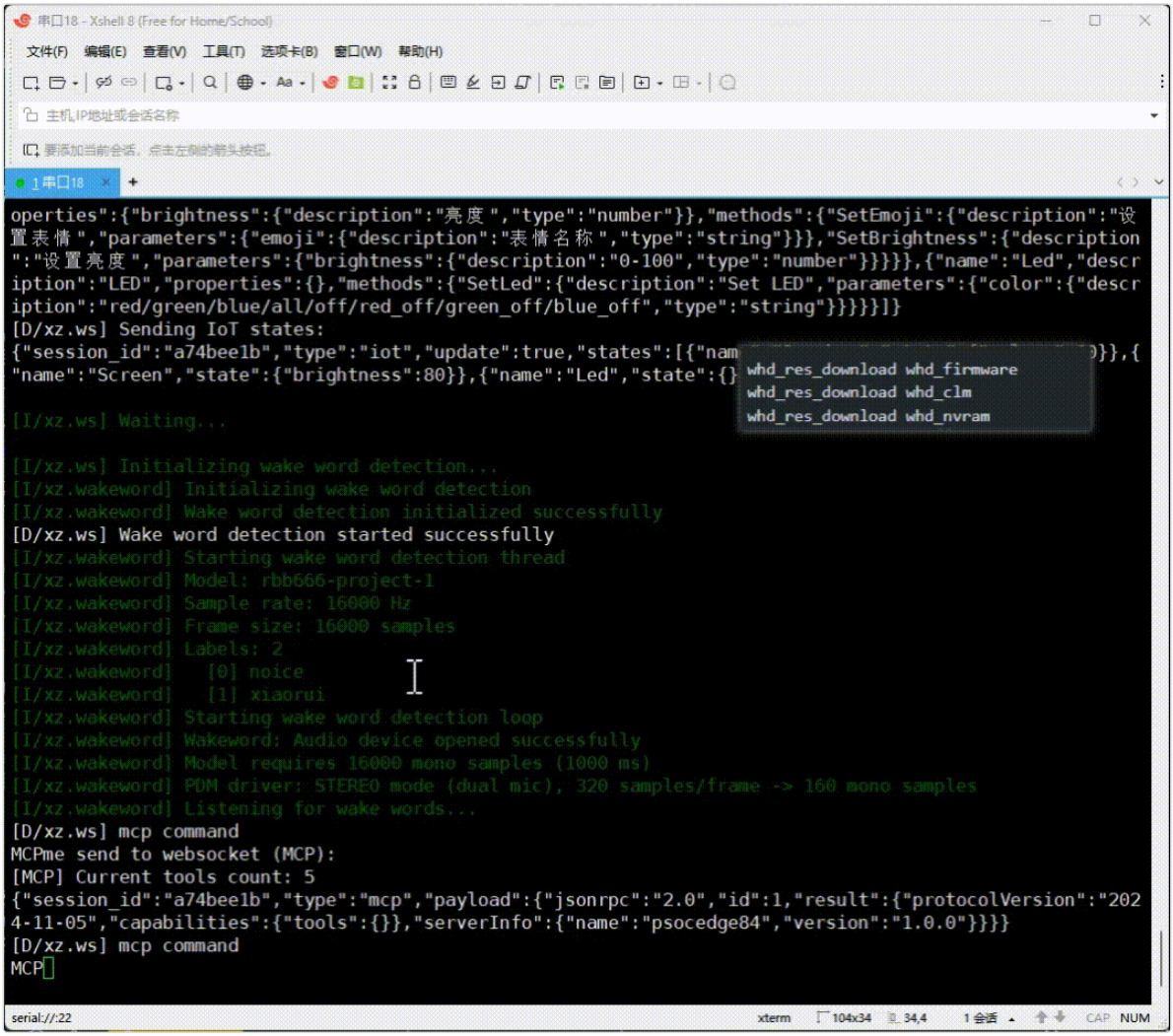
WHD 在启动时需要从 FAL 读取三个外部资源文件：`firmware`（二进制固件）、`clm_blob`（射频法规表）以及 `nvram.txt`（模组校准参数）。这些文件独立于应用固件，重新烧录应用不会自动更新它们，因此首次使用或更换固件包时必须手动写入。Edgi-Talk 默认提供的资源文件位于工程根目录的 `resources/` 文件夹中。

- 在 menuconfig 中保持 `WHD_RESOURCES_IN_EXTERNAL_STORAGE_FAL` 选项开启，并在 FAL 分区表中保留默认的 `whd_firmware`、`whd_clm`、`whd_nvram` 分区（默认分别占用 512 KB + 32 KB + 32 KB 的片上 Flash）。
- 打开串口终端，复位进入 `msh` 命令行，依次执行以下命令：

```
whd_res_download whd_firmware
whd_res_download whd_clm
whd_res_download whd_nvram
```

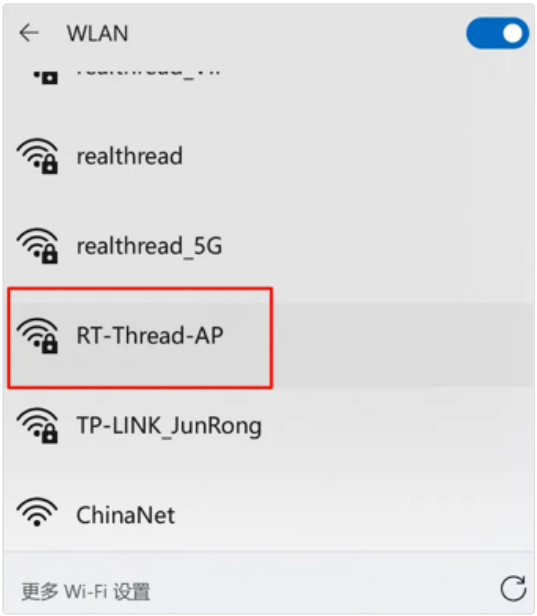
命令会切换到 YMODEM 传输模式，请使用支持 YMODEM 上传的终端软件（如 Xshell）选择根目录 `resources/` 下对应芯片的文件并发送。

- 每次收到 `Download ... success` 提示后再进行下一项
- 三者写入完成后重启开发板即可让 WiFi 读取新的资源；若后续更新固件包，同样需要重新执行 `whd_res_download`。



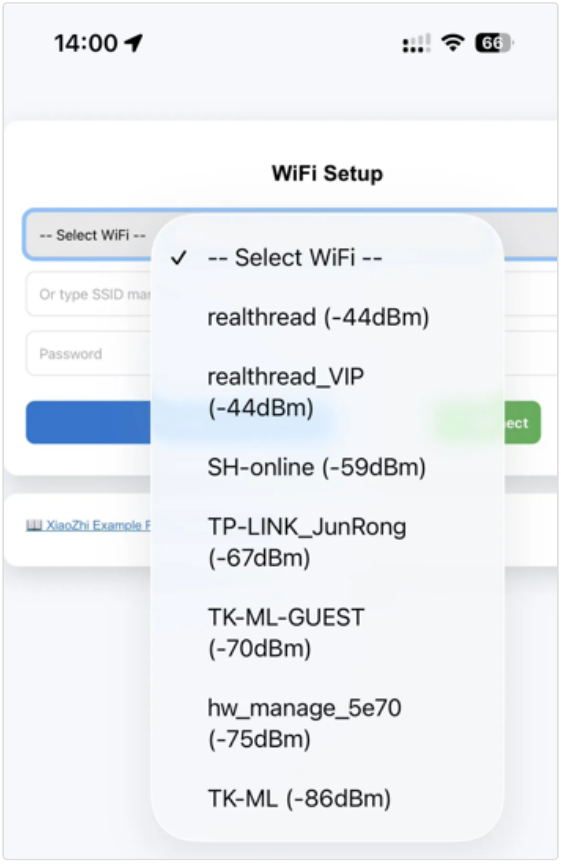
1. 初次使用（AP 配网）

1. 开发板启动会进入 **AP 模式**。
   在手机 / 电脑中连接设备的热点（密码见屏幕显示）：

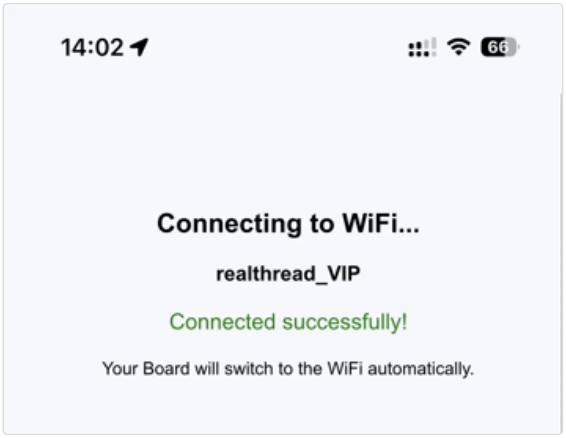


2. 连接成功后，在浏览器中输入 **192.168.169.1** 打开后台界面进行配网：

3. 点击 **Scan** 扫描附近的 Wi-Fi 热点：



4. Wi-Fi 连接成功后，将显示如下界面：



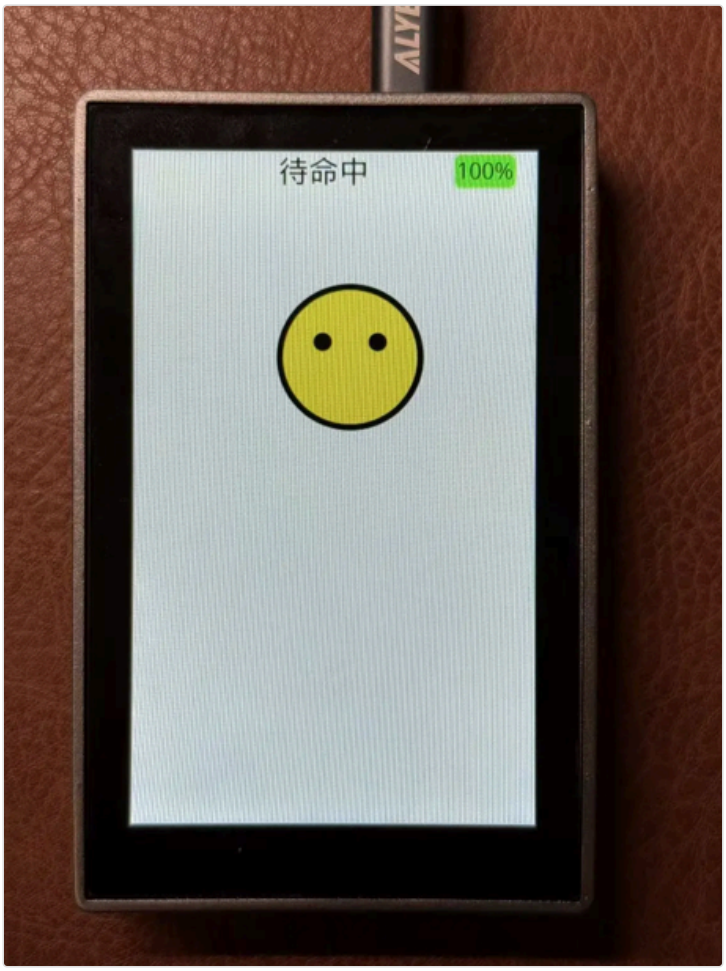
5. 当设备屏幕显示 **“待命中”** 时，表示可以正常进行语音对话：

> ***提示***：*按一下开发板上的* ***第一个用户按键*** *进入语音输入，等待 1–2 秒后，小智会自动回复。*

小智表情含义说明

1. 联网中（请稍候）

2. 监听中（需要按按键开始对话）



3. 聆听中（正在处理对话内容）

4. 对话中（小智正在回复你）

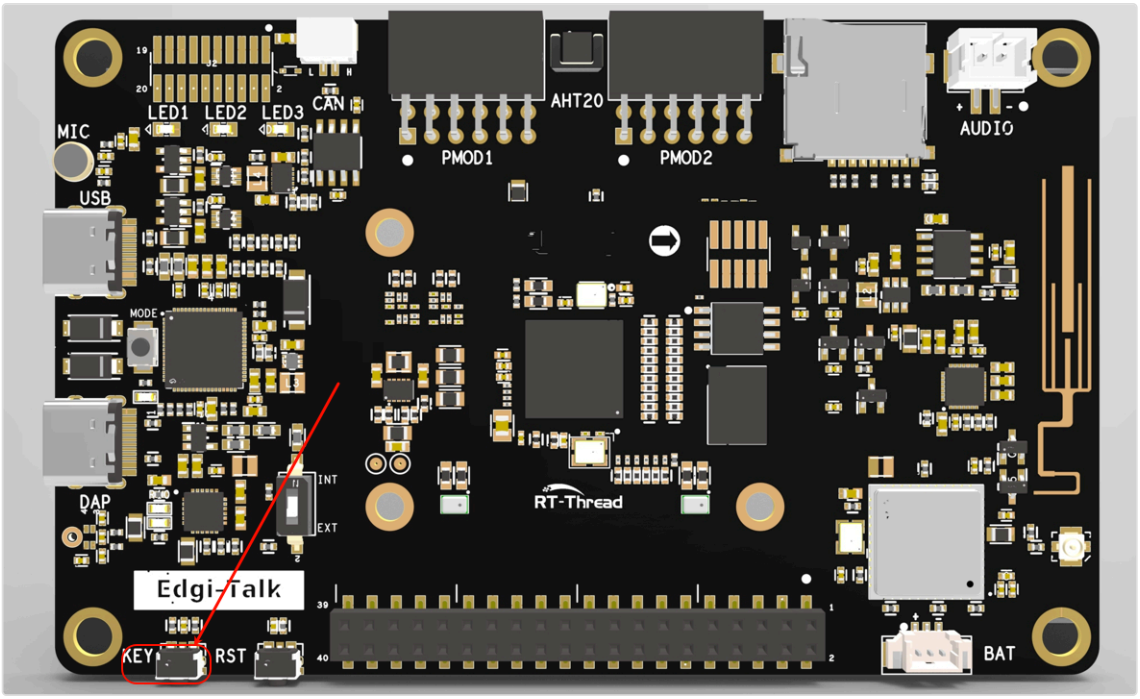
5. 休眠状态（低功耗）

> *从休眠恢复：按一下按键 → 等待进入“待命中” → 可继续对话。 图片大小缩小一点*

运行效果

- 烧录完成后，设备上电即可运行示例工程。

- **按一下**顶部按键对话，可进入 **聆听中** 状态进行语音交互。



注意事项

> ⚠️ ***注意：*** *本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- 第一次需要进入 小智官网 进行后台绑定

  按下用户按键屏幕显示验证码

- 请确保 WiFi 名称与密码正确，并使用 **2.4GHz 频段**。
- 设备需在可访问互联网的环境下使用。
- 如需修改工程的 **图形化配置**，请使用以下工具：

```
tools/device-configurator/device-configurator.exe
```

```
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- 修改完成后保存配置，并重新生成代码。

启动流程

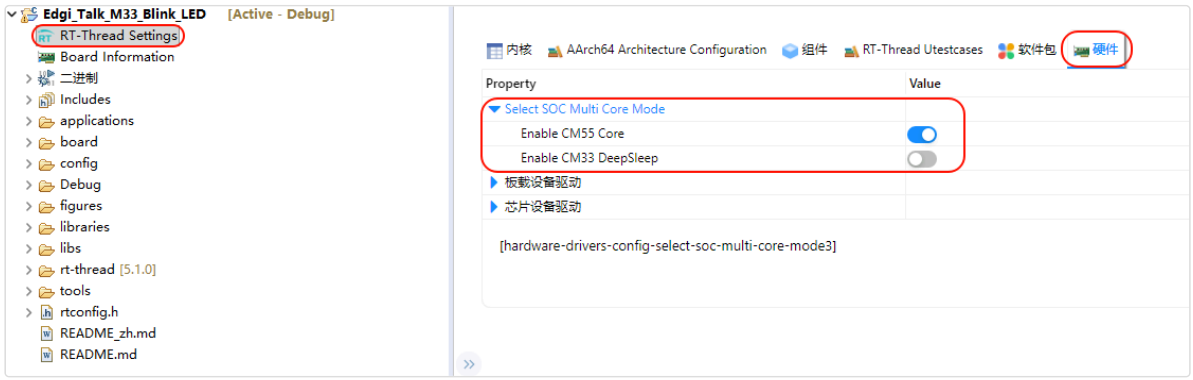
系统启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+--------------------+
|       M55          |
|  (应用处理器启动)   |
+--------------------+
```

⚠️ 请严格按照以上顺序烧写固件，否则设备可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- 若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```



4. USB篇

4.1. Edgi_Talk_Extend_Screen CherryUSB USB 拓展屏示例工程

中文 | **English**

简介

本工程在 **Edgi-Talk M55 核心**上集成 **CherryUSB**，默认配置为 **USB 设备模式**，并使用 **Infineon DWC2** IP，实现 **Windows 下 USB 拓展屏幕**。

默认配置

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- 设备模板：**none**（由用户自行实现）
- 显示协议：Vendor 接口传输 RGB565 帧

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件。
3. 使用 Type-C 接口连接 USB 进行枚举。

配置方法（切换模式）

在 RT-Thread Studio 中打开：

```
RT-Thread Settings -> USB -> CherryUSB
```

- **设备模式**：开启 `RT_CHERRYUSB_DEVICE`，在 **CHERRYUSB_DEVICE_IP** 中选择 IP（默认 `RT_CHERRYUSB_DEVICE_DWC2_INFINEON`）。
- **设备类模板**：开启对应类驱动后，在 **Select usb device template** 中选择模板。

若 IP 或类驱动需要额外参数，请修改：

- `libraries/Common/board/ports/usb/usb_config.h`

Windows IDD 驱动程序

Windows 间接显示驱动程序 (IDD) 模型提供简单的用户模式驱动程序模型，以支持未连接到传统 GPU 显示输出的监视器。

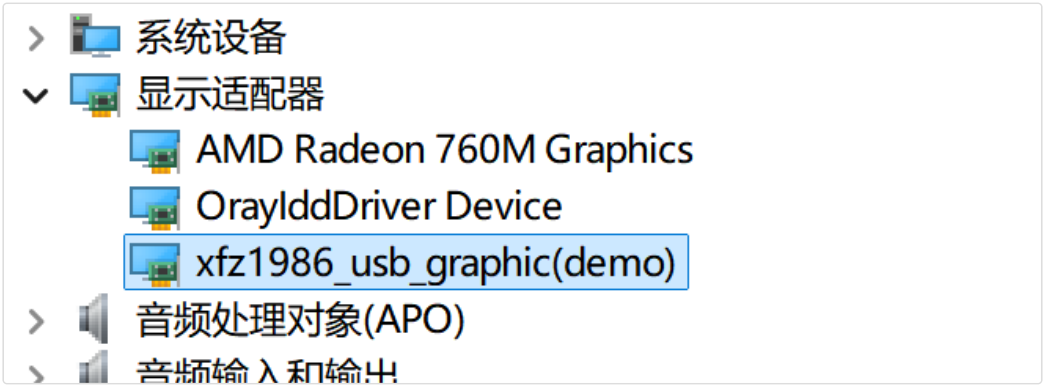
参 考： https://learn.microsoft.com/zh-cn/windows-hardware/drivers/display/indirect-display-driver-model-overview

本驱动参考自：https://github.com/chuanjinpang/win10_idd_xfz1986_usb_graphic_driver_display

如需修改，请自行下载后重新编译。

如何使用

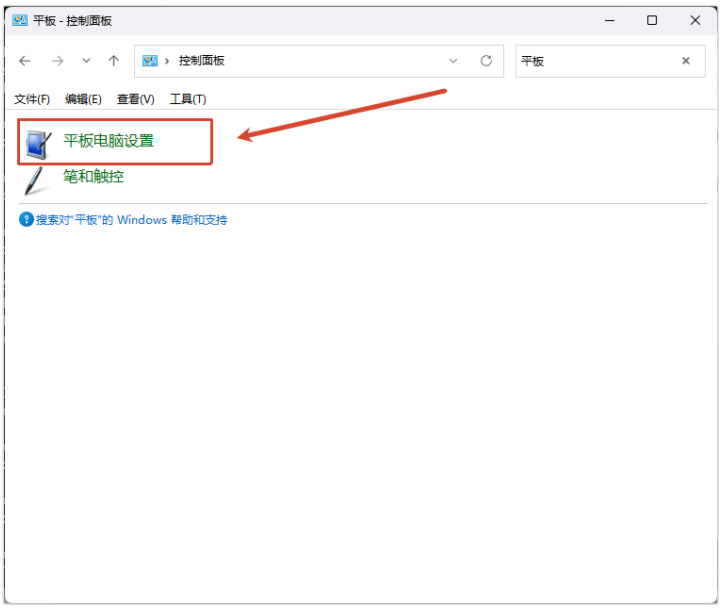
1. 安装 `resources\USB_Graphic\xfz1986_usb_graphic_250224_rc_sign.exe` 签名驱动
2. 安装完毕后，在显示适配器下面出现一个新的显示器，即安装成功



注意事项

> ⚠️ ***注意：*** *本工程要求使用* ***RT-Thread Studio 2.2.9*** *或以上版本。*

- 该驱动通过 VENDOR 接口与设备进行通信，支持多种分辨率和图片格式，通过接口字符描述符来控制，具体参考：https://github.com/chuanjinpang/win10_idd_xfz1986_usb_graphic_driver_display/blob/main/README.md
- 驱动仅支持 Windows 10 及 Windows 11 系统，其他系统请自行测试。
- 如果发现触摸报点有误，触摸后却在操控其他的屏幕，可以按照如下步骤进行调整：


平板 - 控制面板
控制面板
平板
文件(F)
编辑(E)
查看(V)
工具(T)
平板电脑设置
笔和触控
搜索对"平板"的 Windows 帮助和支持

平板电脑设置
显示
配置
配置你的笔和触控显示。
设置(S)...
选择屏幕旋转顺序转至方向
确定
取消
应用(A)

启动流程

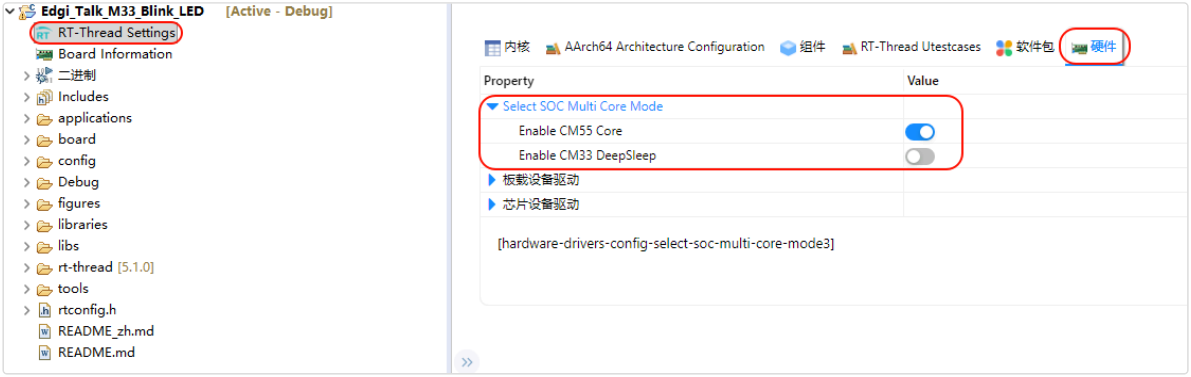
M55 依赖 M33 启动流程，烧录顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+--------------------+
|       M55          |
|  (应用处理器启动)   |
+--------------------+
```

说明

- 本工程面向 M55 核心的 USB 设备模式。
- M55 USB 角色切换示例请参考 projects/Edgi_Talk_CherryUSB/Edgi_Talk_M55_USB_RoleSwitch/README.md。
- M33 USB 角色切换示例请参考 projects/Edgi_Talk_CherryUSB/Edgi_Talk_M33_USB_RoleSwitch/README.md。
- 若 M55 工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程。
- 在 **M33 工程** 中开启 CM55：

```
RT-Thread Settings -> 硬件 -> select SOC Multi Core Mode -> Enable CM55 Core
```


Edgi_Talk_M33_Blink_LED [Active - Debug]
RT-Thread Settings
Board Information
二进制
Includes
applications
board
config
Debug
figures
libraries
libs
rt-thread [5.1.0]
tools
rtconfig.h
README_zh.md
README.md
内核
AArch64 Architecture Configuration
组件
RT-Thread Utestcases
软件包
硬件
Property
Value
Select SOC Multi Core Mode
Enable CM55 Core
Enable CM33 DeepSleep
板载设备驱动
芯片设备驱动
[hardware-drivers-config-select-soc-multi-core-mode3]

4.2. Edgi_Talk_M33_USB_RoleSwitch CherryUSB 示例工程

中文 | **English**

简介

本工程在 **Edgi-Talk M33 核心** 上集成 **CherryUSB**，用于 **USB 设备模式**，并使用 Infineon **DWC2** IP。

默认配置

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- 设备模板：**CDC**（用户应用）

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件。
3. 使用 Type-C USB 接口连接主机，进行 USB 设备枚举。
4. 可以使用 CherryUSB 测试脚本对 CDC 设备进行性能测试。

配置方法（切换模式）

通过板级 Kconfig 入口切换 USB 角色。USB Host/Device 的选择不需要重新生成图形化板级配置。

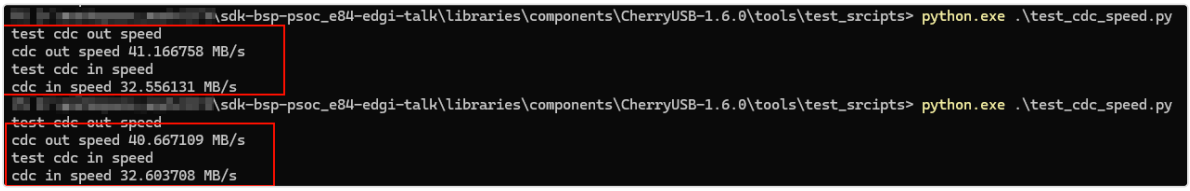
```
RT-Thread Settings ->
Hardware Drivers Config ->
Onboard Peripheral Drivers ->
Enable USB ->
USB role
```

- **Device CDC ACM**：选择 `RT_CHERRYUSB_DEVICE`、高速模式、Infineon DWC2、CDC ACM 以及 CDC ACM 设备模板。
- **Host MSC**：选择 `RT_CHERRYUSB_HOST`、Infineon DWC2、MSC、DFS 以及 ElmFat。

如果 IP 或类驱动需要额外参数，请修改：

- `libraries/Common/board/ports/usb/usb_config.h`

USB CDC 设备测试效果



主机插入 U 盘效果

```
***************** PSOC Edge MCU: CM33 Secure Mode*****************
PSRAM Cache is Enabled
PSRAM init successful
***************** PSOC Edge MCU: CM33 Secure Mode Exit*****************


 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Jan 28 2026 13:27:14
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
This core is cortex-m33
msh />usbh_init 0 0x44900000
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:05e3,idProduct:0610,bcdDevice:0663
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading hub class driver on interface 0
[I/usbh_hub] Ep=81 Attr=03 Mps=1 Interval=12 Mult=00
[I/usbh_hub] Register HUB Class:/dev/hub2
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 2 connected
[I/usbh_core] New device found,idVendor:0930,idProduct:6545,bcdDevice:0110
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading msc class driver on interface 0
[I/usbh_msc] Get max LUN:1
[I/usbh_msc] Ep=81 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Ep=02 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Register MSC Class:/dev/sda
[I/usbh_msc] Capacity info:
[I/usbh_msc] Block num:121995264,block size:512
udisk: /dev/sda mount successfully
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 4 connected
[I/usbh_core] New device found,idVendor:0bda,idProduct:8153,bcdDevice:3100
[I/usbh_core] The device has 2 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[E/usbh_core] Do not support Class:0xff, Subclass:0xff, Protocl:0x00 on interface 0

msh />
msh />ls
Directory /:
grub                <DIR>
ventoy              <DIR>
EFI                 <DIR>
ENROLL_THIS_KEY_IN_M829
tool                <DIR>
System Volume Inform<DIR>
msh />
```

启动流程

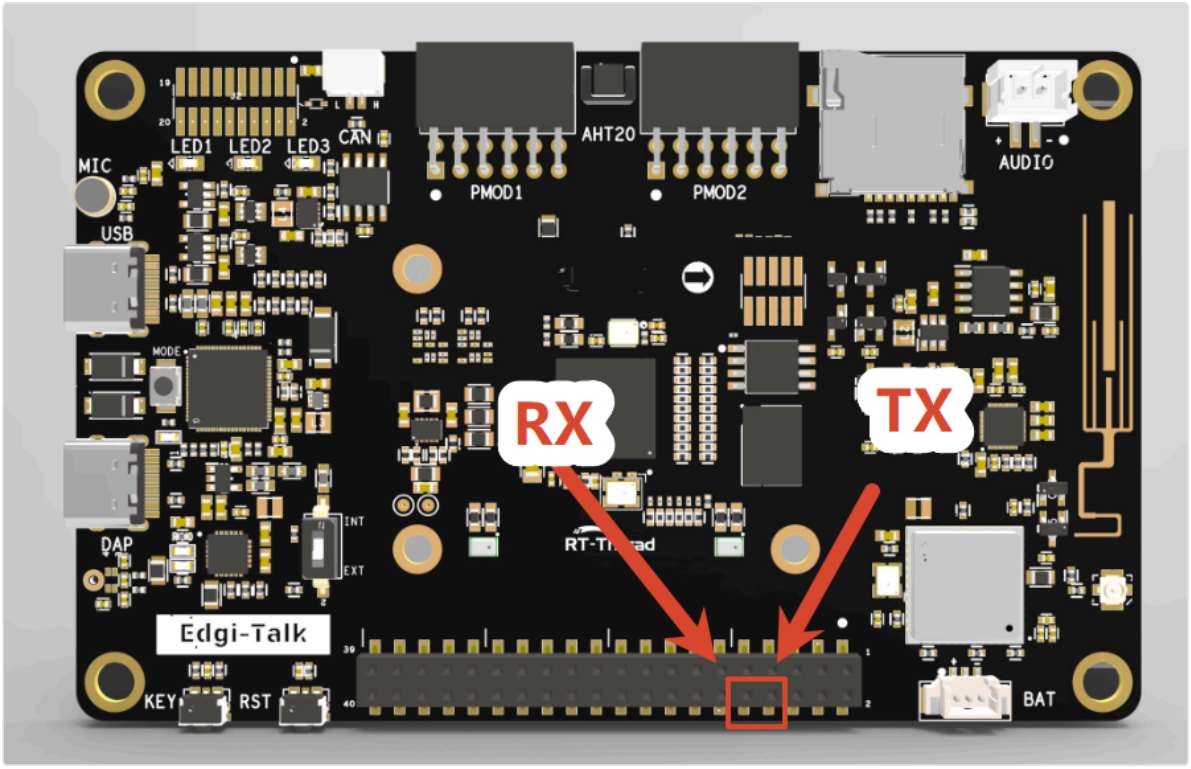
M55 依赖 M33 启动流程，烧录顺序如下：

```
+------------------+
|   Secure M33     |
|    (安全核心)    |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核心)   |
+------------------+
          |
          v
+-------------------+
|       M55         |
|   (应用处理器)    |
+-------------------+
```

说明

> *注意：本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- M33 工程的串口日志不会通过板载 DAP 虚拟串口直接输出。查看 `msh />` 和 demo 日志时，需要额外连接 USB 转串口硬件，例如 CH340。
  位置如图下所示，RX 接串口硬件的 TX，TX 接串口硬件的 RX，上位机波特率 115200：



- 本工程面向 M33 核心，同时支持 USB 设备 CDC ACM 和 USB 主机 MSC 两种角色。
- M55 USB 角色切换示例请参考 projects/Edgi_Talk_CherryUSB/Edgi_Talk_M55_USB_RoleSwitch/README.md。

- 如需运行 M55 工程，建议先烧写 **Edgi_Talk_M33_Template**。该工程是干净的 M33 工程，适合作为 M55 启动前的基础固件。

4.3. Edgi_Talk_M55_USB_RoleSwitch CherryUSB 示例工程

中文 | **English**

简介

本工程在 **Edgi-Talk M55 核心** 上集成 **CherryUSB**，默认配置为 **USB 设备 CDC ACM 模式**，并且可以通过板级 Kconfig 切换到 **USB 主机 MSC 模式**，不需要重新生成图形化板级配置文件。

默认配置

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- `RT_CHERRYUSB_DEVICE_CDC_ACM = y`
- 设备模板：**CDC ACM**

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件。
3. 默认设备模式下使用 Type-C USB 接口连接主机，进行 USB 设备枚举。
4. 切换到主机 MSC 模式后，可连接 U 盘进行主机操作。

配置方法（切换模式）

通过板级 Kconfig 入口切换 USB 角色。USB Host/Device 的选择不需要重新生成图形化板级配置。

```
RT-Thread Settings ->
Hardware Drivers Config ->
Onboard Peripheral Drivers ->
Enable USB ->
USB role
```

- **Device CDC ACM**：选择 `RT_CHERRYUSB_DEVICE`、高速模式、Infineon DWC2、CDC ACM 以及 CDC ACM 设备模板。
- **Host MSC**：选择 `RT_CHERRYUSB_HOST`、Infineon DWC2、MSC、DFS 以及 ElmFat。

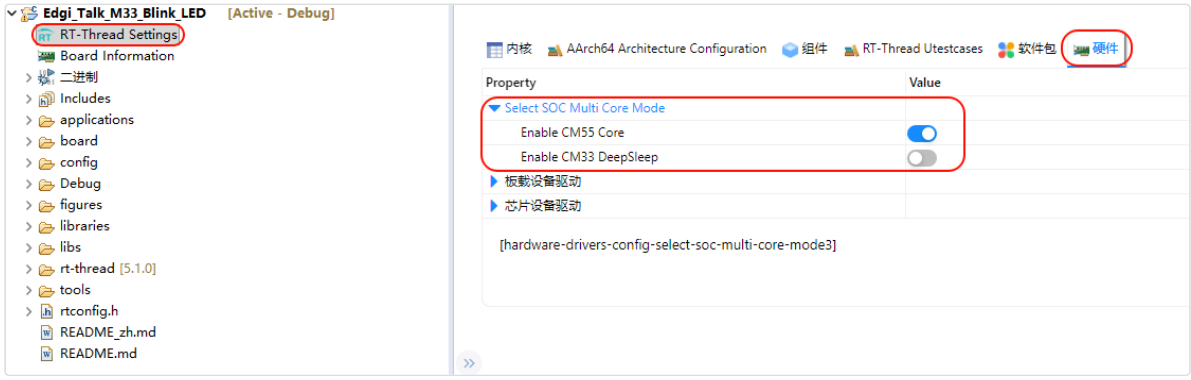
如果 IP 或类驱动需要额外参数，请修改：

- `libraries/Common/board/ports/usb/usb_config.h`

主机插入 U 盘效果

```
****************** PSOC Edge MCU: CM33 Secure Mode******************
PSRAM Cache is Enabled
PSRAM init successful
****************** PSOC Edge MCU: CM33 Secure Mode Exit******************


 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Jan 28 2026 13:27:14
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
This core is cortex-m33
msh />usbh_init 0 0x44900000
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:05e3,idProduct:0610,bcdDevice:0663
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading hub class driver on interface 0
[I/usbh_hub] Ep=81 Attr=03 Mps=1 Interval=12 Mult=00
[I/usbh_hub] Register HUB Class:/dev/hub2
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 2 connected
[I/usbh_core] New device found,idVendor:0930,idProduct:6545,bcdDevice:0110
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading msc class driver on interface 0
[I/usbh_msc] Get max LUN:1
[I/usbh_msc] Ep=81 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Ep=02 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Register MSC Class:/dev/sda
[I/usbh_msc] Capacity info:
[I/usbh_msc] Block num:121995264,block size:512
udisk: /dev/sda mount successfully
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 4 connected
[I/usbh_core] New device found,idVendor:0bda,idProduct:8153,bcdDevice:3100
[I/usbh_core] The device has 2 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[E/usbh_core] Do not support Class:0xff, Subclass:0xff, Protocl:0x00 on interface 0

msh />
msh />ls
Directory /:
grub                <DIR>
ventoy              <DIR>
EFI                 <DIR>
ENROLL_THIS_KEY_IN_M829
tool                <DIR>
System Volume Inform<DIR>
msh />
```

CM55 启用配置



启动流程

M55 依赖 M33 启动流程，烧录顺序如下：

```
+------------------+
|   Secure M33     |
|    (安全核心)    |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核心)   |
+------------------+
          |
          v
+------------------+
|       M55        |
|   (应用处理器)   |
+------------------+
```

说明

> *注意：本工程要求使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- 本工程面向 M55 核心，同时支持 USB 设备 CDC ACM 和 USB 主机 MSC 两种角色。
- M33 USB 角色切换示例请参考 projects/Edgi_Talk_CherryUSB/Edgi_Talk_M33_USB_RoleSwitch/README.md。
- 如果 M55 工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程。
- 在 **M33 工程** 中启用 CM55：

```
RT-Thread Settings -> Hardware -> select SOC Multi Core Mode -> Enable CM55 Core
```

4.4. Edgi_Talk_M55_USB_UVC CherryUSB 示例工程

中文 | English

简介

本工程运行在 **Edgi-Talk M55 核心**，基于 **CherryUSB Host + Infineon DWC2**，并默认开启 **USB Video Class (UVC) 主机类驱动**。

工程上电后会自动调用 `usbh_initialize(0, USBHS_BASE, NULL)` 初始化 USB Host，主循环中闪烁绿色 LED（P16_6）作为运行指示。

默认配置

当前工程关键宏（见 `rtconfig.h`）：

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_HOST = y`
- `RT_CHERRYUSB_HOST_DWC2_INFINEON = y`
- `RT_CHERRYUSB_HOST_MSC = y`
- `RT_CHERRYUSB_HOST_VIDEO = y`

说明：该工程已具备 UVC 主机枚举所需的类驱动配置，便于后续对接 UVC 设备应用逻辑。

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件。
3. 将 UVC 摄像头或其他 USB 设备连接到 Type-C 口进行测试。

配置方法

在 RT-Thread Studio 中打开：

```
RT-Thread Settings -> USB -> CherryUSB
```

建议确认：

- 开启 `RT_CHERRYUSB_HOST`
- 在 Host IP 中选择 `RT_CHERRYUSB_HOST_DWC2_INFINEON`
- 开启 `RT_CHERRYUSB_HOST_VIDEO`（UVC）
- 按需保留/关闭 `MSC` 等其它 Host 类驱动

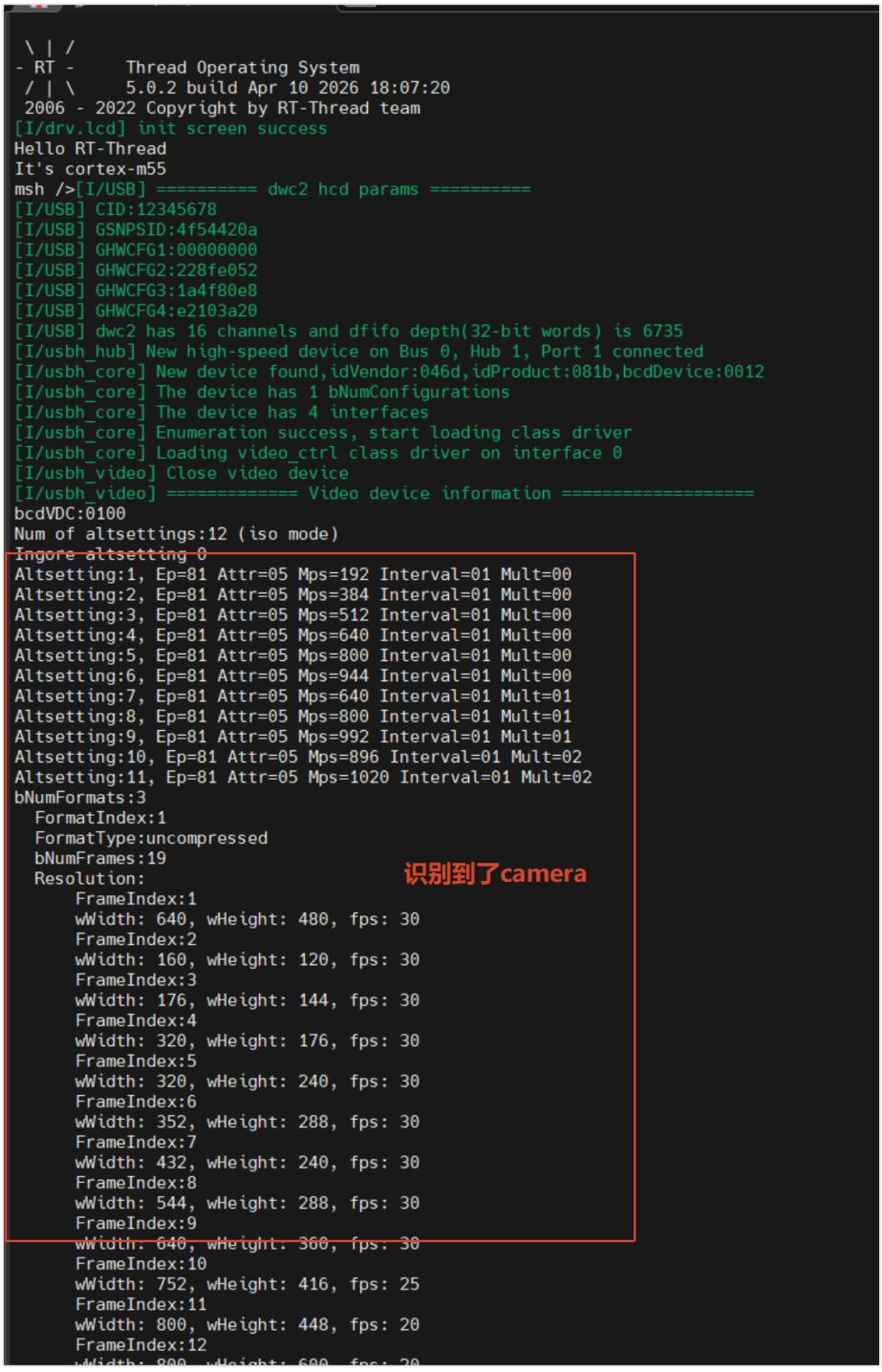
若需调整 USB 参数，可修改：

- `libraries/Common/board/ports/usb/usb_config.h`

运行效果

1. 插入USB摄像头到开发板USB-OTG接口

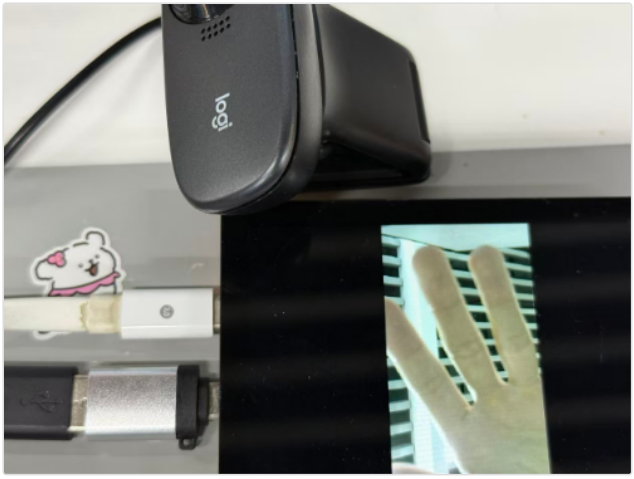
2. 系统启动会自动识别摄像头（分辨率，格式等信息）




```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Apr 10 2026 18:07:20
 2006 - 2022 Copyright by RT-Thread team
[I/drv.lcd] init screen success
Hello RT-Thread
It's cortex-m55
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:046d,idProduct:081b,bcdDevice:0012
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 4 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading video_ctrl class driver on interface 0
[I/usbh_video] Close video device
[I/usbh_video] ============= Video device information ==================
bcdVDC:0100
Num of altsettings:12 (iso mode)
Ingore altsetting 0
Altsetting:1, Ep=81 Attr=05 Mps=192 Interval=01 Mult=00
Altsetting:2, Ep=81 Attr=05 Mps=384 Interval=01 Mult=00
Altsetting:3, Ep=81 Attr=05 Mps=512 Interval=01 Mult=00
Altsetting:4, Ep=81 Attr=05 Mps=640 Interval=01 Mult=00
Altsetting:5, Ep=81 Attr=05 Mps=800 Interval=01 Mult=00
Altsetting:6, Ep=81 Attr=05 Mps=944 Interval=01 Mult=00
Altsetting:7, Ep=81 Attr=05 Mps=640 Interval=01 Mult=01
Altsetting:8, Ep=81 Attr=05 Mps=800 Interval=01 Mult=01
Altsetting:9, Ep=81 Attr=05 Mps=992 Interval=01 Mult=01
Altsetting:10, Ep=81 Attr=05 Mps=896 Interval=01 Mult=02
Altsetting:11, Ep=81 Attr=05 Mps=1020 Interval=01 Mult=02
bNumFormats:3
  FormatIndex:1
  FormatType:uncompressed
  bNumFrames:19
  Resolution:
      FrameIndex:1
      wWidth: 640, wHeight: 480, fps: 30
      FrameIndex:2
      wWidth: 160, wHeight: 120, fps: 30
      FrameIndex:3
      wWidth: 176, wHeight: 144, fps: 30
      FrameIndex:4
      wWidth: 320, wHeight: 176, fps: 30
      FrameIndex:5
      wWidth: 320, wHeight: 240, fps: 30
      FrameIndex:6
      wWidth: 352, wHeight: 288, fps: 30
      FrameIndex:7
      wWidth: 432, wHeight: 240, fps: 30
      FrameIndex:8
      wWidth: 544, wHeight: 288, fps: 30
      FrameIndex:9
      wWidth: 640, wHeight: 360, fps: 30
      FrameIndex:10
      wWidth: 752, wHeight: 416, fps: 25
      FrameIndex:11
      wWidth: 800, wHeight: 448, fps: 20
      FrameIndex:12
```

3. 在MSH终端，输入 `usbh_uvc_start 0 320 240` （0 = YUYV, 1 = MJPEG，320x240 分辨率）

需要注意：需要根据实际的USB摄像头支持的情况来选择，分辨率最大仅支持432x240



4. 可以查看串口LOG信息，LOG会1S打印一次帧率信息

```
msh />usbh_uvc_start 0 320 240
[I/uvc_app] UVC start: 320x240 format=yuyv
[I/USB] uvc rx fifo limit:2172
msh />[I/uvc_app] UVC fps:0 disp:0 drop:0 urb:0 eof:0 fid:0 err:0 empty:0 data:0 hdronly:0
 payload:0 done:0 last_off:0
[I/uvc_disp] LCD: 512x800 16bpp fb=0x262c8000 size=819200
[I/usbh_video] Probe fmt:1 frame:5 interval:333333 maxframe:153600 maxpayload:1600
[I/usbh_video] Commit fmt:1 frame:5 interval:333333 maxframe:153600 maxpayload:1600
[I/usbh_video] Ep=81 Attr=05 Mps=992 Interval=01 Mult=01
[I/usbh_video] Open video and select formatidx:1, frameidx:5, altsetting:9
[I/USB] uvc selected size:320x240
[I/USB] uvc mps :1600
[I/USB] uvc iso urb depth:32 packets (51200 bytes)
[I/USB] start uvc stream...
[I/uvc_app] frame #1  size=153600 fmt=yuyv
[I/uvc_app] UVC fps:29 disp:0 drop:17 urb:656 eof:30 fid:30 err:0 empty:0 data:6054 hdro
y:1713 payload:4559700 done:29 last_off:153600
[I/uvc_app] frame #31  size=153600 fmt=yuyv
[I/uvc_app] UVC fps:29 disp:11 drop:72 urb:1381 eof:119 fid:119 err:0 empty:0 data:24167 h
dronly:6800 payload:18272000 done:118 last_off:153600
[I/uvc_app] frame #61  size=153600 fmt=yuyv
```

启动流程

M55 依赖 M33 启动流程，建议烧录顺序如下：

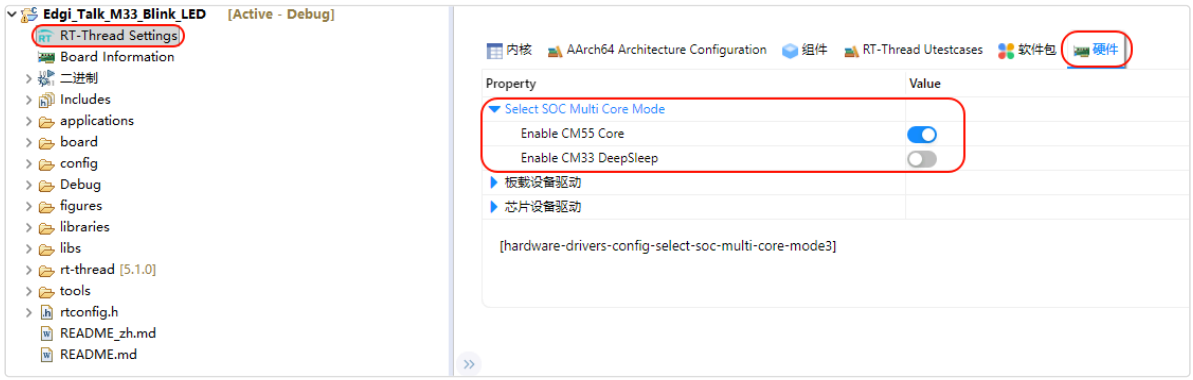
```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+--------------------+
|       M55          |
|  (应用处理器启动)   |
+--------------------+
```

说明

> ***注意：** 推荐使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- 若 M55 工程无法正常运行，建议先编译并烧录 `Edgi_Talk_M33_Template`。
- 在 M33 工程中开启 CM55：

```
RT-Thread Settings -> 硬件 -> select SOC Multi Core Mode -> Enable CM55 Core
```



4.5. Edgi_Talk_USB_Deepcraft 示例工程

中文 | **English**

简介

本工程运行在 **Edgi-Talk M55 核心**，通过 **CherryUSB Device CDC ACM** 将板端传感器数据以 **Protocol Buffers 协议**对接到上位机 Deepcraft/Imagimob 流式工具链。

程序启动后会：

1. 创建协议上下文（`protocol_create`）并生成 16 字节设备序列号。
2. 注册设备驱动（当前包含麦克风与 LSM6DS3 IMU）。
3. 初始化 USB CDC 设备并等待主机连接。
4. 在主循环中执行 `protocol_process_request`，处理主机请求与数据流。

默认配置

当前工程关键宏（见 `rtconfig.h`）：

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- `RT_CHERRYUSB_DEVICE_CDC_ACM = y`
- `RT_CHERRYUSB_DEVICE_TEMPLATE_CDC_ACM = y`

USB 标识（见 `applications/deepcraft/deepcraft_usbd.c`）：

- VID: `0x058B`
- PID: `0x027D`
- Product String: `Imagimob Streaming Device`

已对接设备能力

当前 `system_load_device_drivers()` 中加载：

- **Microphone（PDM/PCM）**
- 选项：增益、单/双声道、采样率
- 流类型：`S16` 音频输出流
- **LSM6DS3 六轴传感器**
- 选项：采样率、加速度量程、陀螺仪量程、输出模式
- 支持 Combined / Split / Only Accel / Only Gyro

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。

2. 通过 KitProg3 (DAP) 下载固件。

3. 使用 Type-C 将板卡连接到主机，等待 CDC 设备枚举。

启动成功后串口会打印：

- `deepcraft: ready, waiting for host commands.`

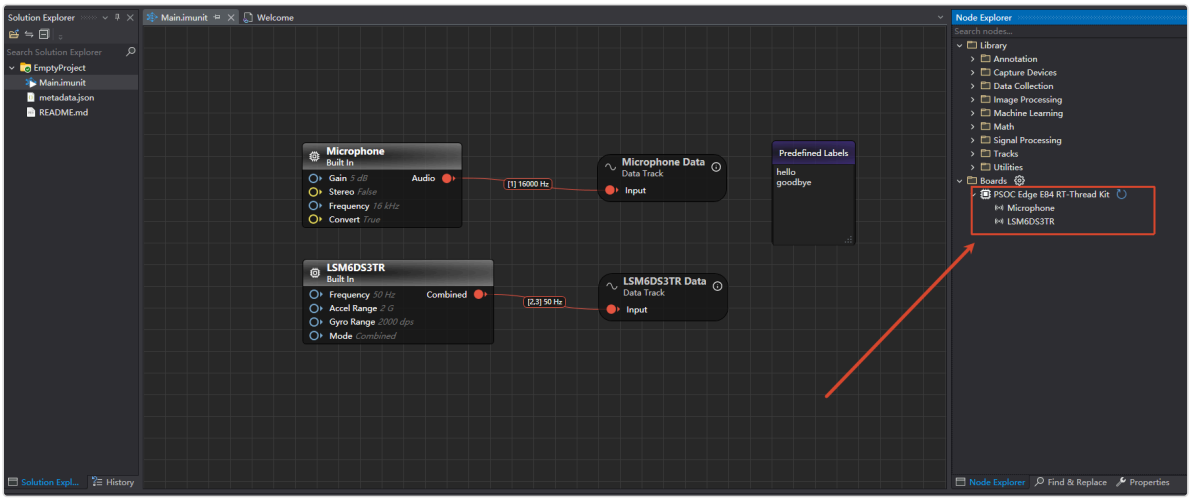
```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Apr 13 2026 11:34:52
 2006 - 2022 Copyright by RT-Thread team
[I/I2C] I2C bus [i2c0] registered
[I/i2s] ES8388 init success.
[I/drv.mic] audio pdm registered.

[I/drv.mic] !!!Note: pdm depends on i2s0, they share clock.

lsm6ds3tr: init done on i2c0
msh >[I/USB] ========== dwc2 dcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 9 endpoints and dfifo depth(32-bit words) is 6735
[I/USB] fifo0 size:0010, offset:021f
[I/USB] fifo1 size:0300, offset:022f
[I/USB] fifo2 size:0300, offset:052f
[I/USB] fifo3 size:0300, offset:082f
[I/USB] fifo4 size:0300, offset:0b2f
[I/USB] fifo5 size:0300, offset:0e2f
[I/USB] fifo6 size:0300, offset:112f
[I/USB] fifo7 size:0300, offset:142f
[I/USB] fifo8 size:0300, offset:172f
deepcraft: ready, waiting for host commands.
```

使用说明

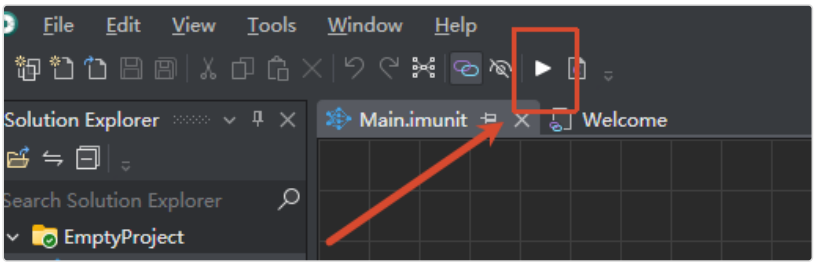
1. 下载并安装deepcraftstudio工具，详情参考教程：Install Download Studio

2. 创建空白项目，详情参考教程：Real-Time Data Streaming with PSOC™ Edge E84 HMI

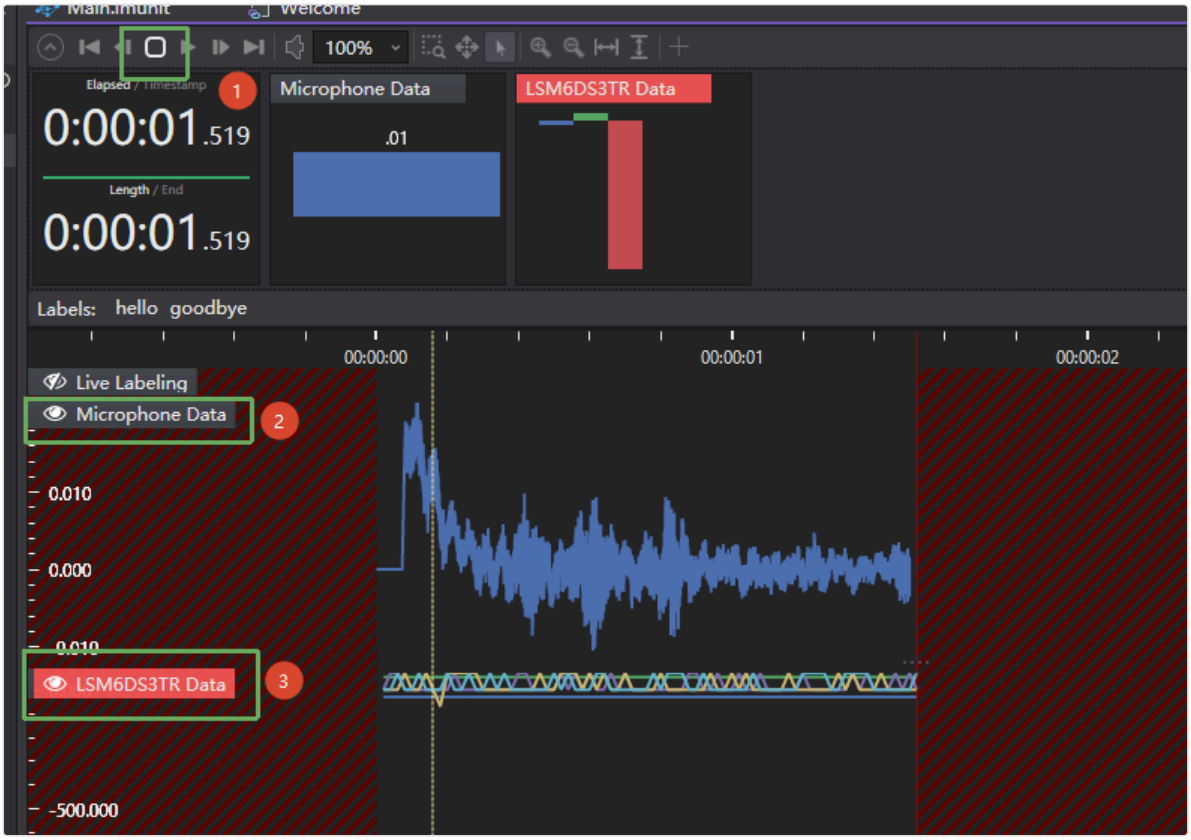
3. 开发板和电脑使用USB数据线连接后，在deepcraftstudio中可以查看到设备



4. 点击编译运行选项，进入采集窗口界面



5. 点击录制选项后，可以看到麦克风和3轴数据实时的被显示出来了



> *更多教程文档请参考：Data Collection Microphone*

主机通信说明

- 传输通道：USB CDC ACM
- 编码格式：nanopb / protobuf
- 主循环错误处理：
- 当解析请求失败（`PROTOCOL_STATUS_FAILED_TO_DECODE_REQUEST`）时，会调用 `deepcraft_usbd_resync_rx()` 清空接收环形缓冲并重同步。

启动流程

M55 依赖 M33 启动流程，建议烧录顺序如下：

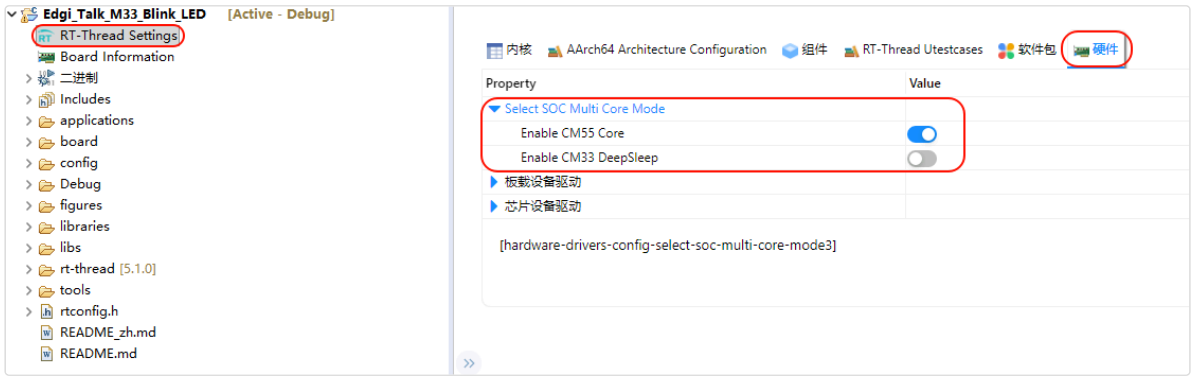
```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+--------------------+
|       M55          |
|  (应用处理器启动)   |
+--------------------+
```

说明

> ***注意：** 推荐使用 **RT-Thread Studio 2.2.9** 或以上版本。*

- 若 M55 工程无法正常运行，建议先编译并烧录 `Edgi_Talk_M33_Template`。
- 在 M33 工程中开启 CM55：

```
RT-Thread Settings -> 硬件 -> select SOC Multi Core Mode -> Enable CM55 Core
```



5. IPC篇

5.1. Edgi_Talk_M33_IPC 双核通信示例工程

中文 | **English**

简介

本工程在 **Edgi-Talk M33 核心**上实现 **IPC（Inter-Processor Communication）** 双核通信功能，演示了 **Cortex-M33** 与 **Cortex-M55** 之间的消息传递机制。

M33 作为**发送端**，定期向 M55 发送 "Hello M33" 消息，并接收 M55 的 "Hello M55" 回复。

默认配置

- 通信频率：1000ms
- 消息格式：ASCII 编码的文本消息
- 使用 Infineon PSoC E84 片内 IPC Pipe 硬件

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件到 M33 核心。
3. 同时需要编译并下载 M55 工程才能完成双核通信。
4. 连接串口查看通信日志，输出如下如图：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Mar 26 2026 16:14:12
 2006 - 2022 Copyright by RT-Thread team

=======================================
   RT-Thread on Cortex-M33 Core
=======================================
Command: ipc_test_run
Status:  Running
=======================================

msh />ipc_test_run

=======================================
[M33] IPC Demo Started
---------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
=======================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     5403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     5413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     6403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     6413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     3 | Time:     7403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     3 | Time:     7413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     4 | Time:     8403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     4 | Time:     8413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     5 | Time:     9403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     5 | Time:     9413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     6 | Time:    10403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     6 | Time:    10413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     7 | Time:    11403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     7 | Time:    11413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     8 | Time:    12403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     8 | Time:    12413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     9 | Time:    13403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     9 | Time:    13413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:    10 | Time:    14403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:    10 | Time:    14413 ms
---------------------------------------
[M33] Statistics: TX=10, RX=10
```

注意事项

> ⚠️ ***注意：*** *本工程要求使用* ***RT-Thread Studio 2.2.9*** *或以上版本。*

串口连接

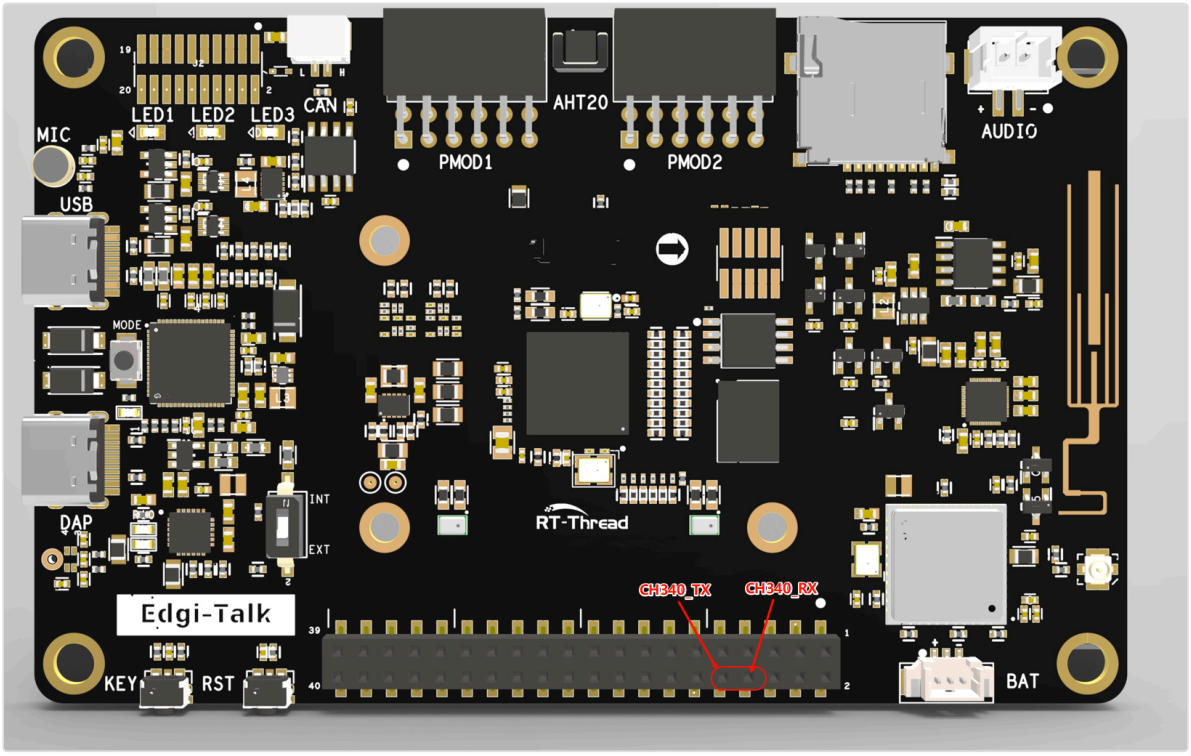
本工程使用 **UART5** 作为调试串口，需要外接 **CH340 USB 转串口模块**：

- **串口**: UART5
- **波特率**: 115200
- **数据位**: 8
- **停止位**: 1
- **校验位**: None

CH340 模块连接

请按以下方式连接 CH340 模块到开发板：

连接位置示意图：



> ⚠️ ***重要提示：***
> *1. TX 和 RX 需要交叉连接（CH340 的 TX 接开发板的 RX）*
> *2. 确保两个设备的 GND 共地*

- 如需运行 M55 工程，建议先烧写 **Edgi_Talk_M33_Template**。该工程是干净的 M33 工程，适合作为 M55 启动前的基础固件。

使用方法

1. 配置和编译

在 RT-Thread Studio 中打开：

```
RT-Thread Settings -> Hardware Drivers Config -> On-chip Peripheral Drivers
```

勾选 `Enable IPC` 选项。

2. 运行示例

第一步：启动 M55 接收端

在 M55 的串口终端输入：

```
msh> ipc_test_run
```

M55 将进入监听模式，等待来自 M33 的消息。

第二步：启动 M33 发送端

在 M33 的串口终端输入：

```
msh> ipc_test_run
```

M33 将开始每秒发送 "Hello M33" 消息，并接收 M55 的 "Hello M55" 回复。

3. 观察输出

M33 输出示例：

```
========================================
[M33] IPC Demo Started
----------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
========================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     1234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     1256 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     2234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     2256 ms
----------------------------------------
[M33] Statistics: TX=10, RX=10
----------------------------------------
```

M55 输出示例：

```
========================================
[M55] IPC Listener Running
----------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
========================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     1250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     1252 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     2250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     2252 ms
----------------------------------------
[M55] Statistics: RX=10, TX=10
----------------------------------------
```

数据协议

IPC 通信使用 `edge_rc_frame_t` 结构体：

```
typedef struct {
    uint8_t client_id;          // 客户端 ID
    uint16_t intr_mask;         // 中断掩码
    uint8_t role;               // 角色标识 (M33/M55_ECHO)
    uint32_t magic;             // 魔术字 (0x5243444DU)
    uint32_t seq;               // 序列号
    uint16_t channel[8];        // 8通道数据（存储 ASCII 消息）
```

```
    uint32_t checksum;          // 校验和
} edge_rc_frame_t;
```

消息 "Hello M33" 编码方式：

```
channel[0] = 0x4865  /* 'He' */
channel[1] = 0x6C6C  /* 'll' */
channel[2] = 0x6F20  /* 'o '  (space) */
channel[3] = 0x4D33  /* 'M3' */
channel[4] = 0x0033  /* '3' */
```

启动流程

M55 依赖 M33 启动流程，烧录顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (IPC 发送端)   |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  (IPC 接收端)     |
+-------------------+
```

硬件连接

本示例使用 Infineon PSoC E84 的片内 IPC 硬件：

- 使用 CM0P 中的 IPC Pipe 硬件
- CM33 使用 EP1 (Endpoint 1)
- CM55 使用 EP2 (Endpoint 2)
- 使用信号量进行互斥访问
- 不需要外部连线

配置参数

在 `libraries/HAL_Drivers/ipc_common.h` 中可修改以下参数：

- `EDGE_IPC_FRAME_POOL_SIZE`：发送缓冲池大小（默认 64）
- `EDGE_IPC_RX_QUEUE_SIZE`：接收队列大小（默认 128）
- `EDGE_IPC_SEMA_RETRY_MAX`：信号量重试次数（默认 2）

5.2. Edgi_Talk_M55_IPC 双核通信示例工程

中文 | **English**

简介

本工程在 **Edgi-Talk M55 核心**上实现 **IPC（Inter-Processor Communication）** 双核通信功能，演示了 **Cortex-M55** 与 **Cortex-M33** 之间的消息传递机制。

M55 作为**接收端和响应端**，接收来自 M33 的 "Hello M33" 消息，并回复 "Hello M55"。

默认配置

- 响应模式：自动回复
- 消息格式：ASCII 编码的文本消息
- 使用 Infineon PSoC E84 片内 IPC Pipe 硬件

编译与下载

1. 使用 RT-Thread Studio 或 SCons 编译工程。
2. 通过 KitProg3 (DAP) 下载固件到 M55 核心。
3. 同时需要编译并下载 M33 工程才能完成双核通信。
4. 连接串口查看通信日志，输出如下如图：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Mar 26 2026 16:12:28
 2006 - 2022 Copyright by RT-Thread team

=======================================
   RT-Thread on Cortex-M55 Core
=======================================
Command: ipc_test_run
Status:  Running
=======================================

msh />ipc_test_run
msh />
=======================================
[M55] IPC Listener Running
---------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
=======================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     5429 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     5429 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     6429 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     6429 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     3 | Time:     7439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     3 | Time:     7439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     4 | Time:     8439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     4 | Time:     8439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     5 | Time:     9439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     5 | Time:     9439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     6 | Time:    10439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     6 | Time:    10439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     7 | Time:    11449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     7 | Time:    11449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     8 | Time:    12449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     8 | Time:    12449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     9 | Time:    13449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     9 | Time:    13449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:    10 | Time:    14449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:    10 | Time:    14449 ms
---------------------------------------
[M55] Statistics: RX=10, TX=10
```

使用方法

1. 配置和编译

在 RT-Thread Studio 中打开：

```
RT-Thread Settings -> Hardware Drivers Config -> On-chip Peripheral Drivers
```

勾选 `Enable IPC` 选项。

2. 运行示例

在 M55 的串口终端输入：

```
msh> ipc_test_run
```

M55 将启动监听线程，等待来自 M33 的消息并自动回复。

3. 观察输出

M55 输出示例：

```
========================================
[M55] IPC Listener Running
----------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
========================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     1250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     1252 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     2250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     2252 ms
----------------------------------------
[M55] Statistics: RX=10, TX=10
----------------------------------------
```

对应的 M33 输出：

```
========================================
[M33] IPC Demo Started
----------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
========================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     1234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     1256 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     2234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     2256 ms
----------------------------------------
[M33] Statistics: TX=10, RX=10
----------------------------------------
```

数据协议

IPC 通信使用 `edge_rc_frame_t` 结构体：

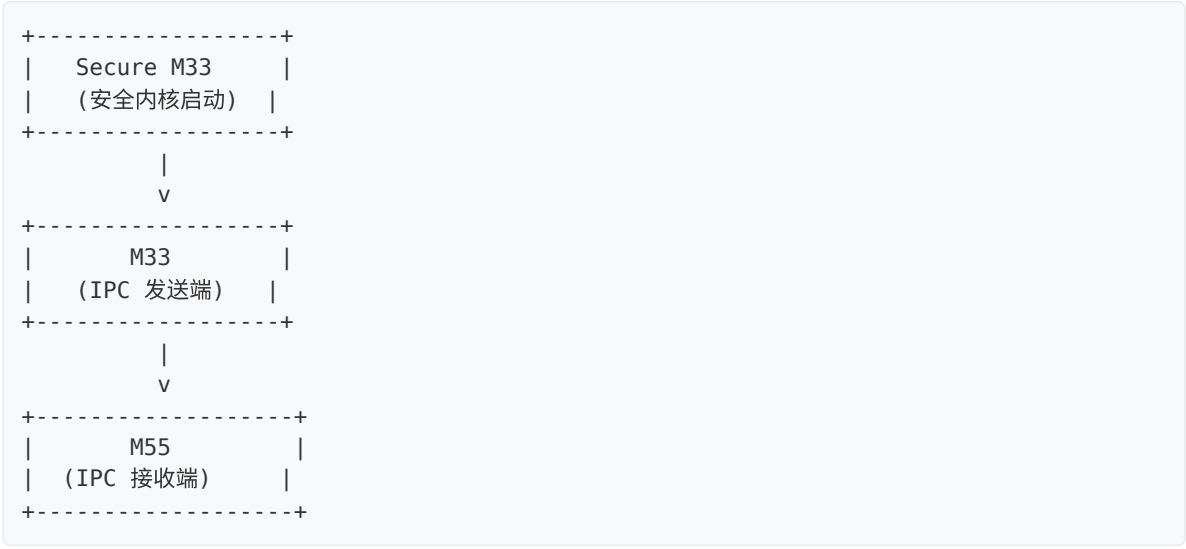
```
typedef struct {
    uint8_t client_id;          // 客户端 ID
    uint16_t intr_mask;         // 中断掩码
    uint8_t role;               // 角色标识 (M33/M55_ECHO)
    uint32_t magic;             // 魔术字 (0x5243444DU)
    uint32_t seq;               // 序列号
    uint16_t channel[8];        // 8通道数据（存储 ASCII 消息）
    uint32_t checksum;          // 校验和
} edge_rc_frame_t;
```

消息 "Hello M55" 编码方式：

```
channel[0] = 0x4865  /* 'He' */
channel[1] = 0x6C6C  /* 'll' */
channel[2] = 0x6F20  /* 'o '  (space) */
channel[3] = 0x4D35  /* 'M5' */
channel[4] = 0x0035  /* '5' */
```

启动流程

M55 依赖 M33 启动流程，烧录顺序如下：



硬件连接

本示例使用 Infineon PSoC E84 的片内 IPC 硬件：

- 使用 CM0P 中的 IPC Pipe 硬件
- CM33 使用 EP1 (Endpoint 1)
- CM55 使用 EP2 (Endpoint 2)
- 使用信号量进行互斥访问
- 不需要外部连线

工作原理

1. **接收消息**：M55 在独立线程中循环读取 IPC 设备
2. **验证消息**：检查魔术字、角色标识和校验和
3. **解析消息**：验证是否为 "Hello M33" 消息
4. **发送回复**：构造 "Hello M55" 消息并回复给 M33
5. **统计信息**：每处理 10 条消息打印一次统计

配置参数

在 `libraries/HAL_Drivers/ipc_common.h` 中可修改以下参数：

- `EDGE_IPC_FRAME_POOL_SIZE`：发送缓冲池大小（默认 64）
- `EDGE_IPC_RX_QUEUE_SIZE`：接收队列大小（默认 128）
- `EDGE_IPC_SEMA_RETRY_MAX`：信号量重试次数（默认 2）

6. AI篇

6.1. Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision

English | 中文（当前）

1. 项目简介

本工程是 `PSOC Edge MCU: Machine learning - DEEPCRAFT deploy vision` 的 **RT-Thread 实现版本**，运行在 Edgi-Talk 的 **CM55** 核心上，使用：

- CherryUSB Host (DWC2, Infineon)
- UVC USB 摄像头采集
- DEEPCRAFT/TFLM 模型推理
- LCD 实时显示检测框与标签

工程目标与 ModusToolbox 示例一致：实时视觉目标检测 + 串口日志输出；区别在于系统框架切换为 RT-Thread（MSH、SCons、RT-Thread Studio）。

---

2. 与原始 ModusToolbox 示例的主要差异

- OS/工程体系：ModusToolbox -> RT-Thread 工程组织与配置方式
- 运行控制：通过 MSH 命令启动/停止 UVC 流（`usbh_uvc_start` / `usbh_uvc_stop`）
- USB 应用层：位于 `libraries/Common/board/ports/usb`
- AI 应用层：位于 `libraries/Common/deepcraft_ai` 与 `applications/uvc_ai_app.c`
- 模型文件已单独整理到：
- `libraries/Common/deepcraft_ai/model/model.c`
- `libraries/Common/deepcraft_ai/model/model.h`

---

3. 目录说明（关键路径）

- 应用入口：`applications/main.c`
- AI 应用逻辑：`applications/uvc_ai_app.c`
- UVC 应用层：`libraries/Common/board/ports/usb/usbh_uvc_app.c`
- UVC 流处理：`libraries/Common/board/ports/usb/usbh_uvc_stream.c`
- 模型文件：`libraries/Common/deepcraft_ai/model/model.c`、`libraries/Common/deepcraft_ai/model/model.h`
- AI 核心处理：`libraries/Common/deepcraft_ai/src/uvc_ai.c`

---

4. 环境与依赖

- RT-Thread Studio（推荐 2.2.9 或更新）
- GNU Arm 工具链（支持 Cortex-M55 + Helium/MVE）
- 板卡：PSOC Edge E84 系列（Edgi-Talk 对应硬件）
- 摄像头：UVC USB 摄像头（建议 320x240 YUYV）
- 显示：工程默认 LCD 配置

> *说明：本工程依赖 M33 启动链路使能 CM55。若 CM55 不启动，请先确认 M33 工程已正确配置并烧录。*

---

5. 关键配置项

在 `rtconfig.h` / `.config` 中建议确认：

- `RT_USING_CHERRYUSB`
- `RT_CHERRYUSB_HOST`
- `RT_CHERRYUSB_HOST_DWC2_INFINEON`
- `RT_CHERRYUSB_HOST_VIDEO`
- `BSP_USING_DEEPCRAFT_AI`
- `RT_USING_MSH`、`RT_USING_FINSH`

如果你需要修改 `usbh_uvc_stream.c` / `usbh_uvc_port.c` 并让改动生效，请开启：

- `RT_CHERRYUSB_HOST_BUILD_FROM_SOURCE`

---

6. 编译与下载

6.1 RT-Thread Studio

1. 导入 `projects/Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision`
2. 检查配置（menuconfig/Settings）
3. Build
4. 通过 KitProg3 下载

6.2 命令行（SCons）

在工程目录执行：

```
scons -j8
```

生成产物如：

- `rt-thread.elf`
- `rtthread.hex`

---

7. 运行步骤

1. 串口连接，115200 8N1
2. 接入 LCD 与 UVC 摄像头
3. 上电启动后在 MSH 执行：

```
usbh_uvc_start 0 320 240
```

参数说明：

- `0` = YUYV
- `1` = MJPEG

> *注意：AI 模式会强制使用 `YUYV + 320x240`（即使输入其他参数也会被修正）。*

停止命令：

```
usbh_uvc_stop
```

7.1 运行效果截图

- 图 1：系统运行与检测显示

- 图 2：摄像头画面与检测框叠加

- 图 3：终端日志与识别结果输出

---

8. 日志与现象

- UVC 状态日志：
- `UVC fps:...`
- `start uvc stream...`
- AI 日志：
- `AI inference ...`
- `AI #... bbox=[...]

---

9. 异常处理说明

若出现：

- `uvc abort1` / `uvc abort2`

当前实现会触发简化恢复流程（自动重启 UVC 流）并打印：

- `UVC stream restart (...)`

如频繁出现，可优先排查：

- 摄像头供电/线材
- UVC 带宽与分辨率配置
- 应用层负载（显示刷新、AI 推理、日志量）

---

10. 模型替换说明

本工程已采用“模型单独目录”方式管理（参考官方示例结构）：

- `libraries/Common/deepcraft_ai/model/model.c`
- `libraries/Common/deepcraft_ai/model/model.h`

替换模型时，请保持导出接口与现有调用一致（如 `IMAI_init / IMAI_compute / IMAI_finalize`），并重新编译工程。

---

11. 启动链路（多核）

推荐烧录顺序与链路：

1. CM33 Secure
2. CM33 Non-Secure（使能 CM55）
3. CM55 本工程

若 CM55 无法运行，请先确认 M33 工程中已启用 CM55。

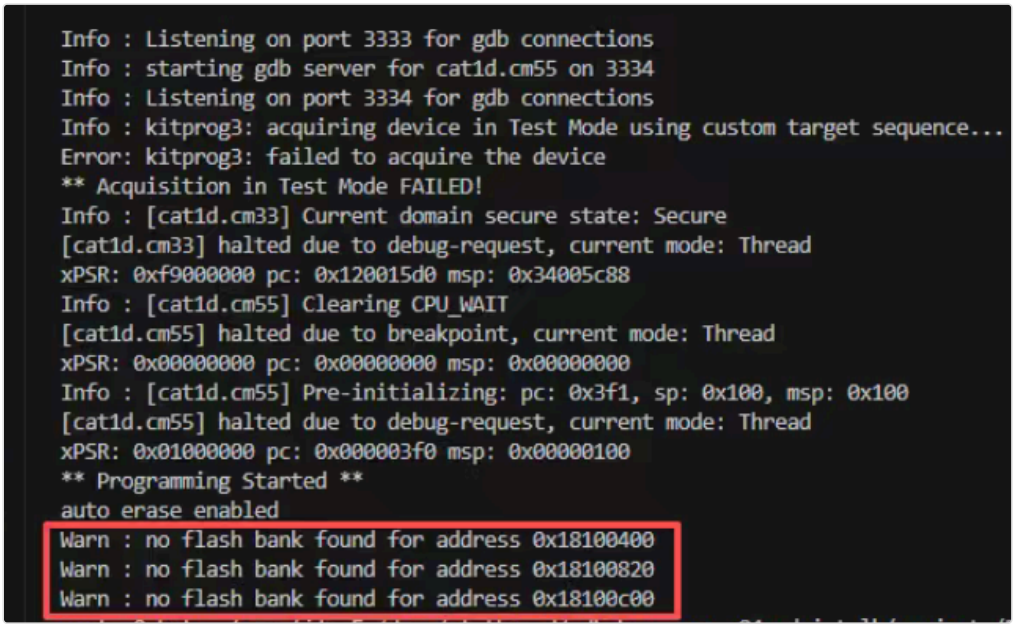
7. 常见问题

7.1. 常见问题解答（FAQ）

本文档收集了用户在使用过程中遇到的常见问题及解决方案。

一、烧录

Q1. 烧录固件出错



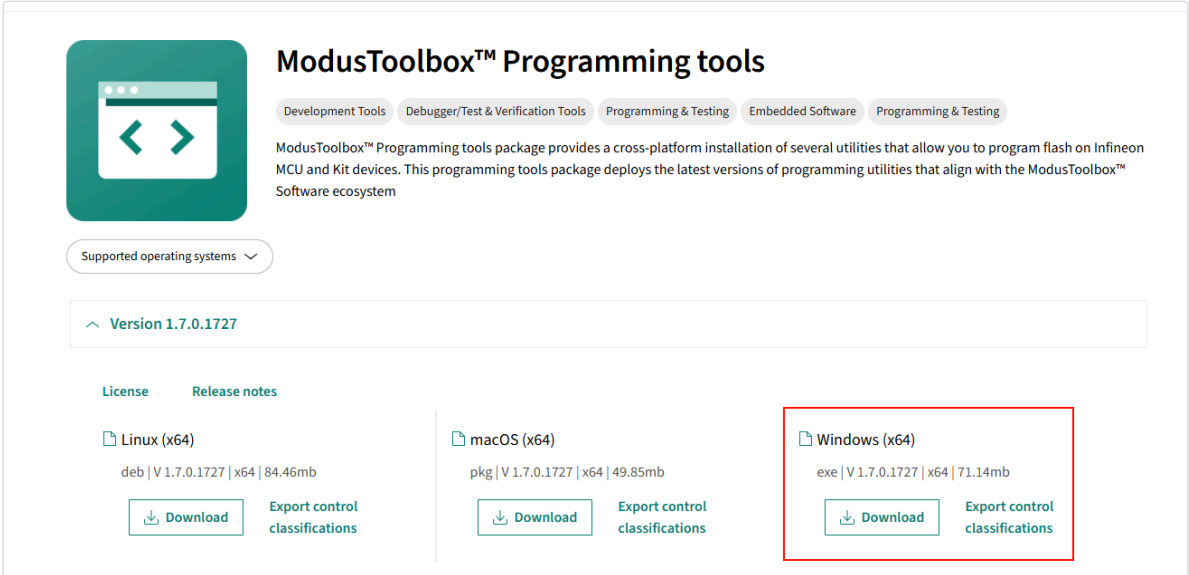
Log Console

```
** RRAM Boot version: 2.0.0.7127
** SE RT Services Base version: 1.0.0.2361
** SE RT Services version: 0.0.0.0
** Extended Boot version: 1.1.0.1700
** Boot Status : CYBOOT_SUCCESS
** Life Cycle  : DEVELOPMENT
**********************************
Info : [cat1d.sys] Examination succeed
Info : [cat1d.sys33] Examination succeed
Info : [cat1d.cm33] Examination succeed
Info : [cat1d.cm55] Clearing CPU_WAIT
Warn : [cat1d.cm55] Failed to clear CPU_WAIT:
Info : [cat1d.cm55] Examination succeed
Info : gdb port disabled
Info : gdb port disabled
Info : starting gdb server for cat1d.cm33 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for cat1d.cm55 on 3334
Info : Listening on port 3334 for gdb connections
Info : SWD DPIDR 0x4c013477
Warn : Connecting DP: stalled AP operation, issuing ABORT
Info : SWD DPIDR 0x4c013477
Error executing event examine-end on target cat1d.cm55:
../scripts/target/infineon/cat1d/func_cat1d_pse84.cfg:18: Error:
in procedure 'program'
in procedure 'ocd_process_reset'
in procedure 'ocd_process_reset_inner' called at file "embedded:startup.tcl", line 1225
at file "../scripts/target/infineon/cat1d/cat1d.cfg", line 81
at file "../scripts/target/infineon/cat1d/func_cat1d_pse84.cfg", line 18
** Acquisition in Test Mode FAILED!
Info : [cat1d.cm33] Current domain secure state: Secure
[cat1d.cm33] halted due to debug-request, current mode: Thread
xPSR: 0xf9000000 pc: 0x120015d0 msp: 0x34005c88
Info : [cat1d.cm55] Clearing CPU_WAIT
Warn : [cat1d.cm55] Failed to clear CPU_WAIT:
** Unable to reset target **
shutdown command invoked
```

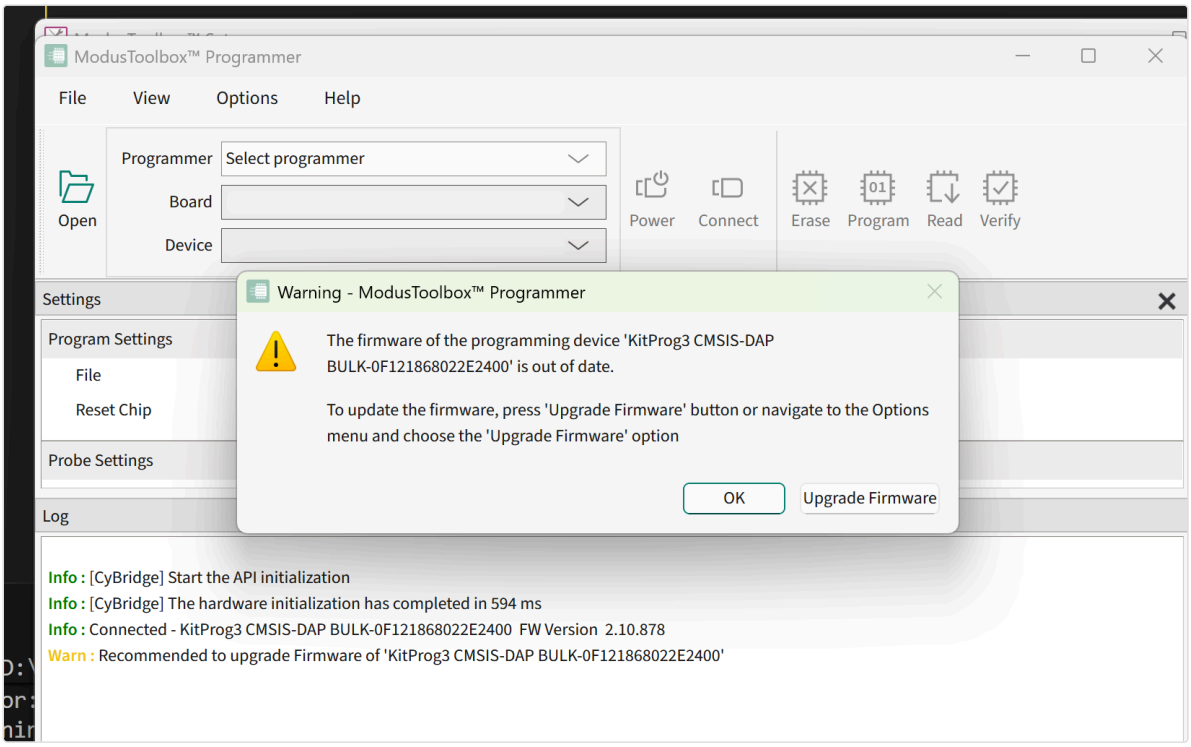
解决方法：

首先下载如下链接的上位机，我们需要进行固件更新：

ModusToolbox™ Programming tools - Infineon Developer Center



如果提示如下的警告，说明你开发板上面的DAP-Link固件需要更新，才可以烧录固件，这时需要点击 **Upgrade Fimeware** 按钮



更新完成后，重新上电开发板，然后即可下载程序了

```
[cat1d.cm55] halted due to breakpoint, current mode: Thread
xPSR: 0x00000000 pc: 0x41e9ac50 msp: 0xb71c5c44
Info : [cat1d.cm55] Pre-initializing: pc: 0x3f1, sp: 0x100, msp: 0x100
[cat1d.cm55] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x000003f0 msp: 0x00000100
** Programming Started **
[ 50%] [################                ] [ Erasing     ]
[100%] [################################] [ Erasing     ]
[ 12%] [###                             ] [ Programming ]
[ 14%] [####                            ] [ Programming ]
[ 18%] [#####                           ] [ Programming ]
[ 25%] [########                        ] [ Programming ]
[ 30%] [#########                       ] [ Programming ]
[ 37%] [###########                     ] [ Programming ]
[ 38%] [############                    ] [ Programming ]
[ 44%] [##############                  ] [ Programming ]
[ 50%] [################                ] [ Programming ]
[ 58%] [##################              ] [ Programming ]
[ 63%] [####################            ] [ Programming ]
[ 71%] [######################          ] [ Programming ]
[ 76%] [########################        ] [ Programming ]
[ 85%] [##########################      ] [ Programming ]
[ 89%] [############################    ] [ Programming ]
[ 99%] [##############################  ] [ Programming ]
[100%] [################################] [ Programming ]
** Programming Finished **
shutdown command invoked
```

Q2. 为什么烧录了Edgi_Talk_M55_XXX工程后，没有现象？

解决方法：

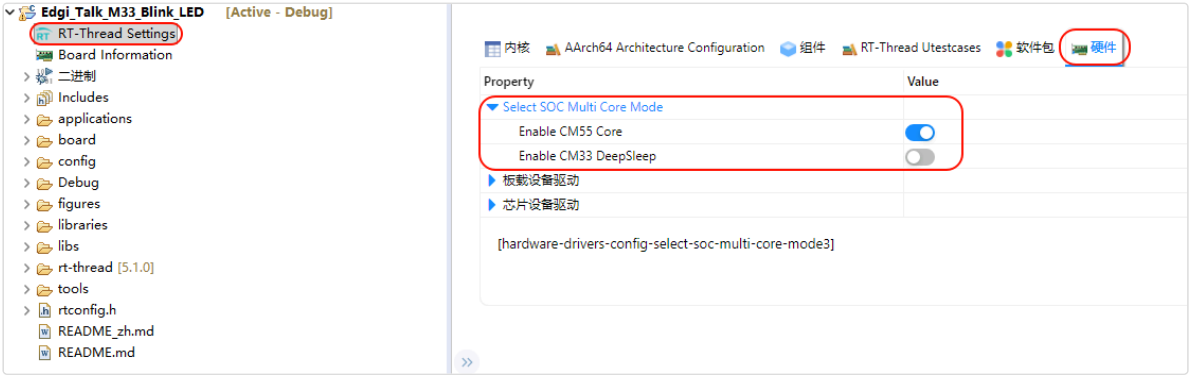
芯片启动顺序如下：

```
+------------------+
|   Secure M33     |
|   (安全内核启动)  |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (非安全核启动)  |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  (应用处理器启动)  |
+-------------------+
```

⚠️ 请严格按照以上顺序烧写固件，否则系统可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- 若要开启 M55，需要在 **M33 工程** 中打开配置：

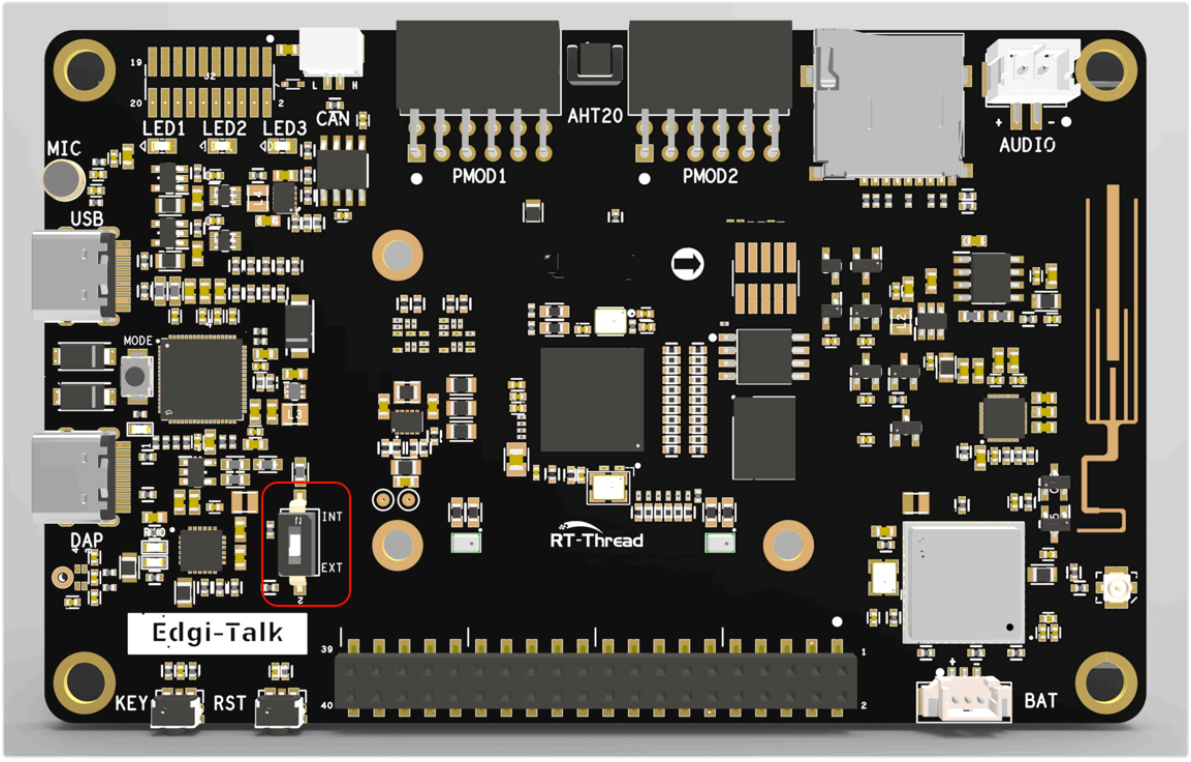
```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```



Q3.烧录固件出错

```
Info : kitprog3: acquiring device in Test Mode using custom target sequence...
Error: kitprog3: failed to acquire the device
Info : clock speed 4000 kHz
Info : SWD DPIDR 0x4c013477
** Target is not in Test Mode
**************************************
** Silicon: 0xED94, Family: 0x115, Rev.: 0x21 (B0)
** Detected Device: PSE846GPS2DBZC4A
** SROM Boot version: 2.0.0.6022
** RRAM Boot version: 2.0.0.7127
** SE RT Services Base version: 1.0.0.2361
** SE RT Services version: 0.0.0.0
** Extended Boot version: 1.1.0.1700
** Boot Status : CYBOOT_SUCCESS
** Life Cycle  : DEVELOPMENT
**************************************
Info : [cat1d.sys] Examination succeed
Info : [cat1d.sys33] Examination succeed
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Error: [cat1d.cm33] Examination failed
Warn : target cat1d.cm33 examination failed
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Error: [cat1d.cm55] Examination failed
Warn : target cat1d.cm55 examination failed
Info : gdb port disabled
Info : gdb port disabled
Info : starting gdb server for cat1d.cm33 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for cat1d.cm55 on 3334
Info : Listening on port 3334 for gdb connections
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Info : kitprog3: acquiring device in Test Mode using custom target sequence...
Error: kitprog3: failed to acquire the device
** Acquisition in Test Mode FAILED!
Info : SWD DPIDR 0x4c013477
Info : SWD DPIDR 0x4c013477
Info : [cat1d.cm33] AP write error, reset will not halt
Info : SWD DPIDR 0x4c013477
Info : SWD DPIDR 0x4c013477

Info : cat1d.dap: powering down debug domain...
Info : SWD DPIDR 0x4c013477
```

解决方法：



1. 将 BOOT 配置切换至 **INT**
2. 重启开发板
3. 下载程序
4. 将 BOOT 配置切换至 **EXT**
5. 再次重启开发板

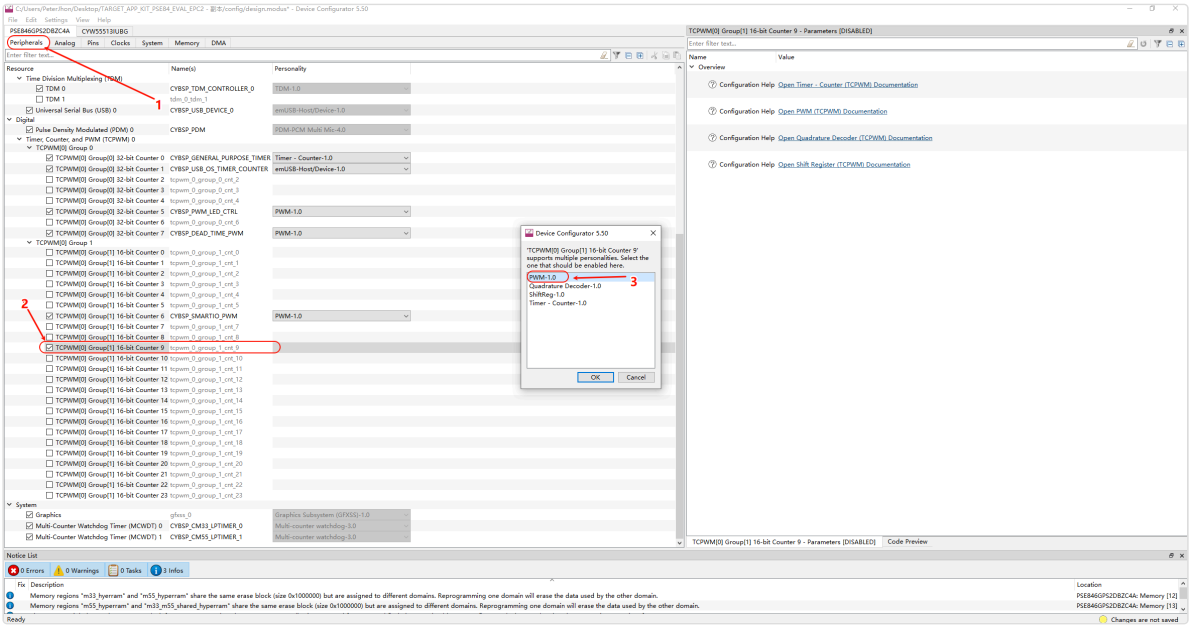
7.2. PWM Device Configurator 配置指南

打开配置文件

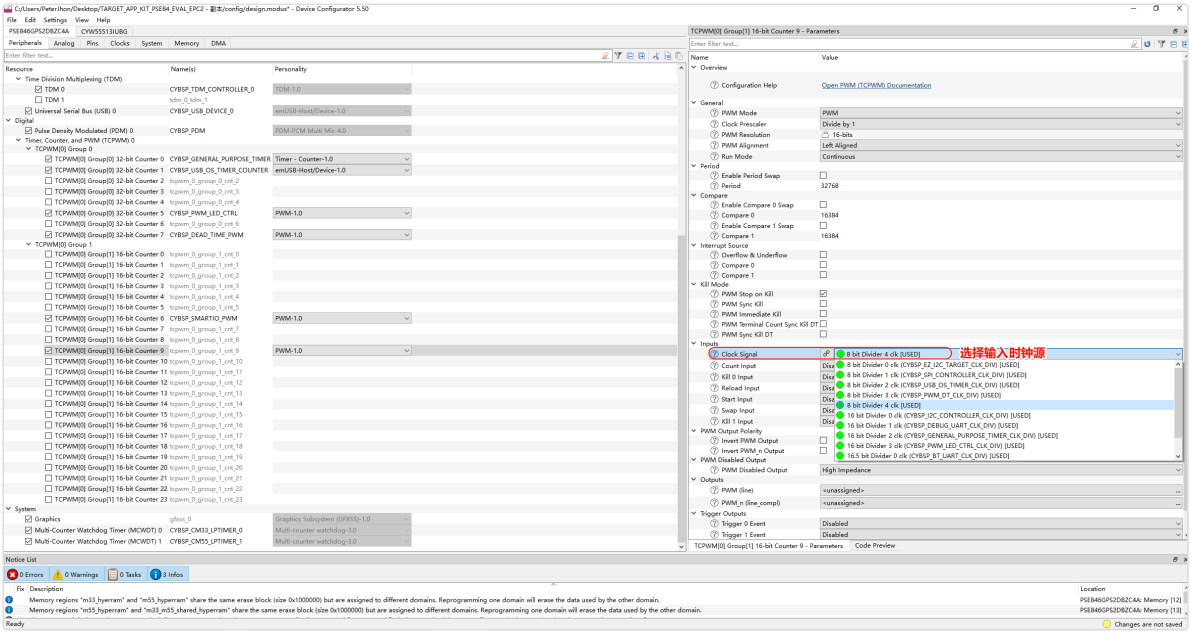
请使用以下工具打开配置文件：

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

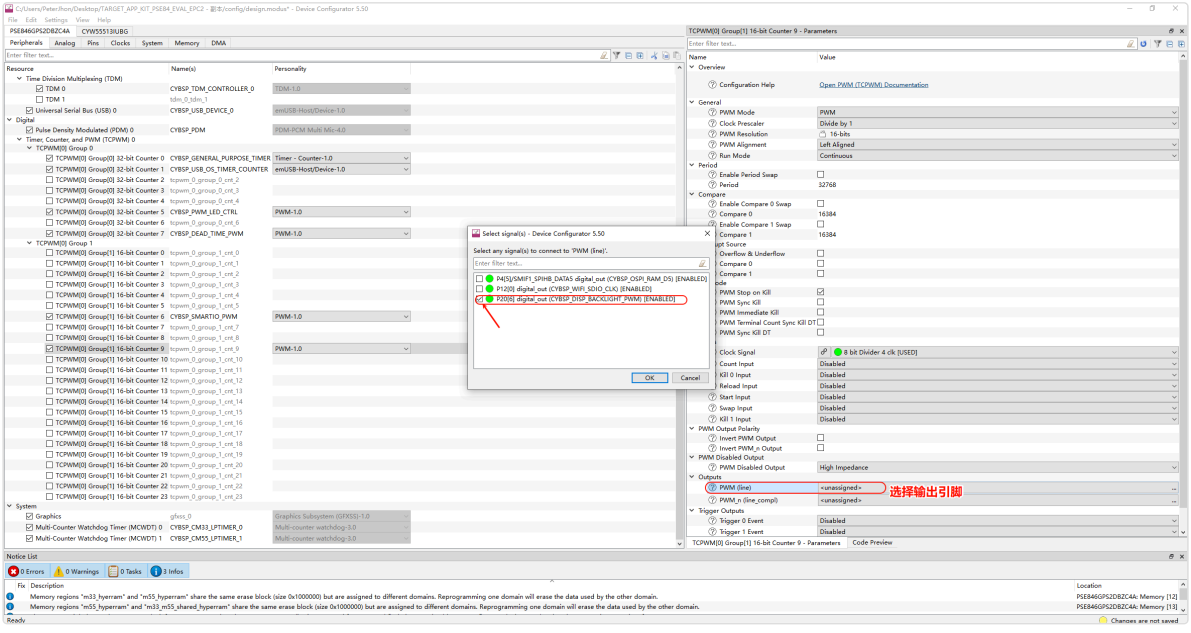
1. 配置 TCPWM 为 PWM 模式



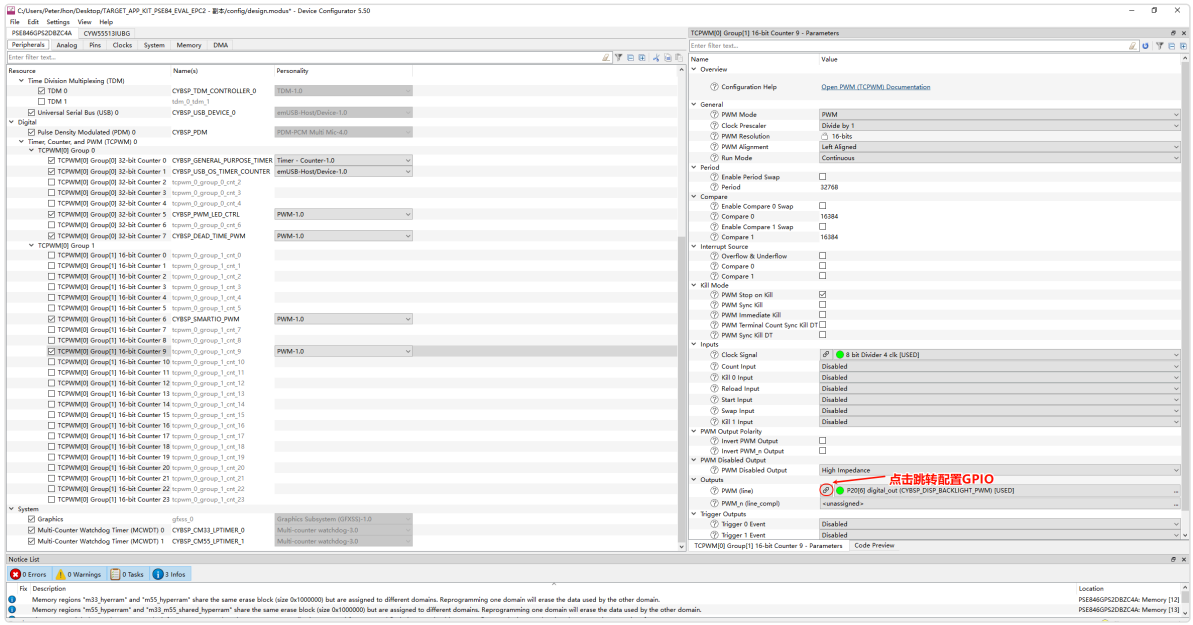
2. 选择输入时钟源

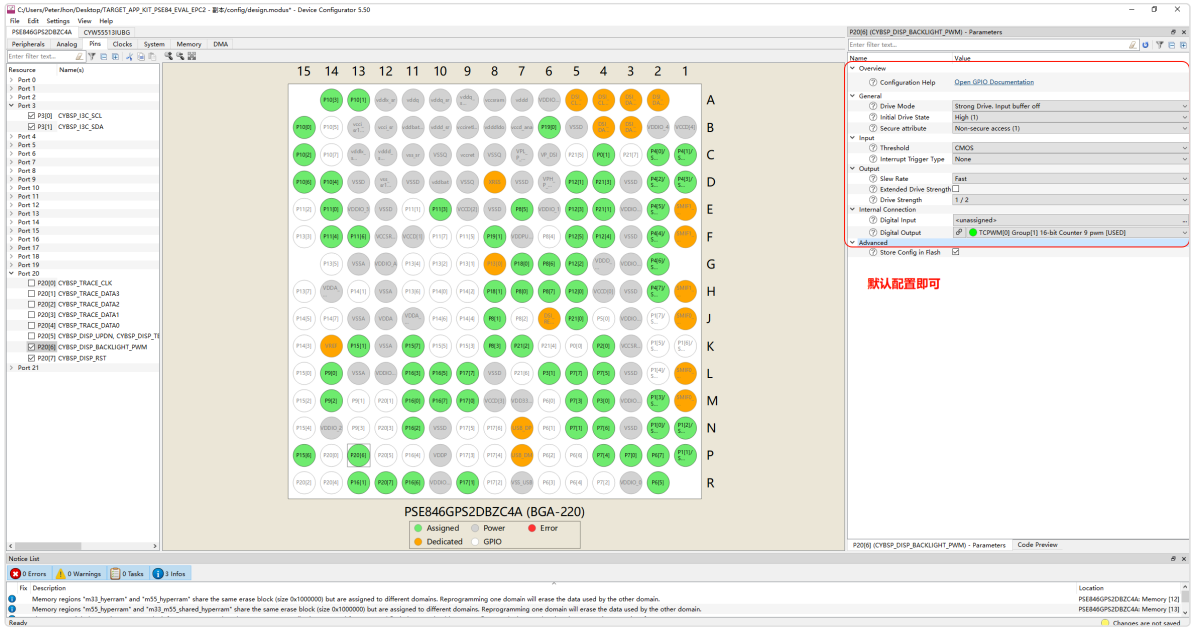


3. 设置 PWM 输出引脚



4. 设置输入引脚模式





保存配置

完成全部配置后，按 `Ctrl + S` 保存即可。